

Université Hassan II de
Casablanca
Faculté des Sciences & Techniques
Mohammedia

Licence Informatique, Réseaux & Multimédia

Traitement des Données Multimodales

Outils de Traitement

Guide Complet des Outils Python pour le Traitement
des Données Tabulaires, Textuelles, Visuelles, Audio
et Graphiques

Réalisé par :
Prof. Abdellah ADIB

May 3, 2026

Contents

1	Introduction	5
1.1	Contexte et Objectifs	5
1.2	Les 5 Classes d'Outils	5
1.3	Le Pipeline de Traitement Multimodal	5
2	NumPy et Pandas – Données Tabulaires	7
2.1	Principe Fondamental	7
2.1.1	Architecture du Pipeline Tabulaire	8
2.1.2	Installation	8
2.2	Manipulation avec Pandas	8
2.2.1	Chargement et Inspection	8
2.2.2	Nettoyage des Données	9
2.2.3	Agrégation et Fusion	11
2.3	Fonctions Pandas essentielles	12
2.3.1	Création de DataFrames et Series	12
2.3.2	Exploration et Informations	12
2.3.3	Sélection et Indexation	13
2.3.4	Nettoyage des Données	13
2.3.5	Manipulation et Transformation	14
2.3.6	Statistiques et Agrégations	14
2.3.7	Regroupement (GroupBy)	15
2.3.8	Fusion et Jointure	15
2.3.9	Pivot et Reshape	15
2.3.10	Séries Temporelles	16
2.3.11	Manipulation de Texte (str)	16
2.4	Calcul Vectorisé avec NumPy	16
2.4.1	Normalisation et Standardisation	16
2.4.2	Opérations Matricielles et Vectorisation	18
2.5	Fonctions NumPy les plus utilisées	19
2.5.1	Création de tableaux	19
2.5.2	Mathématiques de base	19
2.5.3	Trigonométrie	20
2.5.4	Statistiques	20
2.5.5	Manipulation de tableaux	21
2.5.6	Algèbre linéaire (np.linalg)	21
2.5.7	Génération aléatoire (np.random)	21
2.5.8	Comparaisons et Logique	22
2.5.9	Informations sur les tableaux	22
2.6	Comparaison des Outils Tabulaires	23
2.7	Bonnes Pratiques	23

3	NLTK – Traitement du Texte	24
3.1	Principe Fondamental	24
3.2	Architecture du Pipeline TAL	24
3.3	Fonctions NLTK les plus utilisées	24
3.3.1	Installation et Configuration	24
3.3.2	Tokenisation	25
3.3.3	Stopwords et Nettoyage	26
3.3.4	Racinisation (Stemming)	27
3.3.5	Lemmatisation	28
3.3.6	Étiquetage Grammatical (POS Tagging)	29
3.3.7	Fréquences et N-grammes	30
3.3.8	Corpus et Ressources Intégrés	31
3.3.9	Vectorisation et Représentation	32
3.3.10	Reconnaissance d'Entités Nommées (NER)	33
3.3.11	Analyse de Sentiment et Classification	34
3.3.12	Expressions Régulières et Utilitaires	35
3.4	Comparaison des Outils TAL	36
3.5	Bonnes Pratiques	36
4	OpenCV – Traitement des Images	37
4.1	Principe Fondamental	37
4.1.1	Architecture du Pipeline Vision	37
4.1.2	Installation	37
4.2	Lecture, Écriture et Conversion	38
4.2.1	Chargement et Inspection	38
4.2.2	Conversion de l'Espace Couleur	39
4.3	Filtrage et Preprocessing	40
4.3.1	Réduction du Bruit et Lissage	40
4.3.2	Détection de Contours	41
4.4	Détection de Visages par Haar Cascades	43
4.5	Fonctions NLTK les plus utilisées	44
4.5.1	Installation et Configuration	44
4.5.2	Lecture, Écriture et Affichage	45
4.5.3	Espaces Colorimétriques	46
4.5.4	Transformations Géométriques	47
4.5.5	Filtrage et Lissage	49
4.5.6	Détection de Contours et Gradients	50
4.5.7	Seuillage et Binarisation	51
4.5.8	Morphologie Mathématique	52
4.5.9	Détection de Contours et Formes	53
4.5.10	Dessin et Annotations	54
4.5.11	Détection d'Objets et Reconnaissance	55
4.5.12	Traitement Vidéo et Webcam	56
4.5.13	Histogrammes et Analyse Statistique	58
4.6	Comparaison des Outils de Vision	59
4.7	Bonnes Pratiques	59
5	Pydub – Traitement Audio	61
5.1	Principe Fondamental	61

5.1.1	Architecture de Pydub	61
5.1.2	Installation	61
5.2	Chargement et Inspection	62
5.3	Découpe et Concaténation	63
5.4	Normalisation et Conversion de Format	65
5.5	Fonctions Pydub les plus utilisées	67
5.5.1	Installation et Configuration	67
5.5.2	Chargement et Sauvegarde	68
5.5.3	Propriétés du Signal Audio	69
5.5.4	Découpage Temporel et Slicing	70
5.5.5	Concaténation, Mixage et Overlay	71
5.5.6	Volume et Normalisation	72
5.5.7	Fondus et Effets Audio	73
5.5.8	Détection de Silence et Segmentation	75
5.5.9	Conversion de Formats et Paramètres	76
5.5.10	Lecture et Reproduction Audio	77
5.5.11	Génération Audio et Tonalités	78
5.5.12	Accès aux Données Brutes et Intégration NumPy	79
5.6	Comparaison des Outils Audio	81
5.7	Bonnes Pratiques	81
6	Librosa – Analyse Audio Avancée	82
6.1	Principe Fondamental	82
6.1.1	Architecture de Librosa	82
6.1.2	Installation	83
6.2	Chargement et Inspection	83
6.3	Analyse Spectrale (STFT, Mel, MFCC)	84
6.4	Tempo, Beats et Caractéristiques Musicales	86
6.5	Visualisation et Export	87
6.6	Fonctions Librosa les plus utilisées	90
6.6.1	Installation et Configuration	90
6.6.2	Chargement et Sauvegarde	91
6.6.3	Représentation du Signal et Conversions	92
6.6.4	Transformée de Fourier Court Terme (STFT)	93
6.6.5	Mel-Spectrogramme et MFCC	94
6.6.6	Caractéristiques Spectrales	95
6.6.7	Tempo, Beats et Rythme	96
6.6.8	Analyse Harmonique et Chroma	97
6.6.9	Effets Audio et Transformations	98
6.6.10	Visualisation des Spectrogrammes	100
6.7	Comparaison Pydub vs Librosa	101
6.8	Bonnes Pratiques	102
7	Matplotlib – Visualisation Multimodale	103
7.1	Principe Fondamental	103
7.1.1	Architecture de Matplotlib	103
7.1.2	Installation	103
7.2	Graphiques de Base	103
7.2.1	Configuration Globale	103

7.2.2	Visualisation de Donnees Tabulaires	104
7.3	Visualisation des Donnees Image	106
7.4	Visualisation des Donnees Audio	108
7.5	Dashboard Multimodal Integratif	109
7.6	Bonnes Pratiques	112
8	Pipeline Integratif Multimodal	113
8.1	Architecture Generale	113
8.2	Implementation Complete du Pipeline	113
9	Conclusion	117
9.1	Tableau Comparatif des 5 Classes d'Outils	117
9.2	Bonnes Pratiques Generales	117
9.3	Arbre de Decision	118
10	Bibliographie et Ressources	119
10.1	Ouvrages de Reference	119
10.2	Documentation Officielle	119
10.3	Tutoriels Recommandes	119
10.4	Jeux de Donnees pour Pratiquer	120
10.5	Aspects Legaux et Ethiques	120

Chapitre 1 Introduction

1.1 Contexte et Objectifs

Le traitement des données multimodales est au cœur des systèmes d'intelligence artificielle modernes. Après avoir acquis des données brutes (images, sons, textes, tableaux), l'étape suivante – et tout aussi fondamentale – consiste à les transformer, les nettoyer et les préparer pour les modèles d'apprentissage automatique. Ce rapport se concentre sur les **outils de traitement** : les bibliothèques Python qui permettent de manipuler chaque type de donnée de manière efficace et reproductible.

Chaque outil présenté dans ce guide a été sélectionné pour sa maturité, sa documentation, son adoption industrielle et sa complémentarité avec les autres. Ensemble, ils couvrent l'intégralité du spectre multimodal.

1.2 Les 5 Classes d'Outils

Cinq classes d'outils couvrent la totalité des modalités rencontrées en pratique :

1. **NumPy + Pandas** : Données tabulaires – calcul vectorisé et manipulation de DataFrames structures.
2. **NLTK** : Traitement du langage naturel – tokenisation, filtrage, racinisation, extraction de caractéristiques textuelles.
3. **OpenCV** : Traitement des images et de la vidéo – lecture, filtrage, détection, annotation et transformations géométriques.
4. **Pydub** : Traitement audio haut niveau – découpe, concaténation, normalisation, export multi-format.
5. **Matplotlib** : Visualisation multimodale – graphiques, dashboards, rapports visuels combinés.

1.3 Le Pipeline de Traitement Multimodal

Un pipeline de traitement multimodal complet suit une logique commune indépendamment de la modalité traitée :

1. **Chargement** : Lire la donnée brute depuis le disque, le réseau ou la mémoire.
2. **Inspection** : Vérifier la forme, le type, les valeurs manquantes.
3. **Nettoyage** : Supprimer les artefacts, corriger les types, gérer les valeurs aberrantes.
4. **Normalisation** : Ramener les données dans une plage standard (0–1, z-score).
5. **Transformation** : Extraire des caractéristiques (features) pertinentes.

6. **Export** : Sauvegarder le résultat dans un format exploitable par le modèle.

Conseil

Choisissez la classe d'outils en fonction de la **nature** de la donnée :

- Donnee = tableau CSV / Excel ? → NumPy + Pandas
- Donnee = texte, documents, sous-titres ? → NLTK
- Donnee = image, video, frame ? → OpenCV
- Donnee = son, parole, musique ? → Pydub
- Donnee = visualisation, rapport ? → Matplotlib

Chapitre 2 NumPy et Pandas – Données Tabulaires

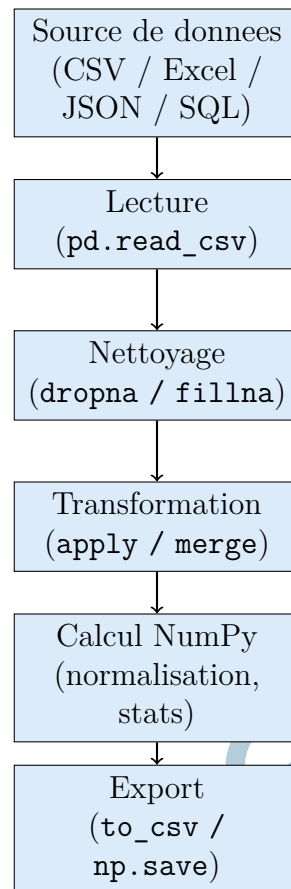
2.1 Principe Fondamental

Les données tabulaires sont omniprésentes dans les projets multimodaux : métadonnées des fichiers médias, annotations de datasets, résultats de mesures audio, caractéristiques extraites d'images. **NumPy** et **Pandas** constituent le socle indispensable pour les manipuler efficacement.

NumPy (Numerical Python) est la bibliothèque de calcul scientifique de référence en Python. Elle fournit le type `ndarray`, un tableau N-dimensionnel homogène sur lequel toutes les opérations sont vectorisées (sans boucle Python explicite), ce qui garantit des performances proches du C.

Pandas s'appuie sur NumPy pour offrir des structures de données hétérogènes (`DataFrame`, `Series`) dotées d'un index et de méthodes de haut niveau pour le nettoyage, la fusion et l'agrégation de données.

2.1.1 Architecture du Pipeline Tabulaire



2.1.2 Installation

Listing 2.1: Installation de NumPy et Pandas

```
pip install numpy pandas openpyxl
# openpyxl est requis pour lire les fichiers Excel (.xlsx)
```

2.2 Manipulation avec Pandas

2.2.1 Chargement et Inspection

Listing 2.2: Chargement et inspection d'un catalogue media

```
import pandas as pd
import numpy as np

def charger_et_inspecter(chemin_csv: str) -> pd.DataFrame:
    """
    Charge un fichier CSV et affiche un rapport d'inspection
    complet.

    Args:
        chemin_csv: Chemin vers le fichier CSV (ex:
                    'media_catalog.csv')
```

```

Returns:
    DataFrame charge et pret pour le nettoyage

Exemple CSV attendu : titre, duree_s, taille_ko,
    categorie, format
"""
# Chargement avec gestion de l'encodage
df = pd.read_csv(chemin_csv, encoding='utf-8', sep=',')

print("=== Inspection du DataFrame ===")
print(f"Dimensions      : {df.shape[0]} lignes x
      {df.shape[1]} colonnes")
print(f"Colonnes         : {list(df.columns)}")
print(f"\nTypes des colonnes :")
print(df.dtypes)
print(f"\nValeurs manquantes par colonne :")
print(df.isnull().sum())
print(f"\nStatistiques descriptives :")
print(df.describe())
print(f"\nPremiers exemples :")
print(df.head(3))

return df

# Exemple d'utilisation
df = charger_et_inspecter("media_catalog.csv")

```

2.2.2 Nettoyage des Données

Le nettoyage est l'étape la plus chronophage en pratique. Elle comprend la gestion des valeurs manquantes, la correction des types incorrects, la suppression des doublons et le traitement des valeurs aberrantes.

Listing 2.3: Pipeline de nettoyage complet d'un DataFrame

```

import pandas as pd
import numpy as np

def nettoyer_dataframe(df: pd.DataFrame) -> pd.DataFrame:
    """
    Applique un pipeline de nettoyage standard sur un
    DataFrame media.

    Etapes :
        1. Suppression des doublons complets
        2. Correction des types numeriques
        3. Gestion des valeurs manquantes
        4. Suppression des valeurs aberrantes (IQR)
        5. Standardisation des chaines de caracteres

    Returns:
    """

```

```

    DataFrame nettoye
"""
df_clean = df.copy()
n_initial = len(df_clean)

# 1. Suppression des lignes complètement dupliquées
df_clean = df_clean.drop_duplicates()
print(f"Doublons supprimés : {n_initial - len(df_clean)}")

# 2. Forcer les colonnes numériques
cols_numeriques = ['duree_s', 'taille_ko']
for col in cols_numeriques:
    if col in df_clean.columns:
        df_clean[col] = pd.to_numeric(df_clean[col],
                                       errors='coerce')

# 3. Gestion des valeurs manquantes
# Colonnes critiques : on supprime si manquantes
cols_critiques = ['titre', 'categorie']
df_clean = df_clean.dropna(subset=cols_critiques)

# Colonnes numériques : on remplace par la médiane
for col in cols_numeriques:
    if col in df_clean.columns:
        mediane = df_clean[col].median()
        df_clean[col] = df_clean[col].fillna(mediane)
        print(f" {col} : {df_clean[col].isna().sum()}
              manquants remplacés par {mediane:.2f}")

# 4. Suppression des valeurs aberrantes par la méthode IQR
if 'duree_s' in df_clean.columns:
    Q1 = df_clean['duree_s'].quantile(0.25)
    Q3 = df_clean['duree_s'].quantile(0.75)
    IQR = Q3 - Q1
    masque = (df_clean['duree_s'] >= Q1 - 1.5 * IQR) & \
              (df_clean['duree_s'] <= Q3 + 1.5 * IQR)
    n_avant = len(df_clean)
    df_clean = df_clean[masque]
    print(f"Aberrants supprimés (IQR) : {n_avant -
          len(df_clean)}")

# 5. Standardisation des chaînes
if 'categorie' in df_clean.columns:
    df_clean['categorie'] = (df_clean['categorie']
                             .str.strip()
                             .str.lower()
                             .str.replace(' ', '_'))

print(f"\nDataFrame final : {len(df_clean)} lignes
      ({n_initial} au départ)")
return df_clean.reset_index(drop=True)

```

```
# Exemple
df_propre = nettoyer_dataframe(df)
```

2.2.3 Agrégation et Fusion

Listing 2.4: Agregation par categorie et fusion de DataFrames

```
import pandas as pd

def analyser_par_categorie(df: pd.DataFrame) -> pd.DataFrame:
    """
    Calcule des statistiques agregées par categorie de media.

    Returns:
        DataFrame avec une ligne par categorie et les
        statistiques associees
    """
    stats = df.groupby('categorie').agg(
        nb_fichiers    = ('titre',    'count'),
        duree_totale   = ('duree_s',   'sum'),
        duree_moyenne  = ('duree_s',   'mean'),
        duree_max      = ('duree_s',   'max'),
        taille_moy_ko  = ('taille_ko', 'mean')
    ).round(2)

    stats['duree_totale_min'] = (stats['duree_totale'] /
                                60).round(1)

    print("=== Statistiques par categorie ===")
    print(stats.to_string())
    return stats

def fusionner_annotations(
    df_media: pd.DataFrame,
    df_annot: pd.DataFrame,
    cle: str = 'titre'
) -> pd.DataFrame:
    """
    Fusionne un catalogue media avec des annotations externes.

    Args:
        df_media: DataFrame principal (catalogue)
        df_annot: DataFrame des annotations (ex: labels,
            scores)
        cle: Colonne commune servant de cle de jointure

    Returns:
        DataFrame fusionne (jointure gauche)
    """
    df_fusion = pd.merge(
```

```

    df_media,
    df_annot,
    on=cle,
    how='left',
    suffixes=('_media', '_annot')
)

# Rapport de fusion
n_matches = df_fusion[df_fusion['label'].notna()].shape[0]
print(f"Lignes fusionnees avec annotation :
      {n_matches}/{len(df_fusion)}")

return df_fusion

# Exemple
stats_cat = analyser_par_categorie(df_propre)

```

2.3 Fonctions Pandas essentielles

2.3.1 Création de DataFrames et Series

Fonction	Fonctionnalité	Instruction
DataFrame()	Créer un DataFrame depuis un dict	pd.DataFrame({'A': [1,2]})
Series()	Créer une Series	pd.Series([1, 2, 3])
read_csv()	Lire un fichier CSV	pd.read_csv('file.csv')
read_excel()	Lire un fichier Excel	pd.read_excel('file.xlsx')
read_json()	Lire un fichier JSON	pd.read_json('file.json')
read_sql()	Lire depuis une base de données SQL	pd.read_sql(query, conn)
to_csv()	Exporter vers un fichier CSV	df.to_csv('file.csv')
to_excel()	Exporter vers un fichier Excel	df.to_excel('file.xlsx')
to_json()	Exporter vers un fichier JSON	df.to_json('file.json')
to_sql()	Exporter vers une base SQL	df.to_sql('table', conn)

2.3.2 Exploration et Informations

Fonction	Fonctionnalité	Instruction
head()	Afficher les n premières lignes	df.head(5)
tail()	Afficher les n dernières lignes	df.tail(5)
info()	Informations générales du DataFrame	df.info()
describe()	Statistiques descriptives	df.describe()
shape	Dimensions du DataFrame	df.shape
dtypes	Types des colonnes	df.dtypes

Fonction	Fonctionnalité	Instruction
<code>columns</code>	Noms des colonnes	<code>df.columns</code>
<code>index</code>	Index du DataFrame	<code>df.index</code>
<code>values</code>	Valeurs sous forme de tableau NumPy	<code>df.values</code>
<code>size</code>	Nombre total d'éléments	<code>df.size</code>
<code>ndim</code>	Nombre de dimensions	<code>df.ndim</code>
<code>memory_usage()</code>	Mémoire utilisée par colonne	<code>df.memory_usage()</code>

2.3.3 Sélection et Indexation

Fonction	Fonctionnalité	Instruction
<code>loc[]</code>	Sélection par étiquette	<code>df.loc[0, 'col']</code>
<code>iloc[]</code>	Sélection par position	<code>df.iloc[0, 1]</code>
<code>at[]</code>	Accès à une valeur par étiquette	<code>df.at[0, 'col']</code>
<code>iat[]</code>	Accès à une valeur par position	<code>df.iat[0, 1]</code>
<code>filter()</code>	Filtrer colonnes/lignes	<code>df.filter(items=['A', 'B'])</code>
<code>query()</code>	Filtrer avec une expression	<code>df.query('A > 5')</code>
<code>isin()</code>	Filtrer par valeurs	<code>df[df['A'].isin([1,2])]</code>
<code>between()</code>	Filtrer dans un intervalle	<code>df[df['A'].between(1,5)]</code>
<code>xs()</code>	Sélection multi-index	<code>df.xs('key', level=0)</code>

2.3.4 Nettoyage des Données

Fonction	Fonctionnalité	Instruction
<code>dropna()</code>	Supprimer les valeurs manquantes	<code>df.dropna()</code>
<code>fillna()</code>	Remplir les valeurs manquantes	<code>df.fillna(0)</code>
<code>isna()</code>	Détecter les valeurs manquantes	<code>df.isna()</code>
<code>notna()</code>	Détecter les valeurs non manquantes	<code>df.notna()</code>
<code>drop_duplicates()</code>	Supprimer les doublons	<code>df.drop_duplicates()</code>
<code>duplicated()</code>	Détecter les doublons	<code>df.duplicated()</code>
<code>replace()</code>	Remplacer des valeurs	<code>df.replace(1, 99)</code>
<code>astype()</code>	Convertir le type d'une colonne	<code>df['A'].astype(float)</code>
<code>rename()</code>	Renommer des colonnes	<code>df.rename(columns={'A': 'B'})</code>
<code>drop()</code>	Supprimer colonnes ou lignes	<code>df.drop('col', axis=1)</code>

Fonction	Fonctionnalité	Instruction
<code>clip()</code>	Limiter les valeurs	<code>df.clip(lower=0, upper=10)</code>
<code>strip()</code>	Supprimer les espaces (texte)	<code>df['A'].str.strip()</code>

2.3.5 Manipulation et Transformation

Fonction	Fonctionnalité	Instruction
<code>apply()</code>	Appliquer une fonction	<code>df['A'].apply(np.sqrt)</code>
<code>map()</code>	Mapper des valeurs (Series)	<code>df['A'].map({1:'a'})</code>
<code>applymap()</code>	Appliquer sur chaque cellule	<code>df.applymap(str)</code>
<code>assign()</code>	Ajouter/modifier une colonne	<code>df.assign(C=df['A']+1)</code>
<code>insert()</code>	Insérer une colonne	<code>df.insert(1,'col',[1,2])</code>
<code>pop()</code>	Extraire et supprimer une colonne	<code>df.pop('col')</code>
<code>sort_values()</code>	Trier par valeurs	<code>df.sort_values('A')</code>
<code>sort_index()</code>	Trier par index	<code>df.sort_index()</code>
<code>reset_index()</code>	Réinitialiser l'index	<code>df.reset_index()</code>
<code>set_index()</code>	Définir une colonne comme index	<code>df.set_index('A')</code>
<code>reindex()</code>	Réindexer le DataFrame	<code>df.reindex([0,1,2])</code>
<code>transpose()</code>	Transposer le DataFrame	<code>df.T</code>
<code>copy()</code>	Copier le DataFrame	<code>df.copy()</code>

2.3.6 Statistiques et Agrégations

Fonction	Fonctionnalité	Instruction
<code>sum()</code>	Somme des valeurs	<code>df.sum()</code>
<code>mean()</code>	Moyenne	<code>df.mean()</code>
<code>median()</code>	Médiane	<code>df.median()</code>
<code>min()</code>	Valeur minimale	<code>df.min()</code>
<code>max()</code>	Valeur maximale	<code>df.max()</code>
<code>std()</code>	Écart-type	<code>df.std()</code>
<code>var()</code>	Variance	<code>df.var()</code>
<code>count()</code>	Nombre de valeurs non nulles	<code>df.count()</code>
<code>cumsum()</code>	Somme cumulée	<code>df.cumsum()</code>
<code>cumprod()</code>	Produit cumulé	<code>df.cumprod()</code>
<code>cummax()</code>	Maximum cumulé	<code>df.cummax()</code>
<code>cummin()</code>	Minimum cumulé	<code>df.cummin()</code>
<code>quantile()</code>	Quantile	<code>df.quantile(0.25)</code>
<code>corr()</code>	Matrice de corrélation	<code>df.corr()</code>
<code>cov()</code>	Matrice de covariance	<code>df.cov()</code>
<code>value_counts()</code>	Fréquence des valeurs uniques	<code>df['A'].value_counts()</code>

Fonction	Fonctionnalité	Instruction
<code>nunique()</code>	Nombre de valeurs uniques	<code>df.nunique()</code>
<code>unique()</code>	Valeurs uniques	<code>df['A'].unique()</code>
<code>agg()</code>	Appliquer plusieurs agrégations	<code>df.agg(['sum', 'mean'])</code>

2.3.7 Regroupement (GroupBy)

Fonction	Fonctionnalité	Instruction
<code>groupby()</code>	Regrouper par colonne	<code>df.groupby('A')</code>
<code>.sum()</code>	Somme par groupe	<code>df.groupby('A').sum()</code>
<code>.mean()</code>	Moyenne par groupe	<code>df.groupby('A').mean()</code>
<code>.count()</code>	Comptage par groupe	<code>df.groupby('A').count()</code>
<code>.agg()</code>	Agrégations multiples par groupe	<code>df.groupby('A').agg('sum')</code>
<code>.apply()</code>	Appliquer une fonction par groupe	<code>df.groupby('A').apply(f)</code>
<code>.transform()</code>	Transformer sans réduire	<code>df.groupby('A').transform('mean')</code>
<code>.filter()</code>	Filtrer des groupes	<code>df.groupby('A').filter(f)</code>
<code>.size()</code>	Taille de chaque groupe	<code>df.groupby('A').size()</code>
<code>.first()</code>	Premier élément de chaque groupe	<code>df.groupby('A').first()</code>
<code>.last()</code>	Dernier élément de chaque groupe	<code>df.groupby('A').last()</code>

2.3.8 Fusion et Jointure

Fonction	Fonctionnalité	Instruction
<code>merge()</code>	Fusionner deux DataFrames	<code>pd.merge(df1, df2, on='A')</code>
<code>join()</code>	Joindre sur l'index	<code>df1.join(df2)</code>
<code>concat()</code>	Concaténer des DataFrames	<code>pd.concat([df1, df2])</code>
<code>append()</code>	Ajouter des lignes	<code>df1.append(df2)</code>
<code>combine()</code>	Combiner deux DataFrames	<code>df1.combine(df2, func)</code>
<code>combine_first()</code>	Compléter les valeurs manquantes	<code>df1.combine_first(df2)</code>
<code>update()</code>	Mettre à jour avec un autre DataFrame	<code>df1.update(df2)</code>

2.3.9 Pivot et Reshape

Fonction	Fonctionnalité	Instruction
<code>pivot()</code>	Tableau croisé simple	<code>df.pivot('A', 'B', 'C')</code>
<code>pivot_table()</code>	Tableau croisé avec agrégation	<code>pd.pivot_table(df, values='C')</code>
<code>melt()</code>	Transformer colonnes en lignes	<code>pd.melt(df, id_vars=['A'])</code>
<code>stack()</code>	Empiler les colonnes	<code>df.stack()</code>
<code>unstack()</code>	Désempiler les colonnes	<code>df.unstack()</code>

Fonction	Fonctionnalité	Instruction
<code>wide_to_long()</code>	Format large vers long	<code>pd.wide_to_long(df, 'val', i='A', j='B')</code>
<code>explode()</code>	Éclater une liste en plusieurs lignes	<code>df.explode('col')</code>

2.3.10 Séries Temporelles

Fonction	Fonctionnalité	Instruction
<code>to_datetime()</code>	Convertir en datetime	<code>pd.to_datetime(df['A'])</code>
<code>date_range()</code>	Créer une plage de dates	<code>pd.date_range('2024-01-01', periods=5)</code>
<code>resample()</code>	Rééchantillonner une série	<code>df.resample('M').mean()</code>
<code>rolling()</code>	Fenêtre glissante	<code>df.rolling(3).mean()</code>
<code>expanding()</code>	Fenêtre expansive	<code>df.expanding().sum()</code>
<code>shift()</code>	Décaler les valeurs	<code>df.shift(1)</code>
<code>diff()</code>	Différence entre périodes	<code>df.diff(1)</code>
<code>dt.year</code>	Extraire l'année	<code>df['A'].dt.year</code>
<code>dt.month</code>	Extraire le mois	<code>df['A'].dt.month</code>
<code>dt.day</code>	Extraire le jour	<code>df['A'].dt.day</code>
<code>dt.weekday</code>	Jour de la semaine	<code>df['A'].dt.weekday</code>
<code>dt.strftime()</code>	Formater une date en texte	<code>df['A'].dt.strftime('%Y-%m-%d')</code>

2.3.11 Manipulation de Texte (str)

Fonction	Fonctionnalité	Instruction
<code>str.lower()</code>	Mettre en minuscules	<code>df['A'].str.lower()</code>
<code>str.upper()</code>	Mettre en majuscules	<code>df['A'].str.upper()</code>
<code>str.strip()</code>	Supprimer les espaces	<code>df['A'].str.strip()</code>
<code>str.replace()</code>	Remplacer une sous-chaîne	<code>df['A'].str.replace('a', 'b')</code>
<code>str.contains()</code>	Vérifier si contient	<code>df['A'].str.contains('mot')</code>
<code>str.startswith()</code>	Vérifier le début	<code>df['A'].str.startswith('x')</code>
<code>str.endswith()</code>	Vérifier la fin	<code>df['A'].str.endswith('x')</code>
<code>str.split()</code>	Diviser une chaîne	<code>df['A'].str.split(',')</code>
<code>str.len()</code>	Longueur de la chaîne	<code>df['A'].str.len()</code>
<code>str.count()</code>	Compter les occurrences	<code>df['A'].str.count('a')</code>
<code>str.extract()</code>	Extraire avec regex	<code>df['A'].str.extract('(\d+)')</code>

2.4 Calcul Vectorisé avec NumPy

2.4.1 Normalisation et Standardisation

La normalisation est indispensable avant tout apprentissage automatique. NumPy permet de la réaliser de façon vectorisée, sans boucle, sur des millions de valeurs en quelques millisecondes.

Listing 2.5: Normalisation et standardisation avec NumPy

```
import numpy as np
import pandas as pd

def normaliser_features(df: pd.DataFrame, cols: list) ->
    pd.DataFrame:
    """
    Applique differentes strategies de normalisation sur les
    colonnes specifiees.

    Strategies implementees :
    - min-max : ramene dans [0, 1]
    - z-score : centrage-reduction (moyenne=0,
      ecart-type=1)
    - robuste : base sur la mediane et IQR (resistant aux
      aberrants)

    Args:
    df : DataFrame source
    cols: Liste des colonnes numeriques a normaliser

    Returns:
    DataFrame enrichi avec les colonnes normalisees
    """
    df_out = df.copy()

    for col in cols:
        if col not in df_out.columns:
            continue

        valeurs = df_out[col].values.astype(float)

        # Min-Max : [0, 1]
        vmin, vmax = valeurs.min(), valeurs.max()
        if vmax > vmin:
            df_out[f"{col}_minmax"] = (valeurs - vmin) / (vmax
                - vmin)
        else:
            df_out[f"{col}_minmax"] = 0.0

        # Z-score : centrage-reduction
        mu, sigma = valeurs.mean(), valeurs.std()
        if sigma > 0:
            df_out[f"{col}_zscore"] = (valeurs - mu) / sigma
        else:
            df_out[f"{col}_zscore"] = 0.0

        # Robuste : mediane + IQR
        med = np.median(valeurs)
        Q1 = np.percentile(valeurs, 25)
        Q3 = np.percentile(valeurs, 75)
```

```

iqr = Q3 - Q1
if iqr > 0:
    df_out[f"{col}_robust"] = (valeurs - med) / iqr
else:
    df_out[f"{col}_robust"] = 0.0

print(f"{col} -> min-max OK | z-score (mu={mu:.2f},
      s={sigma:.2f}) | robust OK")

return df_out

# Exemple
df_norm = normaliser_features(df_propre, cols=['duree_s',
      'taille_ko'])

```

2.4.2 Opérations Matricielles et Vectorisation

Listing 2.6: Calcul de la matrice de corrélation et similarité cosinus

```

import numpy as np

def calculer_correlation(matrice: np.ndarray) ->
    np.ndarray:
    """
    Calcule la matrice de corrélation de Pearson entre les
    colonnes d'une matrice.

    Args:
    matrice: Array (n_echantillons, n_features)

    Returns:
    Matrice de corrélation (n_features, n_features)
    """
    # Centrage : soustraction de la moyenne par colonne
    matrice_centree = matrice - matrice.mean(axis=0)

    # Matrice de covariance
    covariance = np.dot(matrice_centree.T, matrice_centree) /
        (matrice.shape[0] - 1)

    # Ecart-types
    ecarts = np.sqrt(np.diag(covariance))

    # Matrice de corrélation (normalisation)
    corr = covariance / np.outer(ecarts, ecarts)

    # Clip pour éviter les erreurs numériques
    corr = np.clip(corr, -1.0, 1.0)

    print(f"Matrice de corrélation
          ({corr.shape[0]}x{corr.shape[1]}) calculée.")

```

```

return corr

def similarite_cosinus(vec_a: np.ndarray, vec_b:
    np.ndarray) -> float:
    """
    Calcule la similarite cosinus entre deux vecteurs de
    features.

    Returns:
    Score entre -1 (oppose) et 1 (identiques)
    """
    norme_a = np.linalg.norm(vec_a)
    norme_b = np.linalg.norm(vec_b)

    if norme_a == 0 or norme_b == 0:
        return 0.0

    return float(np.dot(vec_a, vec_b) / (norme_a * norme_b))

# Exemple : matrice de features (100 medias, 5 features)
np.random.seed(42)
features = np.random.randn(100, 5)
corr = calculer_correlation(features)

# Similarite entre deux fichiers media
sim = similarite_cosinus(features[0], features[1])
print(f"Similarite entre media 0 et 1 : {sim:.4f}")

```

2.5 Fonctions NumPy les plus utilisées

2.5.1 Création de tableaux

Fonction	Fonctionnalité	Instruction
array	Créer un tableau depuis une liste	np.array([1, 2, 3])
zeros	Tableau rempli de zéros	np.zeros((3, 4))
ones	Tableau rempli de uns	np.ones((3, 4))
full	Tableau rempli d'une valeur donnée	np.full((3, 4), 7)
eye	Matrice identité	np.eye(3)
arange	Tableau avec plage de valeurs	np.arange(0, 10, 2)
linspace	Valeurs espacées régulièrement	np.linspace(0, 1, 50)
empty	Tableau non initialisé	np.empty((3, 3))
reshape	Redimensionner un tableau	arr.reshape(3, 4)
diag	Créer une matrice diagonale	np.diag([1, 2, 3])

2.5.2 Mathématiques de base

Fonction	Fonctionnalité	Instruction
add	Addition	np.add(a, b)
subtract	Soustraction	np.subtract(a, b)
multiply	Multiplication	np.multiply(a, b)
divide	Division	np.divide(a, b)
power	Puissance	np.power(a, 2)
mod	Modulo	np.mod(a, b)
abs	Valeur absolue	np.abs(arr)
sqrt	Racine carrée	np.sqrt(arr)
exp	Exponentielle	np.exp(arr)
log	Logarithme naturel	np.log(arr)
log2	Logarithme base 2	np.log2(arr)
log10	Logarithme base 10	np.log10(arr)
floor	Arrondir vers le bas	np.floor(arr)
ceil	Arrondir vers le haut	np.ceil(arr)
round	Arrondir au plus proche	np.round(arr, 2)

2.5.3 Trigonométrie

Fonction	Fonctionnalité	Instruction
sin	Sinus	np.sin(arr)
cos	Cosinus	np.cos(arr)
tan	Tangente	np.tan(arr)
arcsin	Arc sinus	np.arcsin(arr)
arccos	Arc cosinus	np.arccos(arr)
arctan	Arc tangente	np.arctan(arr)
deg2rad	Degrés vers Radians	np.deg2rad(180)
rad2deg	Radians vers Degrés	np.rad2deg(np.pi)

2.5.4 Statistiques

Fonction	Fonctionnalité	Instruction
sum	Somme des éléments	np.sum(arr, axis=0)
min	Valeur minimale	np.min(arr)
max	Valeur maximale	np.max(arr)
mean	Moyenne	np.mean(arr)
median	Médiane	np.median(arr)
std	Écart-type	np.std(arr)
var	Variance	np.var(arr)
cumsum	Somme cumulée	np.cumsum(arr)
cumprod	Produit cumulé	np.cumprod(arr)
percentile	Percentile	np.percentile(arr, 75)
argmin	Indice du minimum	np.argmin(arr)

Fonction	Fonctionnalité	Instruction
<code>argmax</code>	Indice du maximum	<code>np.argmax(arr)</code>
<code>corrcoef</code>	Coefficient de corrélation	<code>np.corrcoef(a, b)</code>
<code>histogram</code>	Histogramme	<code>np.histogram(arr, bins=10)</code>

2.5.5 Manipulation de tableaux

Fonction	Fonctionnalité	Instruction
<code>concatenate</code>	Concaténer des tableaux	<code>np.concatenate([a,b],axis=0)</code>
<code>stack</code>	Empiler des tableaux	<code>np.stack([a,b],axis=0)</code>
<code>vstack</code>	Empiler verticalement	<code>np.vstack([a, b])</code>
<code>hstack</code>	Empiler horizontalement	<code>np.hstack([a, b])</code>
<code>split</code>	Diviser un tableau	<code>np.split(arr, 3)</code>
<code>flatten</code>	Aplatir en 1D	<code>arr.flatten()</code>
<code>ravel</code>	Aplatir (vue)	<code>arr.ravel()</code>
<code>transpose</code>	Transposer	<code>np.transpose(arr)</code>
<code>flip</code>	Inverser un tableau	<code>np.flip(arr)</code>
<code>sort</code>	Trier un tableau	<code>np.sort(arr)</code>
<code>argsort</code>	Indices du tri	<code>np.argsort(arr)</code>
<code>unique</code>	Valeurs uniques	<code>np.unique(arr)</code>
<code>where</code>	Condition sur tableau	<code>np.where(arr > 0, arr, 0)</code>
<code>clip</code>	Limiter les valeurs	<code>np.clip(arr, 0, 10)</code>
<code>delete</code>	Supprimer des éléments	<code>np.delete(arr, 2, axis=0)</code>
<code>insert</code>	Insérer des éléments	<code>np.insert(arr, 1, 99)</code>
<code>append</code>	Ajouter des éléments	<code>np.append(arr, [4, 5])</code>

2.5.6 Algèbre linéaire (`np.linalg`)

Fonction	Fonctionnalité	Instruction
<code>dot</code>	Produit scalaire/matriciel	<code>np.dot(a, b)</code>
<code>matmul</code>	Multiplication matricielle	<code>np.matmul(a, b)</code>
<code>linalg.inv</code>	Inverse d'une matrice	<code>np.linalg.inv(A)</code>
<code>linalg.det</code>	Déterminant	<code>np.linalg.det(A)</code>
<code>linalg.eig</code>	Valeurs/vecteurs propres	<code>np.linalg.eig(A)</code>
<code>linalg.norm</code>	Norme d'un vecteur	<code>np.linalg.norm(arr)</code>
<code>linalg.solve</code>	Résoudre $Ax = b$	<code>np.linalg.solve(A, b)</code>
<code>linalg.svd</code>	Décomposition SVD	<code>np.linalg.svd(A)</code>
<code>cross</code>	Produit vectoriel	<code>np.cross(a, b)</code>

2.5.7 Génération aléatoire (`np.random`)

Fonction	Fonctionnalité	Instruction
<code>random.rand</code>	Valeurs aléatoires [0,1]	<code>np.random.rand(3, 4)</code>
<code>random.randn</code>	Distribution normale (0,1)	<code>np.random.randn(3, 4)</code>
<code>random.randint</code>	Entiers aléatoires	<code>np.random.randint(0,10,(3,3))</code>
<code>random.choice</code>	Tirage aléatoire	<code>np.random.choice(arr, 5)</code>
<code>random.shuffle</code>	Mélanger aléatoirement	<code>np.random.shuffle(arr)</code>
<code>random.seed</code>	Fixer la graine	<code>np.random.seed(42)</code>
<code>random.normal</code>	Distribution normale	<code>np.random.normal(0, 1, 100)</code>
<code>random.uniform</code>	Distribution uniforme	<code>np.random.uniform(0, 10, 50)</code>

2.5.8 Comparaisons et Logique

Fonction	Fonctionnalité	Instruction
<code>equal</code>	Égalité élément par élément	<code>np.equal(a, b)</code>
<code>greater</code>	Supérieur	<code>np.greater(a, b)</code>
<code>less</code>	Inférieur	<code>np.less(a, b)</code>
<code>logical_and</code>	ET logique	<code>np.logical_and(a, b)</code>
<code>logical_or</code>	OU logique	<code>np.logical_or(a, b)</code>
<code>logical_not</code>	NON logique	<code>np.logical_not(arr)</code>
<code>any</code>	Au moins un vrai	<code>np.any(arr > 0)</code>
<code>all</code>	Tous vrais	<code>np.all(arr > 0)</code>
<code>isnan</code>	Détecter les NaN	<code>np.isnan(arr)</code>
<code>isinf</code>	Détecter les infinis	<code>np.isinf(arr)</code>
<code>isfinite</code>	Détecter les finis	<code>np.isfinite(arr)</code>

2.5.9 Informations sur les tableaux

Propriété/Fonction	Fonctionnalité	Instruction
<code>shape</code>	Dimensions du tableau	<code>arr.shape</code>
<code>ndim</code>	Nombre de dimensions	<code>arr.ndim</code>
<code>size</code>	Nombre total d'éléments	<code>arr.size</code>
<code>dtype</code>	Type des données	<code>arr.dtype</code>
<code>astype</code>	Convertir le type	<code>arr.astype(float)</code>
<code>copy</code>	Copier un tableau	<code>arr.copy()</code>
<code>itemsize</code>	Taille en octets d'un élément	<code>arr.itemsize</code>
<code>nbytes</code>	Taille totale en octets	<code>arr.nbytes</code>

2.6 Comparaison des Outils Tabulaires

Critere	NumPy	Pandas	Polars
Type principal	ndarray N-D	DataFrame 2-D	DataFrame 2-D
Hétérogénéité	Non (1 type)	Oui	Oui
Performance	Très élevée	Élevée	Très élevée
Mémoire	Optimale	Modérée	Optimale
Valeurs manq.	Non géré	NaN	null
Usage ML/DL	Direct	Via <code>.values</code>	Via <code>to_numpy()</code>
Courbe apprent.	Moyenne	Faible	Moyenne

2.7 Bonnes Pratiques

- **Types** : Vérifiez systématiquement les types avec `df.dtypes` avant tout calcul.
- **NaN** : Traitez les valeurs manquantes AVANT la normalisation (les NaN contaminent les calculs).
- **Copie** : Utilisez `df.copy()` pour éviter les effets de bord sur le DataFrame original.
- **Vectorisation** : Évitez les boucles Python sur des colonnes – utilisez `apply`, `np.vectorize` ou les opérations vectorisées directes.
- **Mémoire** : Pour les grands datasets, utilisez `pd.read_csv(..., chunksize=10000)` ou le format Parquet (`df.to_parquet()`).
- **Reproductibilité** : Fixez `np.random.seed(42)` avant tout échantillonnage.

Attention

Ne confondez pas `df['col']` (Series, 1-D) et `df[['col']]` (DataFrame, 2-D). Cette distinction est critique lors du passage à scikit-learn ou TensorFlow, qui attendent des formes spécifiques.

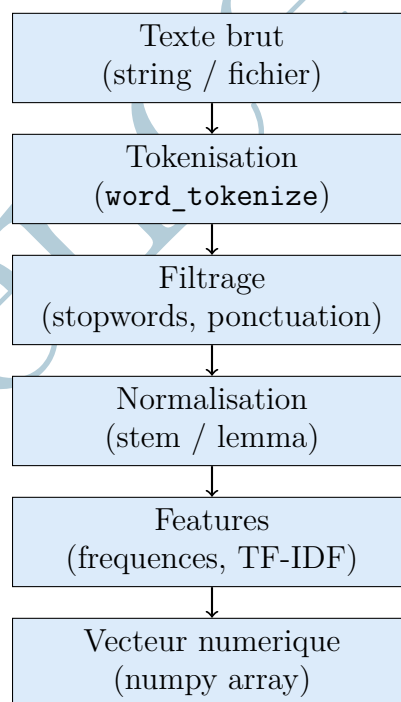
Chapitre 3 NLTK – Traitement du Texte

3.1 Principe Fondamental

Le traitement automatique du langage naturel (TAL ou NLP en anglais) transforme des textes bruts en représentations numériques exploitables par un modèle. **NLTK** (Natural Language Toolkit) est la bibliothèque Python historique de référence pour le TAL : elle fournit plus de 50 corpus, 50 ressources lexicales et un ensemble complet d'outils de traitement.

Dans un pipeline multimodal, NLTK intervient typiquement sur des sous-titres, des transcriptions (issues de Whisper), des commentaires associés aux médias, ou tout autre donnée textuelle accompagnant des fichiers image, audio ou vidéo.

3.2 Architecture du Pipeline TAL



3.3 Fonctions NLTK les plus utilisées

3.3.1 Installation et Configuration

Définition

NLTK (Natural Language Toolkit) est une bibliothèque Python open-source dédiée au traitement automatique du langage naturel (TAL). Avant toute utilisation, elle nécessite le téléchargement de ressources externes (tokeniseurs, listes de stopwords, corpus, modèles) via la fonction `nltk.download()`.

Utilité

La configuration initiale est indispensable : sans les ressources téléchargées, toute tentative de tokenisation ou de filtrage lèvera une erreur `LookupError`. Cette étape est réalisée **une seule fois** sur chaque machine.

Fonction	Fonctionnalité	Instruction
<code>nltk.download()</code>	Télécharger une ressource NLTK	<code>nltk.download('punkt_tab')</code>
<code>nltk.download()</code>	Télécharger les stopwords	<code>nltk.download('stopwords')</code>
<code>nltk.download()</code>	Télécharger WordNet	<code>nltk.download('wordnet')</code>
<code>nltk.download()</code>	Télécharger le tagger POS	<code>nltk.download('averaged_perceptron_tagger_eng')</code>
<code>nltk.download()</code>	Télécharger le corpus Brown	<code>nltk.download('brown')</code>
<code>nltk.data.find()</code>	Vérifier la présence d'une ressource	<code>nltk.data.find('tokenizers/punkt')</code>
<code>nltk.corpus.words()</code>	Accéder au corpus de mots anglais	<code>nltk.corpus.words.words()</code>

3.3.2 Tokenisation**Définition**

La **tokenisation** est l'opération qui consiste à découper un texte brut en unités élémentaires appelées **tokens**. Un token peut être un mot, un nombre, un signe de ponctuation, ou une expression entière. C'est la **première étape obligatoire** de tout pipeline de traitement du langage naturel.

Exemple : "L'analyse des données." → ["L'", "analyse", "des", "données", "."]

Utilité

Sans tokenisation, un texte est une simple chaîne de caractères inutilisable par un

algorithme. Elle permet de :

- Isoler chaque mot pour le comptage de fréquences.
- Identifier la structure grammaticale (POS tagging).
- Préparer les données pour la vectorisation (TF-IDF, sac de mots).

Fonction	Fonctionnalité	Instruction
<code>word_tokenize()</code>	Découper un texte en mots	<code>word_tokenize(texte, language='french')</code>
<code>sent_tokenize()</code>	Découper un texte en phrases	<code>sent_tokenize(texte, language='french')</code>
<code>TweetTokenizer()</code>	Tokeniser des tweets	<code>TweetTokenizer().tokenize(texte)</code>
<code>RegexpTokenizer()</code>	Tokeniser par expression régulière	<code>RegexpTokenizer(r'\w+').tokenize(t)</code>
<code>MWETokenizer()</code>	Grouper les expressions multi-mots	<code>MWETokenizer([('New', 'York')])</code>
<code>SyllableTokenizer()</code>	Tokeniser en syllabes	<code>SyllableTokenizer().tokenize(mot)</code>
<code>PunktTokenizer()</code>	Tokeniseur basé sur Punkt	<code>PunktTokenizer().tokenize(texte)</code>
<code>TreebankWordTokenizer</code>	Tokeniseur Penn Treebank	<code>TreebankWordTokenizer().tokenize(t)</code>

3.3.3 Stopwords et Nettoyage

Définition

Les **stopwords** (mots vides) sont des mots très fréquents dans une langue mais qui n'apportent pas de valeur sémantique discriminante : articles, prépositions, conjonctions (*le, la, de, et, est...*). Le **nettoyage** désigne l'ensemble des opérations qui éliminent le bruit textuel : suppression de la ponctuation, normalisation de la casse, retrait des chiffres et des caractères spéciaux.

Utilité

Supprimer les stopwords et nettoyer le texte réduit le vocabulaire de 30 à 50 % tout en améliorant la qualité des représentations vectorielles. Un modèle TF-IDF entraîné sans filtrage sera dominé par des mots comme « le » ou « de » qui ne permettent pas de distinguer les documents.

Fonction	Fonctionnalité	Instruction
<code>stopwords.words()</code>	Liste des stopwords d'une langue	<code>stopwords.words('french')</code>
<code>stopwords.words()</code>	Stopwords en anglais	<code>stopwords.words('english')</code>
<code>stopwords.fileids()</code>	Langues disponibles	<code>stopwords.fileids()</code>
<code>isalpha()</code>	Garder les tokens alphabétiques	<code>[t for t in tokens if t.isalpha()]</code>
<code>lower()</code>	Mettre en minuscules	<code>texte.lower()</code>
<code>strip()</code>	Supprimer les espaces en bordure	<code>texte.strip()</code>
<code>string.punctuation</code>	Ensemble de la ponctuation Python	<code>set(string.punctuation)</code>
<code>re.sub()</code>	Supprimer les caractères spéciaux	<code>re.sub(r'^a-zA-Z', ' ', t)</code>

3.3.4 Racinisation (Stemming)

Définition

La **racinisation** (ou *stemming*) est une technique de normalisation morphologique qui réduit chaque mot à son **radical** en supprimant les suffixes par des règles algorithmiques. Le résultat n'est pas toujours un mot valide de la langue.

Exemple : analyser, analysons, analyse → analys

Utilité

La racinisation regroupe les formes fléchies d'un même mot sous un radical unique, ce qui réduit la dimensionnalité du vocabulaire et améliore la généralisation des modèles. Elle est **rapide** et **sans lexique**, ce qui la rend adaptée aux grands corpus. Son inconvénient est qu'elle peut produire des radicaux incorrects (*sur-racinisation*).

Fonction	Fonctionnalité	Instruction
<code>SnowballStemmer()</code>	Raciniseur multilingue (recommandé FR)	<code>SnowballStemmer('french')</code>

Fonction	Fonctionnalité	Instruction
<code>stemmer.stem()</code>	Obtenir le radical d'un mot	<code>stemmer.stem('analysons')</code>
<code>PorterStemmer()</code>	Raciniseur anglais classique	<code>PorterStemmer()</code>
<code>porter.stem()</code>	Radical par l'algorithme de Porter	<code>porter.stem('running')</code>
<code>LancasterStemmer()</code>	Raciniseur agressif (Lancaster)	<code>LancasterStemmer()</code>
<code>lancaster.stem()</code>	Radical par l'algorithme Lancaster	<code>lancaster.stem('running')</code>
<code>SnowballStemmer.lang</code>	Langues supportées par Snowball	<code>SnowballStemmer.languages</code>

3.3.5 Lemmatisation

Définition

La **lemmatisation** est une normalisation morphologique plus précise que la racinisation. Elle retourne le **lemme**, c'est-à-dire la forme canonique du mot telle qu'elle apparaît dans le dictionnaire, en tenant compte du contexte grammatical (nom, verbe, adjectif).

Exemple : `running` (verbe) $\rightarrow$ `run` ; `better` (adjectif) $\rightarrow$ `good`

Utilité

Contrairement à la racinisation, la lemmatisation produit toujours un mot valide. Elle est indispensable quand la qualité linguistique du résultat importe, par exemple pour la **traduction automatique**, la **recherche d'information** ou la **génération de texte**. Elle est plus lente car elle nécessite un lexique (WordNet).

Fonction	Fonctionnalité	Instruction
<code>WordNetLemmatizer()</code>	Créer un lemmatiseur WordNet	<code>WordNetLemmatizer()</code>
<code>lemmatizer.lemmatize()</code>	Lemmatiser un mot (nom par défaut)	<code>lem.lemmatize('running')</code>
<code>lemmatize(pos='v')</code>	Lemmatiser un verbe	<code>lem.lemmatize('running', pos='v')</code>

Fonction	Fonctionnalité	Instruction
<code>lemmatize(pos='a')</code>	Lemmatiser un adjectif	<code>lem.lemmatize('better', pos='a')</code>
<code>lemmatize(pos='r')</code>	Lemmatiser un adverbe	<code>lem.lemmatize('fastest', pos='r')</code>
<code>morphy()</code>	Formes morphologiques d'un mot	<code>wordnet.morphy('dogs')</code>
<code>wordnet.synsets()</code>	Obtenir les synsets d'un mot	<code>wordnet.synsets('cat')</code>

3.3.6 Étiquetage Grammatical (POS Tagging)

Définition

L'**étiquetage POS** (*Part-of-Speech Tagging*) associe à chaque token sa **catégorie grammaticale** : nom (NN), verbe (VB), adjectif (JJ), adverbe (RB), préposition (IN), etc. NLTK utilise le jeu d'étiquettes **Penn Treebank** pour l'anglais.

Exemple : `[("The", "DT"), ("cat", "NN"), ("runs", "VBZ")]`

Utilité

Le POS tagging est utilisé pour :

- Extraire uniquement les **noms propres** ou les **verbes** d'un texte.
- Améliorer la **lemmatisation** (le lemme d'un verbe diffère de celui d'un nom).
- Détecter les **entités nommées** (personnes, lieux, organisations).
- Alimenter des modèles de **traduction automatique** ou d'**analyse syntaxique**.

Fonction	Fonctionnalité	Instruction
<code>pos_tag()</code>	Étiqueter une liste de tokens	<code>pos_tag(tokens)</code>
<code>pos_tag_sents()</code>	Étiqueter plusieurs phrases	<code>pos_tag_sents([tokens1, tokens2])</code>
<code>PerceptronTagger()</code>	Tagger par perceptron (rapide)	<code>PerceptronTagger().tag(tokens)</code>
<code>ne_chunk()</code>	Reconnaître les entités nommées	<code>ne_chunk(pos_tag(tokens))</code>
<code>tree2conlltags()</code>	Convertir un arbre en tags CoNLL	<code>tree2conlltags(chunked)</code>

Fonction	Fonctionnalité	Instruction
<code>RegexParser()</code>	Analyseur syntaxique par regex	<code>RegexParser(grammaire).parse(tagged)</code>
<code>nltk.tag.map_tag()</code>	Convertir les tags Penn vers Universal	<code>map_tag('en-ptb', 'universal', tag)</code>

Principales étiquettes Penn Treebank :

Tag	Catégorie	Exemple
NN	Nom singulier	<i>city, tree</i>
NNS	Nom pluriel	<i>cities, trees</i>
NNP	Nom propre singulier	<i>Paris, OpenAI</i>
VB	Verbe (base)	<i>run, analyze</i>
VBZ	Verbe (3e pers. sing.)	<i>runs, analyzes</i>
VBD	Verbe (passé)	<i>ran, analyzed</i>
JJ	Adjectif	<i>fast, beautiful</i>
RB	Adverbe	<i>quickly, never</i>
IN	Préposition	<i>in, of, through</i>
DT	Déterminant	<i>the, a, an</i>
PRP	Pronom personnel	<i>he, she, it</i>
CC	Conjonction	<i>and, but, or</i>

3.3.7 Fréquences et N-grammes

Définition

La **distribution de fréquences** comptabilise le nombre d'occurrences de chaque token dans un corpus. Un **N-gramme** est une séquence contiguë de N tokens :

- **Bigramme** (N=2) : séquence de 2 mots consécutifs.
- **Trigramme** (N=3) : séquence de 3 mots consécutifs.

Exemple : "apprentissage automatique" est un bigramme significatif.

Utilité

Les fréquences permettent d'identifier les **mots-clés** d'un document. Les N-grammes capturent le **contexte local** et les **expressions figées**, que les modèles mot-par-mot ne détectent pas. Ils sont utilisés dans la **correction orthographique**, la **prédiction de texte** et la **classification de documents**.

Fonction	Fonctionnalité	Instruction
<code>FreqDist()</code>	Calculer les fréquences des tokens	<code>FreqDist(tokens)</code>
<code>freq.most_common()</code>	Obtenir les n tokens les plus fréquents	<code>freq.most_common(10)</code>
<code>freq.freq()</code>	Fréquence relative d'un token	<code>freq.freq('chat')</code>
<code>freq.N()</code>	Nombre total de tokens	<code>freq.N()</code>
<code>bigrams()</code>	Générer des bigrammes	<code>list(bigrams(tokens))</code>
<code>trigrams()</code>	Générer des trigrammes	<code>list(trigrams(tokens))</code>
<code>ngrams()</code>	Générer des n-grammes arbitraires	<code>list(ngrams(tokens, 4))</code>
<code>ConditionalFreqDist</code>	Distribution conditionnelle	<code>ConditionalFreqDist(paires)</code>
<code>collocations()</code>	Afficher les collocations fréquentes	<code>freq.collocations()</code>
<code>hapaxes()</code>	Mots apparaissant une seule fois	<code>freq.hapaxes()</code>

3.3.8 Corpus et Ressources Intégrés

Définition

Un **corpus** est un ensemble de textes organisé et annoté, utilisé pour entraîner ou évaluer des modèles de TAL. NLTK intègre plusieurs corpus de référence accessibles directement en Python après téléchargement.

Utilité

Les corpus intégrés permettent de **prototyper rapidement** sans collecter de données : entraîner un classifieur de sentiment avec `movie_reviews`, tester un tokeniseur avec `gutenberg`, ou construire un modèle de langue avec `brown`. **WordNet** est particulièrement utile pour mesurer la **similarité sémantique** entre mots.

Fonction	Fonctionnalité	Instruction
gutenberg	Corpus de livres du Projet Gutenberg	<code>gutenberg.words('austen-emma.txt')</code>
brown	Corpus Brown (1 million de mots)	<code>brown.words(categories='news')</code>
reuters	Corpus Reuters (actualités)	<code>reuters.words()</code>
wordnet	Base lexicale hiérarchique	<code>wordnet.synsets('dog')</code>
stopwords	Listes de mots vides multilingues	<code>stopwords.words('french')</code>
words	Liste de mots anglais courants	<code>words.words()</code>
treebank	Corpus Penn Treebank (annoté)	<code>treebank.tagged_words()</code>
movie_reviews	Avis de films (analyse de sentiment)	<code>movie_reviews.words(categories='pos')</code>
inaugural	Discours d'investiture US	<code>inaugural.fileids()</code>
twitter_samples	Tweets pour entraînement	<code>twitter_samples.strings('positive_tweets.json')</code>

3.3.9 Vectorisation et Représentation

Définition

La **vectorisation** transforme un texte en **représentation numérique** exploitable par un algorithme d'apprentissage automatique. Les méthodes principales sont :

- **Sac de mots (BoW)** : vecteur des fréquences de chaque mot du vocabulaire.
- **TF-IDF** : pondère chaque terme par son importance relative dans le corpus (*fréquence dans le document* $\times$ *rareté dans le corpus*).

Utilité

La vectorisation est l'**étape clé** qui relie le TAL au Machine Learning. Sans elle, aucun classifieur (SVM, réseau de neurones, etc.) ne peut traiter du texte. TF-IDF surpasse le sac de mots car il pénalise les mots trop fréquents (articles, prépositions) et met en valeur les termes **discriminants**.

Fonction	Fonctionnalité	Instruction
<code>FreqDist()</code>	Sac de mots (fréquences brutes)	<code>FreqDist(tokens)</code>
<code>Counter()</code>	Comptage des tokens (Python natif)	<code>Counter(tokens)</code>
<code>TfidfVectorizer()</code>	Matrice TF-IDF (via sklearn)	<code>TfidfVectorizer().fit_transform(corpus)</code>
<code>CountVectorizer()</code>	Sac de mots vectorisé (via sklearn)	<code>CountVectorizer().fit_transform(corpus)</code>
<code>DictVectorizer()</code>	Vectoriser un dictionnaire de features	<code>DictVectorizer().fit_transform(dicts)</code>
<code>pos_tag()</code>	Features grammaticales pour classif.	<code>pos_tag(tokens)</code>
<code>ngrams()</code>	N-grammes comme features	<code>list(ngrams(tokens, 2))</code>

3.3.10 Reconnaissance d'Entités Nommées (NER)

Définition

La **reconnaissance d'entités nommées** (NER, *Named Entity Recognition*) identifie et classe automatiquement les **entités** dans un texte : personnes (**PERSON**), lieux (**GPE**), organisations (**ORGANIZATION**), dates, montants, etc. Elle s'appuie sur le POS tagging comme étape préalable.

Exemple : "Barack Obama est né à Hawaii." → **PERSON**: Barack Obama, **GPE**: Hawaii

Utilité

La NER est utilisée pour :

- Extraire automatiquement des **faits** depuis des corpus non structurés.
- Alimenter des **graphes de connaissances** (knowledge graphs).
- Analyser des flux d'actualités pour détecter les **personnes** ou **entreprises** mentionnées.
- Améliorer les moteurs de **recherche d'information**.

Fonction	Fonctionnalité	Instruction
<code>ne_chunk()</code>	Détecter les entités nommées	<code>ne_chunk(pos_tag(tokens))</code>
<code>ne_chunk(binary=True)</code>	Entités en mode binaire	<code>ne_chunk(tagged, binary=True)</code>
<code>tree2conlltags()</code>	Extraire les tags IOB d'un arbre	<code>tree2conlltags(chunked)</code>
<code>RegexpParser()</code>	Extraire des chunks par patron regex	<code>RegexpParser('NP: {<DT>?<JJ>*<NN>}')</code>
<code>chunk.ne_chunk()</code>	Arbre d'entités nommées	<code>nltk.chunk.ne_chunk(tagged)</code>
<code>conlltags2tree()</code>	Convertir les tags CoNLL en arbre	<code>conlltags2tree(tagged_sents)</code>

3.3.11 Analyse de Sentiment et Classification

Définition

L'**analyse de sentiment** détermine la **polarité affective** d'un texte : positive, négative ou neutre. NLTK propose deux approches :

- **VADER** : analyseur lexical basé sur un dictionnaire de scores, adapté aux textes courts (réseaux sociaux, avis).
- **Classifieurs supervisés** : modèles entraînés sur des données étiquetées (`NaiveBayesClassifier`, `MaxentClassifier`).

Utilité

L'analyse de sentiment est massivement utilisée pour :

- Analyser les **avis clients** sur des produits ou services.
- Surveiller l'**e-réputation** d'une marque sur les réseaux sociaux.
- Détecter des signaux de **détresse** dans des textes de santé mentale.
- Construire des **pipelines de modération** de contenu automatique.

Fonction	Fonctionnalité	Instruction
<code>SentimentIntensityAnalyzer</code>	Analyse de sentiment VADER	<code>SentimentIntensityAnalyzer()</code>
<code>sia.polarity_scores</code>	Scores pos/neg/neu/-compound	<code>sia.polarity_scores(texte)</code>

Fonction	Fonctionnalité	Instruction
<code>NaiveBayesClassifier</code>	Classifieur bayésien naïf	<code>NaiveBayesClassifier.train(train_set)</code>
<code>classifier.classify()</code>	Classer un exemple	<code>classifier.classify(features)</code>
<code>classifier.accuracy()</code>	Évaluer la précision	<code>classify.accuracy(classifier, test_set)</code>
<code>show_most_informative_features()</code>	Features les plus discriminantes	<code>classifier.show_most_informative_features(10)</code>
<code>MaxentClassifier()</code>	Classifieur à entropie maximale	<code>MaxentClassifier.train(train_set)</code>

3.3.12 Expressions Régulières et Utilitaires

Définition

Les **expressions régulières** (regex) sont des patrons de recherche qui permettent de détecter, extraire ou remplacer des sous-chaînes dans un texte selon des règles formelles. NLTK s'appuie sur le module `re` de Python. L'objet `nltk.Text` offre des outils d'exploration interactifs sur un corpus de tokens.

Utilité

Les regex sont indispensables pour :

- Extraire des **adresses e-mail**, numéros de téléphone, URLs d'un corpus.
- Nettoyer des textes bruités (balises HTML, caractères spéciaux, chiffres).
- Construire des **tokeniseurs personnalisés** pour des domaines spécifiques (code source, textes médicaux, tweets).

`nltk.Text` facilite l'**exploration rapide** d'un corpus en ligne de commande ou en notebook Jupyter.

Fonction	Fonctionnalité	Instruction
<code>re.findall()</code>	Extraire tous les matches d'un pattern	<code>re.findall(r'\b\w+\b', texte)</code>
<code>re.sub()</code>	Remplacer selon un pattern	<code>re.sub(r'\d+', "", texte)</code>
<code>re.split()</code>	Découper selon un pattern	<code>re.split(r'\s+', texte)</code>
<code>nltk.Text()</code>	Créer un objet Text NLTK	<code>nltk.Text(tokens)</code>

Fonction	Fonctionnalité	Instruction
<code>text.concordance()</code>	Contexte d'un mot dans le texte	<code>text.concordance('python')</code>
<code>text.similar()</code>	Mots apparaissant dans contextes sembl.	<code>text.similar('good')</code>
<code>text.collocations()</code>	Paires de mots fréquentes	<code>text.collocations()</code>
<code>text.count()</code>	Compter les occurrences d'un token	<code>text.count('the')</code>
<code>text.dispersion_plot()</code>	Graphes de dispersion des mots	<code>text.dispersion_plot(['word1', 'word2'])</code>

3.4 Comparaison des Outils TAL

Critere	NLTK	spaCy	Transformers
Langues	50+	70+	100+
Tokenisation FR	Correcte	Excellente	Excellente
POS Tagging	Anglais nat.	Multi-langues	Multi-langues
Lemmatisation	WordNet	Natif	Natif
NER	Limitee	Tres bonne	Tres bonne
Performance	Lente	Rapide	Variable (GPU)
Courbe apprentissage	Faible	Faible	Elevee

3.5 Bonnes Pratiques

- **Encodage** : Normalisez l'encodage en UTF-8 avant toute tokenisation.
- **Casse** : Appliquez `.lower()` systematiquement avant la creation du vocabulaire.
- **Ponctuation** : Utilisez `string.punctuation` pour supprimer tous les signes.
- **Versions** : Avec NLTK ≥ 3.9 , remplacez `punkt` par `punkt_tab`.
- **Langue** : Pour le francais, verifiez la disponibilite des stopwords avec `stopwords.words('french')`
- **Scalabilite** : Pour de grands corpus ($> 100\,000$ documents), utilisez `scikit-learn` `TfidfVectorizer` qui est optimise pour la vectorisation en memoire.

Remarque

NLTK est ideal pour l'apprentissage et les petits corpus. Pour la production en francais, **spaCy** avec le modele `fr_core_news_sm` offre de meilleures performances et une meilleure gestion des accents. Les deux partagent les memes concepts fondamentaux.

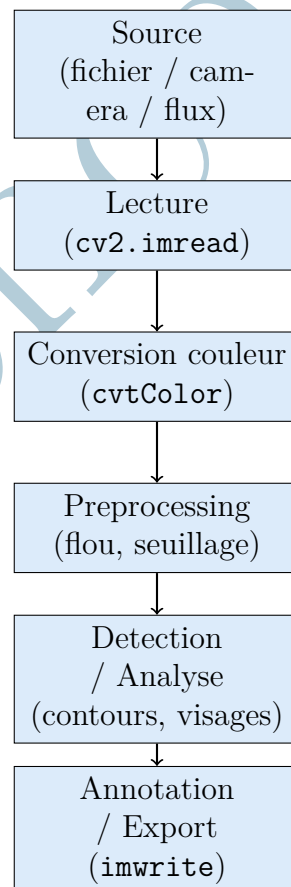
Chapitre 4 OpenCV – Traitement des Images

4.1 Principe Fondamental

OpenCV (Open Source Computer Vision Library) est la bibliothèque de référence mondiale pour la vision par ordinateur. Initialement développée par Intel en 1999, elle regroupe plus de 2 500 algorithmes optimisés couvrant : le filtrage d'images, la détection de contours et d'objets, la calibration de caméras, le flux optique et bien d'autres domaines.

En Python, OpenCV représente chaque image comme un tableau NumPy de forme **(H, W, C)** ou H = hauteur, W = largeur, C = nombre de canaux. Un point crucial à retenir : OpenCV lit et stocke les couleurs au format **BGR** (Bleu-Vert-Rouge) et non RGB, contrairement à Matplotlib et Pillow.

4.1.1 Architecture du Pipeline Vision



4.1.2 Installation

Listing 4.1: Installation d'OpenCV

```
pip install opencv-python opencv-python-headless
# opencv-python          : avec GUI (imshow)
# opencv-python-headless: sans GUI (serveurs, Colab)
```

4.2 Lecture, Écriture et Conversion

4.2.1 Chargement et Inspection

Listing 4.2: Chargement et inspection complète d'une image

```
import cv2
import numpy as np

def inspecter_image(chemin: str) -> dict:
    """
    Charge une image et affiche un rapport d'inspection
    complet.

    Args:
        chemin: Chemin vers le fichier image (.jpg, .png,
               .bmp, .tiff)

    Returns:
        Dictionnaire contenant l'image et ses metadonnées

    Note:
        OpenCV charge les images en BGR (pas RGB) avec des
        valeurs uint8 [0, 255].
    """
    import os

    if not os.path.exists(chemin):
        raise FileNotFoundError(f"Image introuvable :
                               {chemin}")

    # Chargement (BGR par défaut)
    img = cv2.imread(chemin)

    if img is None:
        raise ValueError(f"Impossible de lire : {chemin}
                          (format non supporté ?)")

    # Dimensions
    if img.ndim == 3:
        hauteur, largeur, canaux = img.shape
    else:
        hauteur, largeur = img.shape
        canaux = 1
```

```

# Statistiques par canal
print("=== Inspection de l'image ===")
print(f"Fichier          : {chemin}")
print(f"Dimensions       : {largeur} x {hauteur} pixels")
print(f"Canaux            : {canaux} ({'BGR' if canaux==3
    else 'Niveaux de gris'})")
print(f"Type de donnees  : {img.dtype}")
print(f"Forme NumPy       : {img.shape}")
print(f"Valeur min / max  : {img.min()} / {img.max()}")
print(f"Taille memoire    : {img.nbytes / 1024:.1f} Ko")

if canaux == 3:
    for i, nom in enumerate(['B', 'G', 'R']):
        canal = img[:, :, i]
        print(f"Canal {nom} : moy={canal.mean():.1f},
            ecart-type={canal.std():.1f}")

return {
    'image'      : img,
    'largeur'    : largeur,
    'hauteur'    : hauteur,
    'canaux'     : canaux,
    'dtype'      : str(img.dtype),
    'min'        : int(img.min()),
    'max'        : int(img.max())
}

# Exemple
info = inspecter_image("photo_produit.jpg")
img = info['image']

```

4.2.2 Conversion de l'Espace Couleur

Listing 4.3: Conversions d'espaces couleur essentielles

```

import cv2
import numpy as np

def convertir_espaces(img_bgr: np.ndarray) -> dict:
    """
    Realise les conversions d'espaces couleur les plus
    courantes.

    BGR -> RGB   : pour Matplotlib / Pillow / TensorFlow
    BGR -> GRAY  : niveaux de gris, prerequisite pour la plupart
    des detecteurs
    BGR -> HSV   : teintes (Hue), saturation, valeur -- utile
    pour la segmentation par couleur

    Returns:
        Dictionnaire {nom: image convertie}
    """

```



```

"""
conversions = {
    'bgr' : img_bgr,
    'rgb' : cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB),
    'gray' : cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY),
    'hsv' : cv2.cvtColor(img_bgr, cv2.COLOR_BGR2HSV),
    'lab' : cv2.cvtColor(img_bgr, cv2.COLOR_BGR2LAB),
}

print("=== Conversions d'espaces couleur ===")
for nom, img in conversions.items():
    print(f" {nom:5s} : forme={img.shape},
           dtype={img.dtype}")

return conversions

# Extraction de la couleur dominante (dans l'espace HSV)
def couleur_dominante_hsv(img_bgr: np.ndarray) -> tuple:
    """
    Determine la couleur dominante par analyse de
    l'histogramme HSV.

    Returns:
        (H, S, V) de la couleur dominante
    """
    img_hsv = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2HSV)

    # Histogramme sur le canal Hue (teinte)
    hist_h = cv2.calcHist([img_hsv], [0], None, [180], [0,
        180])
    teinte_dom = int(np.argmax(hist_h))

    hist_s = cv2.calcHist([img_hsv], [1], None, [256], [0,
        256])
    sat_dom = int(np.argmax(hist_s))

    print(f"Couleur dominante HSV : H={teinte_dom},
          S={sat_dom}")
    return (teinte_dom, sat_dom, 128)

espaces = convertir_espaces(img)
hsv = couleur_dominante_hsv(img)

```

4.3 Filtrage et Preprocessing

4.3.1 Réduction du Bruit et Lissage

Listing 4.4: Application des filtres de lissage principaux

```

import cv2
import numpy as np

```

```

def appliquer_filtres(img: np.ndarray) -> dict:
    """
    Applique les filtres de lissage classiques pour la
    reduction du bruit.

    Filtres implementes :
        - Gaussien      : lissage isotrope, preserve les bords
                          moderelement
        - Median        : excellent pour le bruit impulsif (poivre et sel)
        - Bilateral      : preserve les bords tout en lissant
                          les zones uniformes
        - Box (moyen)   : le plus rapide, moins selectif

    Args:
        img: Image en niveaux de gris ou BGR

    Returns:
        Dictionnaire {nom_filtre: image_filtree}
    """
    filtres = {}

    # Filtre Gaussien (noyau 5x5, sigma auto)
    filtres['gaussien_5x5'] = cv2.GaussianBlur(img, (5, 5), 0)

    # Filtre median (kernel 5x5)
    filtres['median_5'] = cv2.medianBlur(img, 5)

    # Filtre bilateral (preserve les contours)
    filtres['bilateral'] = cv2.bilateralFilter(img, d=9,
                                                sigmaColor=75,
                                                sigmaSpace=75)

    # Filtre moyen (box)
    filtres['moyen_5x5'] = cv2.blur(img, (5, 5))

    print("Filtres appliques :")
    for nom, img_f in filtres.items():
        diff = np.abs(img.astype(float) - img_f.astype(float))
        print(f" {nom:20s} : modification moyenne = {diff.mean():.2f}")

    return filtres

# Exemple : appliquer sur niveaux de gris
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
filtres = appliquer_filtres(gray)

```

4.3.2 Détection de Contours

Listing 4.5: Detection de contours avec Canny et findContours

```

import cv2
import numpy as np

def detecter_contours(
    img_bgr: np.ndarray,
    seuil_bas: int = 50,
    seuil_haut: int = 150,
    flou_avant: bool = True
) -> tuple:
    """
    Detecte les contours dans une image par la methode de
    Canny.

    Principe de Canny :
    1. Reduction du bruit (Gaussien)
    2. Calcul du gradient (Sobel)
    3. Suppression des non-maxima
    4. Seuillage par hysteresis (seuil_bas, seuil_haut)

    Args:
        img_bgr      : Image source BGR
        seuil_bas    : Premier seuil d'hysteresis (bords faibles)
        seuil_haut   : Deuxieme seuil d'hysteresis (bords forts)
        flou_avant   : Appliquer un lissage Gaussien avant la
                        detection

    Returns:
        (image_contours, image_annotee, liste_contours)
    """
    # Conversion en niveaux de gris
    gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)

    # Pre-lissage recommande
    if flou_avant:
        gray = cv2.GaussianBlur(gray, (5, 5), 0)

    # Detection de Canny
    contours_img = cv2.Canny(gray, seuil_bas, seuil_haut)

    # Extraction des contours comme structures geometriques
    contours, hierarchie = cv2.findContours(
        contours_img,
        cv2.RETR_EXTERNAL,      # Contours exterieurs
                               uniquement
        cv2.CHAIN_APPROX_SIMPLE # Compression des segments
                               droits
    )

    # Annotation sur l'image originale
    img_annot = img_bgr.copy()

```

```

cv2.drawContours(img_annot, contours, -1, (0, 255, 0), 2)

# Statistiques
print(f"Contours detectes : {len(contours)}")
for i, cnt in enumerate(contours[:5]):
    aire = cv2.contourArea(cnt)
    x, y, w, h = cv2.boundingRect(cnt)
    print(f" [{i}] aire={aire:.0f} px2,
           boite=({x},{y},{w},{h})")

# Sauvegarde
cv2.imwrite("resultat_contours.jpg", img_annot)
cv2.imwrite("masque_canny.jpg", contours_img)

return contours_img, img_annot, contours

img_canny, img_annot, contours = detecter_contours(img,
seuil_bas=50, seuil_haut=150)

```

4.4 Détection de Visages par Haar Cascades

Listing 4.6: Detection de visages avec les classificateurs Haar

```

import cv2
import numpy as np
import os

def detecter_visages(
    img_bgr: np.ndarray,
    facteur_echelle: float = 1.1,
    voisins_min: int = 5,
    taille_min: tuple = (30, 30)
) -> tuple:
    """
    Detecte les visages dans une image via les classificateurs
    Haar.

    Note : Les Haar Cascades sont rapides mais moins precises
    que les CNN modernes.
    Pour la production, privilégiez DNN face detection (OpenCV
    DNN module) ou MTCNN.

    Args:
        img_bgr : Image source BGR
        facteur_echelle: Facteur de reduction de l'image a
            chaque echelle (1.1 = 10%)
        voisins_min : Nombre minimum de rectangles
            candidats pour valider un visage
        taille_min : Taille minimale d'un visage detecte
            (pixels)
    """

```

```

Returns:
    (image_annotee, liste de rectangles (x, y, w, h))
"""
# Chemin vers le fichier Haar (installé avec OpenCV)
chemin_haar = cv2.data.harcascades +
    'haarcascade_frontalface_default.xml'

if not os.path.exists(chemin_haar):
    raise FileNotFoundError(f"Haar cascade introuvable : {chemin_haar}")

# Chargement du classificateur
detecteur = cv2.CascadeClassifier(chemin_haar)

# Detection sur image en niveaux de gris
gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)

visages = detecteur.detectMultiScale(
    gray,
    scaleFactor=facteur_echelle,
    minNeighbors=voisins_min,
    minSize=taille_min,
    flags=cv2.CASCADE_SCALE_IMAGE
)

# Annotation
img_annot = img_bgr.copy()
for i, (x, y, w, h) in enumerate(visages):
    cv2.rectangle(img_annot, (x, y), (x + w, y + h), (0,
        255, 0), 2)
    cv2.putText(img_annot, f"Visage {i+1}", (x, y - 10),
        cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255,
        0), 2)

print(f"Visages detectés : {len(visages)}")
cv2.imwrite("visages_detectes.jpg", img_annot)

return img_annot, visages

img_visages, rects = detecter_visages(img)

```

4.5 Fonctions NLTK les plus utilisées

4.5.1 Installation et Configuration

Définition

OpenCV (*Open Source Computer Vision Library*) est une bibliothèque open-source

écrite en C++ avec interface Python, dédiée à la vision par ordinateur et au traitement d'images. Elle propose plus de 2500 algorithmes optimisés pour la lecture, la manipulation, le filtrage, la segmentation et l'analyse d'images et de vidéos. L'installation se fait via `pip` en deux variantes : `opencv-python` (modules principaux) et `opencv-contrib-python` (modules supplémentaires incluant SIFT, SURF, etc.).

Utilité

La configuration initiale est rapide et ne nécessite pas de téléchargement de ressources externes (contrairement à NLTK). OpenCV s'appuie nativement sur **NumPy** : toute image chargée est représentée comme un tableau `ndarray` de dimensions (**hauteur**, **largeur**, **canaux**). Cette intégration permet d'utiliser toutes les opérations vectorisées de NumPy sur les pixels.

Fonction	Fonctionnalité	Instruction
<code>pip install</code>	Installer OpenCV (modules principaux)	<code>pip install opencv-python</code>
<code>pip install</code>	Installer modules supplémentaires	<code>pip install opencv-contrib-python</code>
<code>import cv2</code>	Importer la bibliothèque	<code>import cv2</code>
<code>cv2.__version__</code>	Vérifier la version installée	<code>print(cv2.__version__)</code>
<code>cv2.getBuildInformation</code>	Infos de compilation OpenCV	<code>print(cv2.getBuildInformation())</code>
<code>import numpy as np</code>	Importer NumPy (dépendance)	<code>import numpy as np</code>
<code>cv2.useOptimized()</code>	Vérifier l'optimisation SIMD	<code>cv2.useOptimized()</code>

4.5.2 Lecture, Écriture et Affichage

Définition

La **lecture** consiste à charger un fichier image ou vidéo depuis le disque en mémoire sous forme de tableau NumPy. L'**écriture** sauvegarde un tableau modifié vers le disque dans un format spécifique (PNG, JPG, TIFF, BMP). L'**affichage** ouvre une fenêtre interactive permettant de visualiser l'image. C'est la **première étape obligatoire** de tout pipeline de vision.

Attention : OpenCV utilise par défaut l'ordre des canaux **BGR** (Blue-Green-Red) et non RGB.

Utilité

Sans lecture correcte, aucun traitement n'est possible. Il faut toujours :

- Vérifier que l'image n'est pas `None` après `imread()`.
- Convertir BGR vers RGB avant tout affichage avec Matplotlib.
- Utiliser `cv2.waitKey(0)` pour bloquer la fenêtre OpenCV.
- Libérer les ressources avec `cv2.destroyAllWindows()`.

Fonction	Fonctionnalité	Instruction
<code>cv2.imread()</code>	Lire une image (BGR par défaut)	<code>img = cv2.imread('photo.jpg')</code>
<code>cv2.imread(., 0)</code>	Lire en niveaux de gris	<code>cv2.imread('photo.jpg', 0)</code>
<code>cv2.imread(., -1)</code>	Lire avec canal alpha	<code>cv2.imread('photo.png', -1)</code>
<code>cv2.imwrite()</code>	Sauvegarder une image	<code>cv2.imwrite('out.png', img)</code>
<code>cv2.imshow()</code>	Afficher dans une fenêtre	<code>cv2.imshow('Fenetre', img)</code>
<code>cv2.waitKey()</code>	Attendre une touche (ms)	<code>cv2.waitKey(0)</code>
<code>cv2.destroyAllWindows()</code>	Fermer toutes les fenêtres	<code>cv2.destroyAllWindows()</code>
<code>cv2.namedWindow()</code>	Créer une fenêtre nommée	<code>cv2.namedWindow('W', cv2.WINDOW_NORMAL)</code>
<code>cv2.imencode()</code>	Encoder en mémoire (buffer)	<code>cv2.imencode('.jpg', img)</code>
<code>cv2.imdecode()</code>	Décoder depuis un buffer	<code>cv2.imdecode(buf, cv2.IMREAD_COLOR)</code>

4.5.3 Espaces Colorimétriques

Définition

Un **espace colorimétrique** est un modèle mathématique de représentation des couleurs. OpenCV supporte plus de 150 conversions via la fonction `cv2.cvtColor()`. Les principaux espaces sont :

- **BGR** : ordre par défaut d'OpenCV (3 canaux 8 bits).

- **RGB** : ordre standard (Matplotlib, PIL).
- **Niveaux de gris** : 1 canal, intensité lumineuse.
- **HSV** : Teinte, Saturation, Valeur (segmentation par couleur).
- **LAB** : perceptuellement uniforme (correction colorimétrique).

Utilité

Le choix de l'espace colorimétrique est déterminant :

- **Niveaux de gris** : indispensable pour la détection de contours, le seuillage et l'analyse morphologique.
- **HSV** : permet de segmenter par couleur indépendamment de la luminosité (détection d'objets rouges, panneaux routiers).
- **LAB** : utilisé pour les corrections de balance des blancs et les mesures de différence de couleur (Delta E).

Fonction	Fonctionnalité	Instruction
<code>cv2.cvtColor()</code>	Convertir l'espace colorimétrique	<code>cv2.cvtColor(img, code)</code>
<code>COLOR_BGR2GRAY</code>	BGR vers niveaux de gris	<code>cvtColor(img, cv2.COLOR_BGR2GRAY)</code>
<code>COLOR_BGR2RGB</code>	BGR vers RGB (pour Matplotlib)	<code>cvtColor(img, cv2.COLOR_BGR2RGB)</code>
<code>COLOR_BGR2HSV</code>	BGR vers HSV	<code>cvtColor(img, cv2.COLOR_BGR2HSV)</code>
<code>COLOR_BGR2LAB</code>	BGR vers LAB	<code>cvtColor(img, cv2.COLOR_BGR2LAB)</code>
<code>COLOR_GRAY2BGR</code>	Gris vers BGR (3 canaux identiques)	<code>cvtColor(gray, cv2.COLOR_GRAY2BGR)</code>
<code>cv2.split()</code>	Séparer les canaux	<code>b, g, r = cv2.split(img)</code>
<code>cv2.merge()</code>	Fusionner les canaux	<code>cv2.merge((b, g, r))</code>
<code>cv2.inRange()</code>	Masque par plage de couleurs (HSV)	<code>cv2.inRange(hsv, lower, upper)</code>

4.5.4 Transformations Géométriques

Définition

Les **transformations géométriques** modifient la position, l'orientation ou la taille des pixels d'une image sans altérer leur valeur. Les principales sont :

- **Redimensionnement** (*resize*) : changement d'échelle.
- **Translation** : déplacement selon (x, y) .
- **Rotation** : autour d'un point pivot, par un angle θ .
- **Affine** : préserve le parallélisme.
- **Perspective** : redresse une image vue en biais.

Utilité

Les transformations géométriques sont utilisées pour :

- **Normaliser** la taille des images en entrée d'un modèle (224×224 pour CNN).
- **Augmenter les données** (*data augmentation*) en générant des rotations et flips pour entraîner des réseaux de neurones.
- **Redresser** des documents scannés ou des photos prises de travers (transformation perspective).
- **Recadrer** une région d'intérêt (ROI) par slicing NumPy.

Fonction	Fonctionnalité	Instruction
<code>cv2.resize()</code>	Redimensionner une image	<code>cv2.resize(img, (300, 200))</code>
<code>INTER_LINEAR</code>	Interpolation bilinéaire (défaut)	<code>resize(img, dim, interpolation=...)</code>
<code>INTER_CUBIC</code>	Interpolation bicubique (qualité)	<code>resize(img, dim, cv2.INTER_CUBIC)</code>
<code>INTER_AREA</code>	Réduction de taille (recommandé)	<code>resize(img, dim, cv2.INTER_AREA)</code>
<code>cv2.flip()</code>	Miroir horizontal/vertical	<code>cv2.flip(img, 1)</code>
<code>cv2.rotate()</code>	Rotation 90/180/270°	<code>cv2.rotate(img, cv2.ROTATE_90_CW)</code>
<code>cv2.getRotationMatrix2D()</code>	Matrice de rotation libre	<code>getRotationMatrix2D(centre, angle, 1.0)</code>
<code>cv2.warpAffine()</code>	Appliquer une transformation affine	<code>cv2.warpAffine(img, M, (w, h))</code>
<code>cv2.getPerspectiveTransform()</code>	Matrice perspective	<code>getPerspectiveTransform(pts1, pts2)</code>
<code>cv2.warpPerspective()</code>	Appliquer une transformation perspective	<code>cv2.warpPerspective(img, M, (w, h))</code>

Fonction	Fonctionnalité	Instruction
<code>img[y1:y2, x1:x2]</code>	Recadrage par slicing NumPy	<code>roi = img[100:300, 50:250]</code>

4.5.5 Filtrage et Lissage

Définition

Le **filtrage** est l'application d'un **noyau de convolution** (*kernel*) sur l'image pour modifier ses pixels selon leurs voisins. Le **lissage** (*smoothing*) est un type de filtrage qui réduit le bruit et atténue les détails fins.

Principaux filtres :

- **Gaussien** : moyenne pondérée, préserve les bords mieux que la moyenne simple.
- **Médian** : remplace par la médiane locale, élimine le bruit sel-poivre.
- **Bilatéral** : lisse les zones homogènes en préservant les contours.

Utilité

Le filtrage est une étape de **prétraitement essentielle** avant la détection de contours ou la segmentation. Un bruit non éliminé produit de **faux contours** et dégrade les performances de toute analyse ultérieure. Le choix du filtre dépend du type de bruit :

- **Bruit gaussien** (capteur) → filtre gaussien.
- **Bruit sel-poivre** (transmission) → filtre médian.
- **Préservation des bords** (médical) → filtre bilatéral.

Fonction	Fonctionnalité	Instruction
<code>cv2.blur()</code>	Moyenne simple (boîte)	<code>cv2.blur(img, (5, 5))</code>
<code>cv2.GaussianBlur()</code>	Lissage gaussien	<code>cv2.GaussianBlur(img, (5,5), 0)</code>
<code>cv2.medianBlur()</code>	Filtre médian (sel-poivre)	<code>cv2.medianBlur(img, 5)</code>
<code>cv2.bilateralFilter</code>	Filtre bilatéral (préserve bords)	<code>cv2.bilateralFilter(img, 9, 75, 75)</code>
<code>cv2.filter2D()</code>	Convolution avec noyau personnalisé	<code>cv2.filter2D(img, -1, kernel)</code>
<code>cv2.boxFilter()</code>	Filtre boîte (avec normalisation)	<code>cv2.boxFilter(img, -1, (5, 5))</code>
<code>cv2.sepFilter2D()</code>	Filtre séparable (rapide)	<code>cv2.sepFilter2D(img, -1, kx, ky)</code>

Fonction	Fonctionnalité	Instruction
<code>cv2.getGaussianKernel</code>	Générer un noyau gaussien 1D	<code>cv2.getGaussianKernel(5, 1.0)</code>
<code>np.ones() / 25</code>	Noyau de moyenne 5×5	<code>np.ones((5,5), np.float32) / 25</code>

4.5.6 Détection de Contours et Gradients

Définition

Un **contour** est une discontinuité brusque d'intensité dans une image, correspondant souvent à la frontière d'un objet. La détection de contours repose sur le calcul du **gradient** : la dérivée spatiale de l'image, qui mesure l'intensité du changement.

Principaux opérateurs :

- **Sobel** : approximation discrète du gradient (horizontal + vertical).
- **Laplacien** : seconde dérivée (détecte les bords dans toutes directions).
- **Canny** : algorithme multi-étapes, référence en détection de contours.

Utilité

La détection de contours est l'**étape clé** qui transforme une image en **représentation structurelle**. Elle est utilisée pour :

- **Reconnaître des formes** (cercles, lignes, polygones).
- **Segmenter** les objets dans une image médicale (tumeurs, organes).
- **Détecter des routes** ou des bâtiments dans l'imagerie satellite.
- Préparer l'image pour la **transformée de Hough** (cercles, droites).

L'algorithme **Canny** est le plus utilisé car il combine lissage gaussien, calcul de gradient, suppression non-maxima et seuillage par hystérésis.

Fonction	Fonctionnalité	Instruction
<code>cv2.Canny()</code>	Détection de contours Canny	<code>cv2.Canny(img, 100, 200)</code>
<code>cv2.Sobel()</code>	Gradient Sobel (X ou Y)	<code>cv2.Sobel(img, cv2.CV_64F, 1, 0, 5)</code>
<code>cv2.Laplacian()</code>	Opérateur Laplacien	<code>cv2.Laplacian(img, cv2.CV_64F)</code>
<code>cv2.Scharr()</code>	Gradient Scharr (plus précis)	<code>cv2.Scharr(img, cv2.CV_64F, 1, 0)</code>
<code>cv2.magnitude()</code>	Magnitude du gradient	<code>cv2.magnitude(gx, gy)</code>

Fonction	Fonctionnalité	Instruction
<code>cv2.phase()</code>	Orientation du gradient	<code>cv2.phase(gx, gy, angleInDegrees=True)</code>
<code>cv2.cornerHarris()</code>	Détection de coins (Harris)	<code>cv2.cornerHarris(gray, 2, 3, 0.04)</code>
<code>cv2.goodFeaturesToTrack()</code>	Coins Shi-Tomasi	<code>goodFeaturesToTrack(gray, 25, 0.01, 10)</code>

4.5.7 Seuillage et Binarisation

Définition

Le **seuillage** (*thresholding*) est une opération qui convertit une image en niveaux de gris en **image binaire** (noir/blanc) en comparant chaque pixel à une valeur seuil T . Plusieurs méthodes existent :

- **Seuillage simple** : seuil fixe global.
- **Seuillage adaptatif** : seuil calculé localement selon le voisinage.
- **Otsu** : seuil optimal automatique calculé par l'histogramme.

Exemple : $\text{pixel} < T \rightarrow 0$ (noir) ; $\text{pixel} \geq T \rightarrow 255$ (blanc).

Utilité

La binarisation simplifie une image complexe en **deux régions** : objet et fond. Elle est indispensable pour :

- **OCR** (reconnaissance optique de caractères) sur documents scannés.
- **Comptage d'objets** (cellules en microscopie, pièces sur tapis).
- **Segmentation médicale** (tumeurs, vaisseaux sanguins).
- Préparation pour les **opérations morphologiques**.

Otsu est la méthode automatique de référence pour les images bimodales (objet clairement distinct du fond).

Fonction	Fonctionnalité	Instruction
<code>cv2.threshold()</code>	Seuillage simple	<code>ret, thr = cv2.threshold(img, 127, 255, 0)</code>
<code>THRESH_BINARY</code>	$\text{pixel} > T : 255$; sinon : 0	<code>cv2.THRESH_BINARY</code>
<code>THRESH_BINARY_INV</code>	Inverse du seuillage binaire	<code>cv2.THRESH_BINARY_INV</code>
<code>THRESH_TRUNC</code>	Tronqué : $\text{pixel} > T : T$	<code>cv2.THRESH_TRUNC</code>

Fonction	Fonctionnalité	Instruction
THRESH_TOZERO	pixel < T : 0 ; sinon inchangé	cv2.THRESH_TOZERO
THRESH_OTSU	Seuil optimal automatique (Otsu)	cv2.THRESH_BINARY + cv2.THRESH_OTSU
cv2.adaptiveThresho	Seuillage adaptatif local	adaptiveThreshold(img, 255, ..., 11, 2)
ADAPTIVE_THRESH_MEAN	Moyenne du voisinage	cv2.ADAPTIVE_THRESH_MEAN_C
ADAPTIVE_THRESH_GAUSSIAN	Gaussienne du voisinage	cv2.ADAPTIVE_THRESH_GAUSSIAN_C

4.5.8 Morphologie Mathématique

Définition

La **morphologie mathématique** regroupe les opérations qui modifient la forme d'objets dans une image binaire à l'aide d'un **élément structurant** (noyau). Les opérations fondamentales sont :

- **Érosion** : amincit les objets, élimine les petits détails.
- **Dilatation** : épaissit les objets, comble les trous.
- **Ouverture** : érosion suivie de dilatation (élimine le bruit blanc).
- **Fermeture** : dilatation suivie d'érosion (comble les trous noirs).

Utilité

La morphologie est appliquée **après le seuillage** pour nettoyer le résultat binaire :

- **Ouverture** : éliminer les petits points blancs parasites.
- **Fermeture** : combler les trous internes des objets segmentés.
- **Gradient morphologique** : extraire le contour d'un objet binaire.
- **Top hat / Black hat** : isoler les détails clairs/sombres.

Indispensable en **imagerie médicale** et **contrôle qualité industriel**.

Fonction	Fonctionnalité	Instruction
cv2.erode()	Érosion	cv2.erode(img, kernel, iterations=1)
cv2.dilate()	Dilatation	cv2.dilate(img, kernel, iterations=1)
cv2.morphologyEx()	Opération morphologique générique	morphologyEx(img, op, kernel)

Fonction	Fonctionnalité	Instruction
MORPH_OPEN	Ouverture (érosion + dilatation)	cv2.MORPH_OPEN
MORPH_CLOSE	Fermeture (dilatation + érosion)	cv2.MORPH_CLOSE
MORPH_GRADIENT	Gradient morphologique (contour)	cv2.MORPH_GRADIENT
MORPH_TOPHAT	Différence (image - ouverture)	cv2.MORPH_TOPHAT
MORPH_BLACKHAT	Différence (fermeture - image)	cv2.MORPH_BLACKHAT
cv2.getStructuringE	Créer un élément structurant	getStructuringElement(MORPH_RECT, (5,5))
MORPH_ELLIPSE	Élément structurant elliptique	cv2.MORPH_ELLIPSE

4.5.9 Détection de Contours et Formes

Définition

La **détection de contours** (*findContours*) extrait les courbes fermées délimitant les objets dans une image binaire. Chaque contour est représenté par une liste de points (x, y) . La **transformée de Hough** détecte parallèlement des formes paramétriques (lignes, cercles) même partiellement masquées.

Exemple : Une image binaire de cellules $\rightarrow$ `findContours` retourne une liste de contours, un par cellule.

Utilité

Les contours permettent une **analyse quantitative** des objets :

- **Compter** les objets segmentés (cellules, pièces).
- Calculer l'**aire**, le **périmètre**, le **centre de masse**.
- Approximer la **forme** (rectangles englobants, cercles inscrits).
- Comparer les formes par **descripteurs de Hu** ou **moments**.

La transformée de Hough est utilisée pour détecter des **lignes droites** (routes, marquages) et des **cercles** (yeux, pupilles, pièces de monnaie).

Fonction	Fonctionnalité	Instruction
cv2.findContours()	Trouver les contours	cnts, h = cv2.findContours(bin, ...)
RETR_EXTERNAL	Contours externes uniquement	cv2.RETR_EXTERNAL
RETR_TREE	Hierarchie complète des contours	cv2.RETR_TREE
CHAIN_APPROX_SIMPLE	Compression des points (rapide)	cv2.CHAIN_APPROX_SIMPLE
cv2.drawContours()	Dessiner les contours	drawContours(img, cnts, -1, (0,255,0), 2)
cv2.contourArea()	Aire d'un contour	cv2.contourArea(cnt)
cv2.arcLength()	Périmètre d'un contour	cv2.arcLength(cnt, True)
cv2.boundingRect()	Rectangle englobant	x,y,w,h = cv2.boundingRect(cnt)
cv2.minEnclosingCircle()	Cercle englobant minimal	(x,y), r = minEnclosingCircle(cnt)
cv2.approxPolyDP()	Approximation polygonale	approxPolyDP(cnt, epsilon, True)
cv2.moments()	Moments du contour	M = cv2.moments(cnt)
cv2.HoughLinesP()	Détection de lignes (Hough probab.)	HoughLinesP(edges, 1, pi/180, 100)
cv2.HoughCircles()	Détection de cercles	HoughCircles(gray, HOUGH_GRADIENT, ...)

4.5.10 Dessin et Annotations

Définition

OpenCV propose un ensemble de fonctions permettant de **dessiner des formes géométriques** (lignes, rectangles, cercles, polygones) et d'**ajouter du texte** directement sur une image. Toutes ces fonctions modifient l'image **en place** et acceptent une couleur au format BGR.

Convention : (B, G, R) avec valeurs entre 0 et 255. Exemple : rouge = (0, 0, 255), vert = (0, 255, 0).

Utilité

Les annotations sont essentielles pour :

- **Visualiser** les résultats d'une détection (boîtes englobantes, points-clés).
- Créer des **interfaces interactives** (sélection de ROI à la souris).
- Générer des **vidéos annotées** pour l'inférence en temps réel (détection d'objets, suivi).
- Produire des **captures de débogage** lors du développement de pipelines.

Fonction	Fonctionnalité	Instruction
<code>cv2.line()</code>	Tracer une ligne	<code>cv2.line(img, p1, p2, (0,255,0), 2)</code>
<code>cv2.rectangle()</code>	Tracer un rectangle	<code>cv2.rectangle(img, p1, p2, color, t)</code>
<code>cv2.circle()</code>	Tracer un cercle	<code>cv2.circle(img, centre, r, color, t)</code>
<code>cv2.ellipse()</code>	Tracer une ellipse	<code>cv2.ellipse(img, c, axes, 0, 0, 360, ...)</code>
<code>cv2.polylines()</code>	Tracer un polygone (non rempli)	<code>cv2.polylines(img, [pts], True, color)</code>
<code>cv2.fillPoly()</code>	Polygone rempli	<code>cv2.fillPoly(img, [pts], color)</code>
<code>cv2.putText()</code>	Écrire du texte sur l'image	<code>putText(img, 'OK', p, font, 1, color, 2)</code>
<code>FONT_HERSHEY_SIMPLEX</code>	Police par défaut	<code>cv2.FONT_HERSHEY_SIMPLEX</code>
<code>cv2.getTextSize()</code>	Taille en pixels d'un texte	<code>(w,h), b = cv2.getTextSize(...)</code>
<code>cv2.arrowedLine()</code>	Tracer une flèche	<code>cv2.arrowedLine(img, p1, p2, color, 2)</code>

4.5.11 Détection d'Objets et Reconnaissance

Définition

La **détection d'objets** consiste à **localiser et classifier** automatiquement des entités visuelles dans une image (visages, voitures, panneaux, etc.). OpenCV propose plusieurs approches :

- **Cascades de Haar** : détecteurs pré-entraînés (visages, yeux, sourires).
- **HOG + SVM** : descripteurs de gradient (piétons).
- **Deep Learning** : module `dnn` pour charger YOLO, SSD, MobileNet.

Exemple : `haarcascade_frontalface_default.xml` détecte les visages frontaux en temps réel.

Utilité

La détection d'objets est utilisée pour :

- **Surveillance vidéo** : détection de personnes, véhicules, intrusions.
- **Conduite autonome** : panneaux, feux, piétons.
- **Reconnaissance biométrique** : visages, iris, empreintes.
- **Imagerie médicale** : détection de lésions, nodules, cellules anormales.
- **Réalité augmentée** : suivi de marqueurs et superposition d'objets 3D.

Le module `dnn` permet d'intégrer n'importe quel modèle pré-entraîné (TensorFlow, Caffe, ONNX) directement dans un pipeline OpenCV.

Fonction	Fonctionnalité	Instruction
<code>cv2.CascadeClassifier()</code>	Charger un classifieur Haar	<code>CascadeClassifier('haarcascade.xml')</code>
<code>detectMultiScale()</code>	Détecter à plusieurs échelles	<code>cascade.detectMultiScale(gray, 1.1, 4)</code>
<code>cv2.HOGDescriptor()</code>	Descripteur HOG	<code>hog = cv2.HOGDescriptor()</code>
<code>hog.detectMultiScale()</code>	Détection de piétons HOG	<code>hog.detectMultiScale(img)</code>
<code>cv2.dnn.readNet()</code>	Charger un modèle deep learning	<code>cv2.dnn.readNet('yolo.weights', 'cfg')</code>
<code>cv2.dnn.blobFromImage()</code>	Préparer l'entrée du réseau	<code>blobFromImage(img, 1/255, (416,416))</code>
<code>net.setInput()</code>	Injecter l'entrée dans le réseau	<code>net.setInput(blob)</code>
<code>net.forward()</code>	Exécuter l'inférence	<code>outputs = net.forward()</code>
<code>cv2.matchTemplate()</code>	Recherche par template	<code>matchTemplate(img, tpl, TM_CCOEFF)</code>
<code>cv2.SIFT_create()</code>	Détecteur SIFT (points-clés)	<code>sift = cv2.SIFT_create()</code>
<code>cv2.ORB_create()</code>	Détecteur ORB (libre, rapide)	<code>orb = cv2.ORB_create()</code>

4.5.12 Traitement Vidéo et Webcam

Définition

OpenCV permet de manipuler des **flux vidéo** provenant de fichiers (MP4, AVI, MOV) ou de **caméras en direct** (webcam, IP cam). L'objet `VideoCapture` encapsule la lecture frame par frame, tandis que `VideoWriter` permet l'écriture d'une vidéo de sortie avec un **codec** spécifié (FOURCC).

Exemple : `cv2.VideoCapture(0)` ouvre la première webcam ; `cv2.VideoCapture('video.mp4')` ouvre un fichier.

Utilité

Le traitement vidéo est essentiel pour :

- **Vidéosurveillance** et détection de mouvement (soustraction de fond).
- **Suivi d'objets** en temps réel (*tracking*).
- **Comptage de personnes** (entrées/sorties magasins).
- **Analyse sportive** (trajectoires, vitesses).
- **Création de timelapses** ou de séquences augmentées.

La gestion des codecs avec `cv2.VideoWriter_fourcc` est cruciale pour garantir la compatibilité des fichiers générés (ex. 'mp4v' pour MP4, 'XVID' pour AVI).

Fonction	Fonctionnalité	Instruction
<code>cv2.VideoCapture()</code>	Ouvrir une vidéo ou webcam	<code>cap = cv2.VideoCapture(0)</code>
<code>cap.isOpened()</code>	Vérifier si la source est ouverte	<code>cap.isOpened()</code>
<code>cap.read()</code>	Lire la frame suivante	<code>ret, frame = cap.read()</code>
<code>cap.get()</code>	Obtenir une propriété (FPS, taille)	<code>cap.get(cv2.CAP_PROP_FPS)</code>
<code>cap.set()</code>	Modifier une propriété	<code>cap.set(cv2.CAP_PROP_POS_FRAMES, 100)</code>
<code>cap.release()</code>	Libérer la ressource	<code>cap.release()</code>
<code>cv2.VideoWriter()</code>	Créer un fichier vidéo en sortie	<code>out = cv2.VideoWriter(...)</code>
<code>cv2.VideoWriter_fourcc</code>	Codec d'encodage	<code>fourcc = VideoWriter_fourcc(*'mp4v')</code>
<code>out.write()</code>	Écrire une frame	<code>out.write(frame)</code>
<code>cv2.createBackgroundSubtractorMOG2()</code>	Soustraction de fond	<code>bg = cv2.createBackgroundSubtractorMOG2()</code>
<code>cv2.calcOpticalFlowPyrLK()</code>	Flot optique (Lucas-Kanade)	<code>calcOpticalFlowPyrLK(prev, next, pts)</code>
<code>cv2.TrackerCSRT_create()</code>	Suivi d'objet (tracker CSRT)	<code>tracker = cv2.TrackerCSRT_create()</code>

4.5.13 Histogrammes et Analyse Statistique

Définition

Un **histogramme** est une représentation graphique de la **distribution des intensités** de pixels dans une image. Pour une image en niveaux de gris, il indique combien de pixels possèdent chaque valeur entre 0 et 255. Pour une image couleur, on calcule un histogramme par canal.

Exemple : Une image sombre a un histogramme concentré sur la gauche ; une image claire à droite ; une image bien contrastée s'étale sur toute la plage.

Utilité

Les histogrammes sont utilisés pour :

- Évaluer la **qualité** d'une image (sous-exposition, sur-exposition).
- **Égaliser** le contraste pour améliorer la visibilité des détails.
- **Comparer** deux images par similarité d'histogramme.
- Calculer le **seuil optimal** (méthode Otsu).
- **Tracker** un objet par sa signature colorimétrique (CamShift).

L'égalisation **CLAHE** (*Contrast Limited Adaptive Histogram Equalization*) est particulièrement utile en **imagerie médicale** pour révéler des détails dans des zones sombres.

Fonction	Fonctionnalité	Instruction
<code>cv2.calcHist()</code>	Calculer l'histogramme	<code>cv2.calcHist([img], [0], None, [256], ...)</code>
<code>cv2.equalizeHist()</code>	Égalisation d'histogramme	<code>cv2.equalizeHist(gray)</code>
<code>cv2.createCLAHE()</code>	CLAHE (égalisation locale)	<code>clahe = cv2.createCLAHE(2.0, (8,8))</code>
<code>clahe.apply()</code>	Appliquer CLAHE	<code>clahe.apply(gray)</code>
<code>cv2.normalize()</code>	Normaliser un tableau	<code>cv2.normalize(img, None, 0, 255, ...)</code>
<code>cv2.compareHist()</code>	Comparer deux histogrammes	<code>compareHist(h1, h2, HISTCMP_CORREL)</code>
<code>cv2.minMaxLoc()</code>	Min, max et leurs positions	<code>min,max,pmin,pmax = minMaxLoc(img)</code>
<code>cv2.meanStdDev()</code>	Moyenne et écart-type	<code>m, s = cv2.meanStdDev(img)</code>
<code>cv2.countNonZero()</code>	Compter pixels non nuls	<code>cv2.countNonZero(binary)</code>

Fonction	Fonctionnalité	Instruction
<code>cv2.calcBackProject</code>	Rétroprojection d'histogramme	<code>calcBackProject([hsv], [0,1], hist, ...)</code>

Principales constantes OpenCV utiles :

Constante	Catégorie	Description
<code>IMREAD_COLOR</code>	Lecture image	Charger en BGR (défaut)
<code>IMREAD_GRAYSCALE</code>	Lecture image	Charger en niveaux de gris
<code>IMREAD_UNCHANGED</code>	Lecture image	Conserver canal alpha
<code>COLOR_BGR2GRAY</code>	Conversion couleur	BGR vers niveaux de gris
<code>COLOR_BGR2HSV</code>	Conversion couleur	BGR vers Teinte-Saturation-Valeur
<code>THRESH_BINARY</code>	Seuillage	Binarisation simple
<code>THRESH_OTSU</code>	Seuillage	Otsu automatique
<code>MORPH_OPEN</code>	Morphologie	Ouverture (érosion + dilatation)
<code>MORPH_CLOSE</code>	Morphologie	Fermeture (dilatation + érosion)
<code>RETR_EXTERNAL</code>	Contours	Contours externes uniquement
<code>CHAIN_APPROX_SIMPLE</code>	Contours	Compression de points
<code>INTER_LINEAR</code>	Interpolation	Bilinéaire (défaut resize)
<code>INTER_AREA</code>	Interpolation	Réduction de taille
<code>FONT_HERSHEY_SIMPLEX</code>	Texte	Police par défaut pour <code>putText</code>
<code>CAP_PROP_FPS</code>	Vidéo	Fréquence d'images
<code>CAP_PROP_FRAME_COUNT</code>	Vidéo	Nombre total de frames

4.6 Comparaison des Outils de Vision

Critere	OpenCV	Pillow	scikit-image
Performance	Tres elevee	Moyenne	Elevee
Video / Camera	Oui (natif)	Non	Non
Detection objets	Oui (Haar, DNN)	Non	Partielle
Espace couleur	BGR (attention !)	RGB	RGB
Format de sortie	ndarray uint8	Objet PIL	ndarray float
Cas d'usage	Production	Manipulation simple	Recherche

4.7 Bonnes Pratiques

- **BGR vs RGB** : Ajoutez systématiquement `cvtColor(BGR2RGB)` avant tout affichage Matplotlib ou passage a TensorFlow/PyTorch.
- **Type** : OpenCV travaille en `uint8` `[0, 255]`. Pour les calculs flottants, convertissez avec `img.astype(np.float32) / 255.0`.

- **Mémoire** : Sur GPU, utilisez `cv2.cuda` pour accélérer les opérations de filtrage.
- **Canny** : Le rapport `seuil_haut / seuil_bas` recommande est entre 2:1 et 3:1.
- **Validité** : Vérifiez que l'image n'est pas `None` après `imread` (chemin incorrect ou format non supporté).

Attention

Le format BGR d'OpenCV est la source d'erreur la plus fréquente chez les débutants. Une image non convertie affichée avec Matplotlib aura les canaux rouge et bleu inversés, donnant un rendu couleur incorrect.

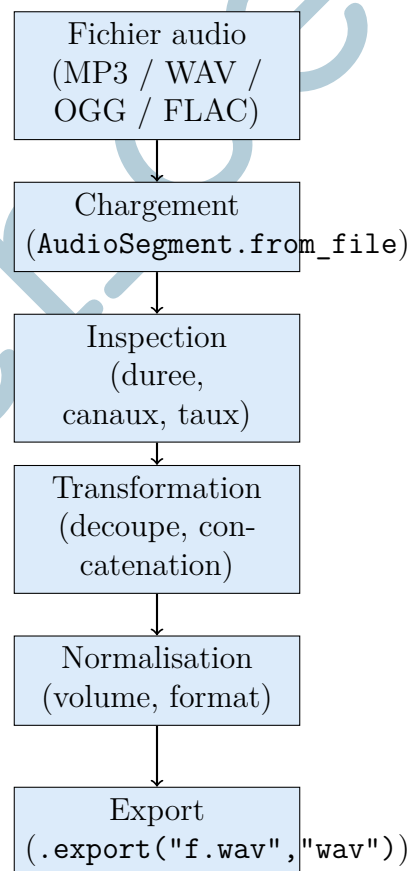
Chapitre 5 Pydub – Traitement Audio

5.1 Principe Fondamental

Pydub est une bibliothèque Python de haut niveau pour la manipulation de fichiers audio. Contrairement à PyAudio (capture temps réel) ou SoundFile (lecture/écriture bas niveau), Pydub offre une interface intuitive basée sur des objets `AudioSegment` qui en-capsulent un signal audio avec toutes ses métadonnées.

L'objet `AudioSegment` permet de découper, concaténer, fusionner, normaliser et convertir des fichiers audio avec une syntaxe très lisible. Pydub s'appuie sur **ffmpeg** pour le décodage des formats compressés (MP3, AAC, OGG).

5.1.1 Architecture de Pydub



5.1.2 Installation

Listing 5.1: Installation de Pydub et de ffmpeg

```

pip install pydub
# ffmpeg est necessaire pour les formats compressees
# Ubuntu/Debian :
sudo apt-get install ffmpeg
# macOS :
brew install ffmpeg
# Windows : telecharger sur https://ffmpeg.org et ajouter au
PATH

```

5.2 Chargement et Inspection

Listing 5.2: Chargement et inspection d'un fichier audio avec Pydub

```

from pydub import AudioSegment
import numpy as np

def inspecter_audio(chemin: str) -> dict:
    """
    Charge un fichier audio et affiche un rapport d'inspection
    complet.

    Proprietes d'un AudioSegment :
    - len(seg)          : duree en millisecondes
    - seg.channels       : nombre de canaux (1=mono,
                          2=stereo)
    - seg.frame_rate    : frequence d'echantillonnage en Hz
    - seg.sample_width  : profondeur en octets (2 = 16 bits)
    - seg.max_dBFS      : niveau de pic en dBFS
    - seg.dBFS          : niveau moyen (RMS) en dBFS

    Args:
        chemin: Chemin vers le fichier audio

    Returns:
        Dictionnaire contenant le segment et ses metadonnees
    """
    seg = AudioSegment.from_file(chemin)

    duree_s = len(seg) / 1000.0
    taille_b = len(seg.raw_data)

    print("=== Inspection audio ===")
    print(f"Fichier          : {chemin}")
    print(f"Duree           : {duree_s:.3f}s
          ({duree_s/60:.1f} min)")
    print(f"Canaux          : {seg.channels} ({'mono' if
          seg.channels==1 else 'stereo'})")
    print(f"Frequence       : {seg.frame_rate} Hz")
    print(f"Profondeur      : {seg.sample_width * 8} bits")
    print(f"Niveau moyen    : {seg.dBFS:.1f} dBFS")

```

```

print(f"Niveau de pic      : {seg.max_dBFS:.1f} dBFS")
print(f"Taille brute      : {taille_b / 1024:.1f} Ko")
print(f"Taille normalise  : {taille_b / (duree_s *
    seg.frame_rate):.2f} octets/s")

# Conversion en array NumPy
samples = np.array(seg.get_array_of_samples(), dtype=float)
if seg.channels == 2:
    samples = samples.reshape((-1, 2))

rms = np.sqrt(np.mean(samples**2))
print(f"RMS (lineaire)    : {rms:.1f}")

return {
    'segment'      : seg,
    'duree_s'      : duree_s,
    'canaux'       : seg.channels,
    'frame_rate'   : seg.frame_rate,
    'sample_width' : seg.sample_width,
    'dBFS'         : seg.dBFS,
    'max_dBFS'     : seg.max_dBFS,
    'samples'      : samples
}

info = inspector_audio("cours_enregistre.mp3")
seg = info['segment']

```

5.3 Découpe et Concaténation

Listing 5.3: Decoupe, concatenation et fondus enchaines

```

from pydub import AudioSegment
from pydub.silence import split_on_silence, detect_silence

def decouper_segments(
    seg: AudioSegment,
    debut_ms: int,
    fin_ms: int,
    fondu_ms: int = 100
) -> AudioSegment:
    """
    Extrait un segment audio avec fondus d'entree et de sortie.

    Args:
        seg      : AudioSegment source
        debut_ms : Debut de l'extrait en millisecondes
        fin_ms   : Fin de l'extrait en millisecondes
        fondu_ms : Duree des fondus enchainas (fade in / fade
                    out)
    """

```



```

Returns:
    AudioSegment extrait avec fondus
"""
extrait = seg[debut_ms:fin_ms]

# Fondu entrant (fade in)
extrait = extrait.fade_in(fondu_ms)

# Fondu sortant (fade out)
extrait = extrait.fade_out(fondu_ms)

print(f"Extrait : {debut_ms}ms -> {fin_ms}ms
      ({(fin_ms-debut_ms)/1000:.2f}s)")
print(f"Niveau apres decoupe : {extrait.dBFS:.1f} dBFS")

return extrait

def segmenter_sur_silences(
    seg: AudioSegment,
    min_silence_ms: int = 700,
    seuil_dBFS: int = -40,
    garder_silence_ms: int = 200
) -> list:
    """
    Segmente un enregistrement en decoupant sur les silences.
    Utile pour diviser un cours en segments de parole.

    Args:
        seg : AudioSegment source
        min_silence_ms : Duree minimale d'un silence valide
                       (ms)
        seuil_dBFS : Niveau en dessous duquel on considere
                   le silence
        garder_silence_ms: Garde-marge autour des silences
                           detectes

    Returns:
        Liste d'AudioSegment (segments de parole)
    """
    print("Segmentation en cours...")

    segments = split_on_silence(
        seg,
        min_silence_len=min_silence_ms,
        silence_thresh=seuil_dBFS,
        keep_silence=garder_silence_ms
    )

    print(f"Segments detectes : {len(segments)}")
    for i, s in enumerate(segments):

```

```

        print(f"    [{i:03d}] {len(s)/1000:.2f}s | {s.dBFS:.1f}
              dBFS")

    return segments

def concatener_segments(
    segments: list,
    pause_ms: int = 300
) -> AudioSegment:
    """
    Concatene plusieurs segments avec une pause entre chacun.

    Args:
        segments: Liste d'AudioSegment
        pause_ms: Duree de la pause en millisecondes

    Returns:
        AudioSegment concatene
    """
    pause = AudioSegment.silent(duration=pause_ms)
    resultat = segments[0]

    for seg in segments[1:]:
        resultat = resultat + pause + seg

    print(f"Concatenation : {len(segments)} segments ->
          {len(resultat)/1000:.2f}s total")
    return resultat

# Exemple
extrait = decouper_segments(seg, debut_ms=0, fin_ms=30000)
segments = segmenter_sur_silences(seg)
complet = concatener_segments(segments[:5])

```

5.4 Normalisation et Conversion de Format

Listing 5.4: Normalisation du volume et conversion multi-format

```

from pydub import AudioSegment
from pydub.effects import normalize, compress_dynamic_range

def normaliser_audio(
    seg: AudioSegment,
    niveau_cible_dBFS: float = -20.0,
    methode: str = 'peak'
) -> AudioSegment:
    """
    Normalise le volume d'un segment audio.

    Methodes disponibles :

```

```

- 'peak' : normalisation au pic (simple, rapide)
- 'rms' : normalisation RMS vers un niveau cible en
          dBFS
- 'lufs' : normalisation LUFS (standard broadcast EBU
          R128)
          (necessite pyloudnorm en complement)

Args:
    seg : AudioSegment source
    niveau_cible_dBFS : Niveau RMS cible (ex: -20 dBFS =
        broadcast standard)
    methode : 'peak' ou 'rms'

Returns:
    AudioSegment normalise
"""
niveau_avant = seg.dBFS
print(f"Niveau avant : {niveau_avant:.1f} dBFS")

if methode == 'peak':
    seg_norm = normalize(seg)
elif methode == 'rms':
    diff_dB = niveau_cible_dBFS - seg.dBFS
    seg_norm = seg + diff_dB
else:
    seg_norm = normalize(seg)

print(f"Niveau apres : {seg_norm.dBFS:.1f} dBFS")
print(f"Gain applique : {seg_norm.dBFS -
    niveau_avant:+.1f} dB")

return seg_norm

def convertir_et_exporter(
    seg: AudioSegment,
    nom_sortie: str,
    format_sortie: str = "wav",
    taux_cible: int = None,
    canaux_cibles: int = None
) -> str:
    """
    Convertit un AudioSegment vers le format et les parametres
    cibles.

    Formats supportes : wav, mp3, ogg, flac, aac

    Args:
        taux_cible : Frequence d'echantillonnage cible (None
            = conserver)
        canaux_cibles: Nombre de canaux cibles (1=mono,
            2=stereo, None=conserver)

```

```

Returns:
    Chemin du fichier exporte
"""
seg_conv = seg

# Reechantillonnage si necessaire
if taux_cible is not None and taux_cible != seg.frame_rate:
    seg_conv = seg_conv.set_frame_rate(taux_cible)
    print(f"Reechantillonnage : {seg.frame_rate} -> {taux_cible} Hz")

# Conversion mono/stereo
if canaux_cibles is not None and canaux_cibles != seg.channels:
    seg_conv = seg_conv.set_channels(canaux_cibles)
    print(f"Canaux : {seg.channels} -> {canaux_cibles}")

# Export
params = {}
if format_sortie == "mp3":
    params = {"bitrate": "192k"}

seg_conv.export(nom_sortie, format=format_sortie, **params)

import os
taille = os.path.getsize(nom_sortie) / 1024
print(f"Exporte : {nom_sortie} ({format_sortie.upper()}, {taille:.1f} Ko)")
return nom_sortie

# Exemple : normaliser et exporter en WAV 16 kHz mono pour ASR
seg_norm = normaliser_audio(seg, niveau_cible_dBFS=-20.0,
                             methode='rms')
convertir_et_exporter(seg_norm, "pour_whisper.wav", "wav",
                      taux_cible=16000, canaux_cibles=1)

```

5.5 Fonctions Pydub les plus utilisées

5.5.1 Installation et Configuration

Définition

Pydub est une bibliothèque Python open-source de **haut niveau** pour la manipulation audio. Elle s'appuie sur **FFmpeg** (ou libav) comme moteur sous-jacent pour gérer plus de 30 formats audio (MP3, WAV, OGG, FLAC, M4A, AAC). Sans FFmpeg installé, seul le format WAV non compressé est lisible nativement. L'installation requiert donc deux étapes : le paquet Python et le binaire FFmpeg.

Utilité

La configuration initiale est cruciale pour pouvoir manipuler des fichiers compressés. Sur Ubuntu/Debian, FFmpeg s'installe via `apt` ; sur Windows, il faut télécharger les binaires et les ajouter au PATH système. Sans cette étape, toute tentative de chargement d'un MP3 lèvera une erreur `CouldntDecodeError`. Pydub s'intègre nativement avec NumPy pour les analyses spectrales avancées via `get_array_of_samples()`.

Fonction	Fonctionnalité	Instruction
<code>pip install</code>	Installer Pydub	<code>pip install pydub</code>
<code>apt install ffmpeg</code>	Installer FFmpeg (Ubuntu/Debian)	<code>sudo apt install ffmpeg</code>
<code>from pydub import</code>	Importer la classe principale	<code>from pydub import AudioSegment</code>
<code>from pydub.playback</code>	Module de lecture	<code>from pydub.playback import play</code>
<code>from pydub.silence</code>	Module de détection de silence	<code>from pydub.silence import split_on_silence</code>
<code>from pydub.effects</code>	Module d'effets audio	<code>from pydub.effects import normalize</code>
<code>AudioSegment.converter</code>	Définir le chemin FFmpeg manuellement	<code>AudioSegment.converter = "/path/ffmpeg"</code>

5.5.2 Chargement et Sauvegarde

Définition

Le **chargement** consiste à lire un fichier audio depuis le disque et à le représenter en mémoire sous forme d'objet `AudioSegment`. La **sauvegarde** (export) écrit cet objet vers un fichier d'un format spécifié, avec conversion automatique via FFmpeg.

Exemple : `AudioSegment.from_mp3("voix.mp3")` charge un MP3, puis `.export("voix.wav", format="wav")` le convertit en WAV.

Utilité

Pydub permet de **convertir entre formats** sans intervention manuelle : MP3 vers WAV pour analyse, FLAC vers OGG pour streaming, M4A vers MP3 pour compatibilité. Cette flexibilité est essentielle dans tout pipeline multimodal qui doit gérer des sources audio hétérogènes (enregistrements vocaux, podcasts, extraits de vidéos). L'option `bitrate` de `export()` contrôle la qualité du fichier de sortie.

Fonction	Fonctionnalité	Instruction
<code>AudioSegment.from_file()</code>	Charger un fichier audio (auto)	<code>audio = AudioSegment.from_file("a.mp3")</code>
<code>AudioSegment.from_mp3()</code>	Charger spécifiquement un MP3	<code>AudioSegment.from_mp3("audio.mp3")</code>
<code>AudioSegment.from_wav()</code>	Charger un WAV	<code>AudioSegment.from_wav("audio.wav")</code>
<code>AudioSegment.from_ogg()</code>	Charger un OGG	<code>AudioSegment.from_ogg("audio.ogg")</code>
<code>AudioSegment.from_flac()</code>	Charger un FLAC	<code>AudioSegment.from_flac("audio.flac")</code>
<code>AudioSegment.empty()</code>	Créer un segment vide	<code>AudioSegment.empty()</code>
<code>AudioSegment.silent()</code>	Créer un silence (durée en ms)	<code>AudioSegment.silent(duration=1000)</code>
<code>audio.export()</code>	Exporter dans un format	<code>audio.export("out.wav", format="wav")</code>
<code>export(bitrate=)</code>	Exporter avec bitrate spécifique	<code>audio.export("out.mp3", bitrate="192k")</code>
<code>export(tags=)</code>	Ajouter des métadonnées ID3	<code>audio.export(..., tags={"artist": "X"})</code>

5.5.3 Propriétés du Signal Audio

Définition

Un objet `AudioSegment` possède plusieurs **attributs** qui décrivent les caractéristiques techniques du signal :

- **frame_rate** : fréquence d'échantillonnage (Hz), ex. 44100 pour CD.
- **channels** : nombre de canaux (1 = mono, 2 = stéréo).
- **sample_width** : profondeur en octets (1 = 8 bits, 2 = 16 bits).
- **duration_seconds** : durée totale en secondes.
- **dBFS** : niveau moyen relatif au maximum (*decibels Full Scale*).

Utilité

La connaissance des propriétés est indispensable pour :

- **Vérifier la qualité** d'un enregistrement (44.1 kHz vs 8 kHz).
- **Uniformiser un corpus** audio (tous les fichiers en mono 16 kHz).

- Calculer la **taille mémoire** avant traitement (risque de saturation).
- Mesurer le **volume effectif** via dBFS (-20 dBFS est typique).
- Préparer les données pour **SpeechRecognition** ou **Whisper** qui exigent souvent du WAV mono 16 kHz.

Fonction	Fonctionnalité	Instruction
<code>audio.duration_seconds</code>	Durée en secondes	<code>print(audio.duration_seconds)</code>
<code>len(audio)</code>	Durée en millisecondes	<code>print(len(audio))</code>
<code>audio.frame_rate</code>	Fréquence d'échantillonnage (Hz)	<code>audio.frame_rate</code>
<code>audio.channels</code>	Nombre de canaux (1 ou 2)	<code>audio.channels</code>
<code>audio.sample_width</code>	Largeur d'échantillon (octets)	<code>audio.sample_width</code>
<code>audio.frame_count()</code>	Nombre total d'échantillons	<code>audio.frame_count()</code>
<code>audio.dBFS</code>	Niveau moyen en dBFS	<code>audio.dBFS</code>
<code>audio.max_dBFS</code>	Niveau maximum en dBFS	<code>audio.max_dBFS</code>
<code>audio.rms</code>	Valeur efficace (RMS)	<code>audio.rms</code>
<code>audio.max</code>	Amplitude maximale	<code>audio.max</code>

5.5.4 Découpage Temporel et Slicing

Définition

Pydub utilise une **syntaxe NumPy-like** pour le découpage temporel : les indices sont exprimés en **millisecondes** et le slicing standard de Python fonctionne directement sur l'objet `AudioSegment`. Cette approche rend extrêmement intuitive l'extraction de portions précises d'un signal.

Exemple : `audio[5000:10000]` extrait les secondes 5 à 10 de l'audio. `audio[-3000:]` récupère les 3 dernières secondes.

Utilité

Le slicing temporel est fondamental pour :

- **Extraire un extrait précis** (refrain d'une chanson, citation).
- Découper un long enregistrement en **segments fixes** (ex. tronçons de 30 secondes pour transcription).
- Supprimer les **silences en début/fin** d'enregistrement.
- Créer un **dataset audio** à partir d'un fichier source unique.
- Comparer deux portions d'un même fichier (refrain vs couplet).

La cohérence avec NumPy facilite l'apprentissage pour tout développeur Python.

Fonction	Fonctionnalité	Instruction
<code>audio[start:end]</code>	Slicing en millisecondes	<code>extrait = audio[1000:5000]</code>
<code>audio[:n]</code>	Garder les n premières ms	<code>debut = audio[:5000]</code>
<code>audio[n:]</code>	À partir de la ms n	<code>reste = audio[10000:]</code>
<code>audio[-n:]</code>	Garder les n dernières ms	<code>fin = audio[-3000:]</code>
<code>audio[:,n]</code>	Échantillonner toutes les n ms	<code>audio[:,2]</code>
<code>audio.reverse()</code>	Inverser le segment temporellement	<code>audio.reverse()</code>
<code>audio.get_sample_slice()</code>	Slicing par index d'échantillon	<code>audio.get_sample_slice(0, 44100)</code>
1000	Constante : 1 seconde en ms	<code>audio[:1000]</code> # 1 seconde
60 * 1000	Constante : 1 minute en ms	<code>audio[:60*1000]</code>

5.5.5 Concaténation, Mixage et Overlay

Définition

Pydub permet de **combiner plusieurs segments audio** grâce à la surcharge des opérateurs Python :

- **Concaténation (+)** : place deux segments l'un après l'autre.
- **Répétition (*)** : duplique un segment n fois.
- **Overlay** : superpose deux segments simultanément (mixage).
- **Append** : ajoute avec un crossfade (transition douce).

Exemple : `intro + couplet + refrain * 2 + outro` construit une chanson complète en une ligne.

Utilité

Ces opérations sont essentielles pour :

- **Assembler des podcasts** : intro + jingle + contenu + outro.
- Créer des **compilations musicales** ou des **mashups**.
- Ajouter une **musique de fond** sous une voix off (overlay).
- Construire des **datasets d'augmentation** en ajoutant du bruit ambiant à des enregistrements vocaux propres.
- Concaténer plusieurs **prises de voix** en un fichier unique.

Fonction	Fonctionna	Instruction
<code>audio1 + audio2</code>	Concaténer deux segments	<code>combine = audio1 + audio2</code>
<code>audio * n</code>	Répéter un segment n fois	<code>boucle = audio * 3</code>
<code>audio.append()</code>	Ajouter avec crossfade	<code>a1.append(a2, crossfade=1500)</code>
<code>audio.overlay()</code>	Superposer deux segments	<code>musique.overlay(voix)</code>
<code>overlay(position=)</code>	Overlay à un instant précis	<code>a1.overlay(a2, position=5000)</code>
<code>overlay(loop=True)</code>	Overlay en boucle	<code>a1.overlay(a2, loop=True)</code>
<code>overlay(times=)</code>	Overlay répété n fois	<code>a1.overlay(a2, times=3)</code>
<code>overlay(gain_during_overlay=)</code>	Atténuer pendant overlay	<code>overlay(a2, gain_during_overlay=-6)</code>
<code>audio.split_to_mono()</code>	Séparer canaux stéréo	<code>left, right = stereo.split_to_mono()</code>
<code>AudioSegment.from_mono_audiosegments()</code>	Fusionner mono → stéréo	<code>AudioSegment.from_mono_audiosegments(L, R)</code>

5.5.6 Volume et Normalisation

Définition

Le **volume** d'un signal audio est exprimé en **décibels** (dB), échelle logarithmique. Pydub permet d'ajuster le gain via la surcharge des opérateurs + et - appliqués à un nombre :

- `audio + 6` : amplifie de 6 dB ($\times 2$ en amplitude).
- `audio - 3` : atténue de 3 dB.

La **normalisation** ramène le pic d'amplitude à un niveau cible (par défaut 0 dBFS), maximisant le volume sans saturation.

Utilité

La gestion du volume est cruciale pour :

- **Uniformiser un corpus** où certains fichiers sont trop faibles ou trop forts.
- **Préparer pour la reconnaissance vocale** : un signal trop faible donne de mauvaises transcriptions.
- Éviter le **clipping** (saturation) lors du mixage de plusieurs sources.
- **Réduire dynamiquement** la musique de fond pour mettre en avant une voix.
- **Compresser dynamiquement** pour homogénéiser un podcast.

Attention : normaliser amplifie aussi le bruit de fond.

Fonction	Fonctionnalité	Instruction
<code>audio + n</code>	Augmenter le gain de n dB	<code>plus_fort = audio + 6</code>
<code>audio - n</code>	Diminuer le gain de n dB	<code>plus_doux = audio - 3</code>
<code>audio.apply_gain()</code>	Appliquer un gain explicite (dB)	<code>audio.apply_gain(-10)</code>
<code>normalize()</code>	Normaliser au pic maximum	<code>from pydub.effects import normalize</code>
<code>normalize(headroom=)</code>	Normaliser avec marge (dB)	<code>normalize(audio, headroom=1.0)</code>
<code>compress_dynamic_range()</code>	Compresseur dynamique	<code>compress_dynamic_range(audio)</code>
<code>audio.apply_gain_stereo()</code>	Gain par canal (L/R)	<code>audio.apply_gain_stereo(-3, +3)</code>
<code>audio.pan()</code>	Panoramique (-1 = G, +1 = D)	<code>audio.pan(-0.5)</code>
<code>audio.invert_phase()</code>	Inverser la phase	<code>audio.invert_phase()</code>

5.5.7 Fondus et Effets Audio

Définition

Les **fondus** (*fades*) sont des transitions progressives de volume :

- **Fade in** : montée progressive du silence vers le volume normal.
- **Fade out** : descente progressive vers le silence.
- **Crossfade** : fondu enchaîné entre deux segments (le premier diminue pendant que le second augmente).

Les **effets** modifient le signal pour créer des variations artistiques ou techniques (vitesse, hauteur, inversion).

Utilité

Les fondus et effets sont indispensables pour :

- Éviter les **clics audibles** en début/fin de segment (transitions douces).
- Créer des **transitions élégantes** entre morceaux de musique.
- **Accélérer un podcast** sans modifier la hauteur de la voix (**speedup**).
- Produire des **effets spéciaux** (lecture inversée, ralenti).
- Respecter les **normes de diffusion** radio/streaming (fade out de 3 secondes en fin).

Fonction	Fonctionnalité	Instruction
<code>audio.fade_in()</code>	Fondu en entrée (durée en ms)	<code>audio.fade_in(2000)</code>
<code>audio.fade_out()</code>	Fondu en sortie	<code>audio.fade_out(3000)</code>
<code>audio.fade()</code>	Fondu personnalisé (gain, position)	<code>audio.fade(to_gain=-120, start=0, ...)</code>
<code>audio.append(crossfade=)</code>	Crossfade entre segments	<code>a1.append(a2, crossfade=1500)</code>
<code>audio.reverse()</code>	Lecture inversée	<code>audio.reverse()</code>
<code>speedup()</code>	Accélérer sans changer la hauteur	<code>from pydub.effects import speedup</code>
<code>speedup(playback_speed=)</code>	Vitesse multiplicative	<code>speedup(audio, playback_speed=1.5)</code>
<code>audio._spawn()</code>	Modifier la fréquence (change pitch)	<code>audio._spawn(audio.raw_data, ...)</code>
<code>low_pass_filter()</code>	Filtre passe-bas (Hz)	<code>audio.low_pass_filter(3000)</code>
<code>high_pass_filter()</code>	Filtre passe-haut (Hz)	<code>audio.high_pass_filter(80)</code>

<code>band_pass_filter()</code>	Filtre passe-bande	<code>audio.band_pass_filter(300, 3400)</code>
---------------------------------	-----------------------	--

5.5.8 Détection de Silence et Segmentation

Définition

La **détection de silence** identifie les portions d'un signal audio dont le niveau est inférieur à un seuil (en dBFS) pendant une durée minimale. La **segmentation** découpe automatiquement un fichier audio long en multiples segments en utilisant ces silences comme séparateurs.

Paramètres clés :

- **min_silence_len** : durée minimale d'un silence (ms).
- **silence_thresh** : seuil de détection (dBFS, ex. -40).
- **keep_silence** : conserver une marge de silence autour des segments.

Utilité

La détection de silence est utilisée pour :

- **Découper automatiquement un podcast** en chapitres ou interventions.
- Créer un **dataset de phrases** à partir d'un long enregistrement vocal (entraînement de modèles ASR).
- Supprimer les **pauses excessives** dans une interview.
- Détecter le **début effectif** d'une chanson (silence initial).
- **Optimiser la taille** d'un fichier en supprimant les blancs.

C'est l'une des fonctionnalités les plus puissantes de Pydub pour le pipeline multi-modal vocal.

Fonction	Fonctionnalité	Instruction
<code>split_on_silence()</code>	Découper aux silences	<code>from pydub.silence import split_on_silence</code>
<code>min_silence_len=</code>	Durée minimale d'un silence (ms)	<code>split_on_silence(audio, min_silence_len=500)</code>
<code>silence_thresh=</code>	Seuil de silence (dBFS)	<code>split_on_silence(audio, silence_thresh=-40)</code>
<code>keep_silence=</code>	Garder marge de silence	<code>split_on_silence(audio, keep_silence=200)</code>
<code>detect_silence()</code>	Localiser les silences (intervalles)	<code>detect_silence(audio, 1000, -40)</code>

Fonction	Fonctionnalité	Instruction
<code>detect_nonsilent()</code>	Localiser les portions sonores	<code>detect_nonsilent(audio, 1000, -40)</code>
<code>detect_leading_silence()</code>	Détecter le silence de tête	<code>detect_leading_silence(audio)</code>
<code>audio.strip_silence()</code>	Supprimer silences début/fin	<code>from pydub.effects import strip_silence</code>
<code>strip_silence(silence)</code>	Avec durée min de silence	<code>strip_silence(audio, silence_len=500)</code>

5.5.9 Conversion de Formats et Paramètres

Définition

Pydub permet de modifier les **paramètres techniques** d'un signal audio : fréquence d'échantillonnage, nombre de canaux et profondeur de bits. Ces opérations sont essentielles pour l'**interopérabilité** avec d'autres outils ou pour réduire la taille des fichiers.

Exemples typiques :

- Téléphone : 8 kHz mono 8 bits (qualité voix).
- Voix qualité radio : 16 kHz mono 16 bits.
- CD audio : 44.1 kHz stéréo 16 bits.
- Studio : 96 kHz stéréo 24 bits.

Utilité

La conversion de paramètres est nécessaire pour :

- **Préparer pour l'ASR** : Whisper et SpeechRecognition exigent souvent du WAV mono 16 kHz.
- **Réduire la taille** d'un corpus en passant de stéréo à mono.
- **Ré-échantillonner** pour respecter le théorème de Nyquist.
- Adapter aux **contraintes hardware** (capteurs, microcontrôleurs).
- **Normaliser un dataset** hétérogène avant entraînement de modèles.

Fonction	Fonctionnalité	Instruction
<code>audio.set_frame_rate()</code>	Modifier la fréquence (Hz)	<code>audio.set_frame_rate(16000)</code>
<code>audio.set_channels()</code>	Modifier nombre de canaux	<code>audio.set_channels(1)</code>

<code>audio.set_sample_width()</code>	Modifier la profondeur (octets)	<code>audio.set_sample_width(2)</code>
<code>export(format="wav")</code>	Convertir en WAV	<code>audio.export("out.wav", format="wav")</code>
<code>export(format="mp3")</code>	Convertir en MP3	<code>audio.export("out.mp3", format="mp3")</code>
<code>export(format="ogg")</code>	Convertir en OGG Vorbis	<code>audio.export("out.ogg", format="ogg")</code>
<code>export(format="flac")</code>	Convertir en FLAC (sans perte)	<code>audio.export("out.flac", format="flac")</code>
<code>export(parameters=)</code>	Paramètres FFmpeg avancés	<code>audio.export(..., parameters=["-ac", "1"])</code>
<code>export(codec=)</code>	Codec d'encodage spécifique	<code>audio.export(..., codec="libmp3lame")</code>

5.5.10 Lecture et Reproduction Audio

Définition

Pydub fournit le module `playback` pour lire un **AudioSegment** directement depuis Python sans avoir à exporter le fichier sur disque. La lecture s'appuie sur des bibliothèques tierces (`simpleaudio`, `pyaudio`, `ffplay`) selon ce qui est disponible sur le système.

Exemple : `from pydub.playback import play ; play(audio)` joue le segment audio en bloquant l'exécution.

Utilité

La lecture en direct est utile pour :

- **Vérifier rapidement** le résultat d'une transformation (filtre, gain, découpage).
- Créer des **notebooks Jupyter interactifs** avec écoute des extraits.
- Construire des **interfaces de prévisualisation** (sélection de pages dans un éditeur).
- **Démontrer** une fonctionnalité lors d'une présentation pédagogique.

Pour la production, on préférera l'export vers un fichier suivi d'une lecture externe (lecteur multimédia, navigateur web).

Fonction	Fonctionnalité	Instruction
<code>play()</code>	Jouer un AudioSegment	<code>from pydub.playback import play</code>
<code>play(audio)</code>	Lecture bloquante	<code>play(audio)</code>
<code>_play_with_simpleaudio()</code>	Backend simpleaudio	<code>from pydub.playback import _play_with_simpleaudio</code>

<code>_play_with_ffplay()</code>	Backend ffplay	<code>from pydub.playback import _play_with_ffplay</code>
<code>_play_with_pyaudio()</code>	Backend pyaudio	<code>from pydub.playback import _play_with_pyaudio</code>
<code>IPython.display.Audio</code>	Lecture dans Jupyter	<code>from IPython.display import Audio</code>
<code>Audio(data=, rate=)</code>	Affichage dans notebook	<code>Audio(audio.raw_data, rate=audio.frame_rate)</code>

5.5.11 Génération Audio et Tonalités

Définition

Pydub permet de **générer des signaux audio synthétiques** via le module **generators**. Les générateurs principaux produisent des formes d'onde mathématiques pures :

- **Sine** : onde sinusoïdale (son pur).
- **Square** : onde carrée (son riche en harmoniques).
- **Sawtooth** : onde en dents de scie.
- **Triangle** : onde triangulaire (son doux).
- **WhiteNoise** : bruit blanc (toutes fréquences).

Exemple : `Sine(440).to_audio_segment(1000)` génère un La (440 Hz) d'une seconde.

Utilité

La génération de signaux synthétiques est utile pour :

- Créer des **tests audio** (calibration de chaîne, vérification de filtres).
- Générer des **bips de notification** ou des **jingles** simples.
- Produire du **bruit de fond** pour augmentation de données (data augmentation en deep learning).
- **Enseigner les principes** du traitement du signal (synthèse additive).
- Construire des **exercices pédagogiques** sur les fréquences musicales.

Fonction	Fonctionnalité	Instruction
<code>from pydub.generators</code>	Importer les générateurs	<code>from pydub.generators import Sine</code>
<code>Sine(freq)</code>	Onde sinusoïdale	<code>Sine(440)</code>
<code>Square(freq)</code>	Onde carrée	<code>Square(220)</code>

Fonction	Fonctionnalité	Instruction
Sawtooth(freq)	Onde en dents de scie	Sawtooth(330)
Triangle(freq)	Onde triangulaire	Triangle(440)
WhiteNoise()	Bruit blanc	WhiteNoise()
Pulse(freq, duty_cycle=)	Onde pulsée	Pulse(440, duty_cycle=0.25)
.to_audio_segment()	Convertir générateur en segment	Sine(440).to_audio_segment(2000)
AudioSegment.silent	Générer un silence	AudioSegment.silent(duration=500)
Sine(440).to_audio_segment	Avec volume initial	Sine(440).to_audio_segment(1000, volume=-10)

5.5.12 Accès aux Données Brutes et Intégration NumPy

Définition

Pydub expose les **échantillons bruts** d'un signal audio sous plusieurs formats, ce qui permet une intégration directe avec **NumPy**, **SciPy** et **Librosa** pour des analyses spectrales avancées (FFT, spectrogrammes, MFCC). L'objet **AudioSegment** stocke les données dans **raw_data** (bytes) et expose **get_array_of_samples()** pour obtenir un tableau Python natif convertible en **ndarray**.

Utilité

L'accès aux données brutes ouvre Pydub vers le **traitement avancé du signal** :

- Calculer la **FFT** (transformée de Fourier rapide) avec NumPy/SciPy.
- Générer des **spectrogrammes** avec Matplotlib (`plt.specgram`).
- Extraire des **MFCC** (coefficients cepstraux) avec Librosa pour la reconnaissance vocale.
- Appliquer des **filtres custom** (passe-bande, débruitage) via SciPy.
- Préparer des **tenseurs PyTorch/TensorFlow** pour le deep learning audio.

Pydub joue ainsi le rôle de **passerelle haut niveau** avant l'analyse fine du signal.

Fonction	Fonctionnalité	Instruction
audio.raw_data	Données brutes (bytes)	<code>raw = audio.raw_data</code>
get_array_of_samples	Tableau Python d'échantillons	<code>samples = audio.get_array_of_samples()</code>

Fonction	Fonctionnalité	Instruction
<code>np.array()</code>	Convertir en ndarray NumPy	<code>np.array(audio.get_array_of_samples())</code>
<code>np.frombuffer()</code>	Depuis raw_data	<code>np.frombuffer(audio.raw_data, np.int16)</code>
<code>audio._data</code>	Attribut interne (bytes)	<code>audio._data</code>
<code>AudioSegment(data=)</code>	Créer depuis bytes bruts	<code>AudioSegment(data=raw, sample_width=2, ...)</code>
<code>scipy.fft.fft()</code>	Transformée de Fourier	<code>scipy.fft.fft(samples)</code>
<code>plt.specgram()</code>	Spectrogramme via Matplotlib	<code>plt.specgram(samples, Fs=audio.frame_rate)</code>
<code>librosa.feature.mfcc()</code>	Coefficients MFCC	<code>librosa.feature.mfcc(y=samples, sr=sr)</code>
<code>scipy.io.wavfile.write()</code>	Écrire un WAV depuis ndarray	<code>wavfile.write("o.wav", sr, samples)</code>

Principaux paramètres et formats Pydub :

Paramètre / Format	Catégorie	Description / Valeur typique
8000 Hz	Fréquence	Qualité téléphone (voix)
16000 Hz	Fréquence	Standard ASR (Whisper, SpeechRecognition)
22050 Hz	Fréquence	Demi-CD (musique légère)
44100 Hz	Fréquence	Qualité CD audio (standard musique)
48000 Hz	Fréquence	Qualité vidéo professionnelle
96000 Hz	Fréquence	Qualité studio (haute résolution)
1	Channels	Mono (1 canal)
2	Channels	Stéréo (2 canaux)
1 (8 bits)	Sample width	Faible qualité (bruit quantification)
2 (16 bits)	Sample width	Standard CD (recommandé)
3 (24 bits)	Sample width	Studio professionnel
4 (32 bits)	Sample width	Float, post-production
0 dBFS	Niveau	Amplitude maximale (saturation)
-3 dBFS	Niveau	Marge de sécurité (broadcast)
-20 dBFS	Niveau	Niveau moyen typique d'une voix
wav, mp3, ogg, flac	Formats principaux	Supportés via FFmpeg
m4a, aac, wma	Formats supplémentaires	Nécessitent FFmpeg avec codecs
128k, 192k, 320k	Bitrate MP3	Qualité croissante (vs taille fichier)

5.6 Comparaison des Outils Audio

Critère	Pydub	librosa	SoundFile
Niveau	Haut niveau	Analyse avancée	Bas niveau
Formats supportés	Via ffmpeg	Via soundfile	WAV, FLAC, OGG
Découpe / concat.	Oui (natif)	Non	Non
MFCC / Spectrogramme	Non	Oui (natif)	Non
Export mp3	Oui	Via Pydub	Non
Sortie NumPy	Via get_array...	Natif	Natif
Cas d'usage	Pipeline média	Analyse musicale	I/O haute fidélité

5.7 Bonnes Pratiques

- **ffmpeg** : Installez ffmpeg sur le système avant d'utiliser Pydub avec des MP3. Vérifiez avec `from pydub.utils import which; print(which("ffmpeg"))`.
- **Mémoire** : Les AudioSegment résident entièrement en RAM. Pour des fichiers >1h, préférez le traitement par tranche avec `seg[i:i+60000]`.
- **ASR** : Toujours convertir en mono 16 kHz avant de transmettre à Whisper ou tout autre modèle de reconnaissance vocale.
- **Nommage** : Utilisez le zero-padding pour les noms de fichiers : `seg_0001.wav`, `seg_0002.wav`... pour garantir un tri correct.
- **Format** : WAV PCM 16-bit est le format le plus universellement compatible pour les pipelines ML.

Conseil

Pour l'analyse spectrale avancée (MFCC, Mel-spectrogramme, chroma) indispensable à la reconnaissance musicale ou émotionnelle, complétez Pydub avec **librosa** : `pip install librosa`. Les deux bibliothèques sont complémentaires et non concurrentes.

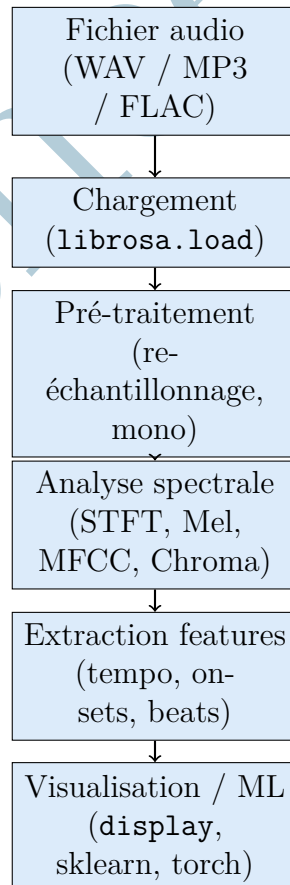
Chapitre 6 Librosa – Analyse Audio Avancée

6.1 Principe Fondamental

Librosa est une bibliothèque Python spécialisée dans l'**analyse audio et musicale**. Contrairement à Pydub qui se concentre sur la manipulation haut niveau (découpe, mixage, conversion), Librosa offre une boîte à outils complète pour l'**extraction de caractéristiques** (*features*) du signal : spectrogrammes, MFCC, chromagrammes, détection de tempo, segmentation harmonique/percussive.

Librosa retourne tous ses résultats sous forme de `ndarray` NumPy, ce qui en fait l'outil de prédilection pour les pipelines de **Machine Learning audio** : classification de genres musicaux, reconnaissance d'instruments, détection d'émotions vocales, ou préparation de données pour des modèles de deep learning (CNN sur spectrogrammes, RNN sur séquences MFCC).

6.1.1 Architecture de Librosa



6.1.2 Installation

Listing 6.1: Installation de Librosa et de ses dépendances

```
pip install librosa
# Dépendances utiles pour l'écosystème audio
pip install soundfile      # I/O audio bas niveau (
    utilise par librosa)
pip install matplotlib     # Visualisation des
    spectrogrammes
pip install scikit-learn   # Modeles ML sur features
    extraites

# Sur Ubuntu/Debian, libsndfile est requis :
sudo apt-get install libsndfile1
```

6.2 Chargement et Inspection

Listing 6.2: Chargement et inspection d'un fichier audio avec Librosa

```
import librosa
import numpy as np

def inspecter_audio_librosa(chemin: str, sr_cible: int =
    None) -> dict:
    """
    Charge un fichier audio avec Librosa et affiche un rapport
    d'inspection.

    Particularités de librosa.load :
    - Retourne (signal, sr) où signal est un ndarray float32
      normalisé [-1, 1]
    - Re-échantillonne automatiquement si sr est fourni
      (défaut: 22050 Hz)
    - Convertit en mono par défaut (mono=True)
    - Pour conserver l'original : sr=None, mono=False

    Args:
    chemin      : Chemin vers le fichier audio
    sr_cible    : Fréquence d'échantillonnage cible (None =
                  original)

    Returns:
    Dictionnaire contenant le signal et ses métadonnées
    """
    # Chargement (signal float32 normalisé + fréquence)
    y, sr = librosa.load(chemin, sr=sr_cible, mono=True)

    durée_s = librosa.get_duration(y=y, sr=sr)
    rms      = float(np.sqrt(np.mean(y**2)))
    pic      = float(np.max(np.abs(y)))
```

```

# Conversion en dB
rms_dB = 20 * np.log10(rms + 1e-10)
pic_dB = 20 * np.log10(pic + 1e-10)

print("=== Inspection audio (Librosa) ===")
print(f"Fichier           : {chemin}")
print(f"Frequence          : {sr} Hz")
print(f"Duree              : {duree_s:.3f}s
      ({duree_s/60:.1f} min)")
print(f"Echantillons       : {len(y)}")
print(f"Type                : {y.dtype} (normalise [-1,
      1])")
print(f"Pic d'amplitude     : {pic:.4f} ({pic_dB:.1f}
      dBFS)")
print(f"RMS                 : {rms:.4f} ({rms_dB:.1f}
      dBFS)")
print(f"Memoire              : {y.nbytes / 1024:.1f} Ko")

return {
    'signal' : y,
    'sr'      : sr,
    'duree_s' : duree_s,
    'rms'     : rms,
    'pic'     : pic,
    'rms_dB'  : rms_dB,
    'pic_dB'  : pic_dB
}

# Chargement avec re-echantillonnage a 22050 Hz (standard
  librosa)
info = inspecter_audio_librosa("musique.mp3",
    sr_cible=22050)
y, sr = info['signal'], info['sr']

```

6.3 Analyse Spectrale (STFT, Mel, MFCC)

Listing 6.3: Extraction de spectrogrammes et coefficients MFCC

```

import librosa
import numpy as np

def calculer_spectrogrammes(
y: np.ndarray,
sr: int,
n_fft: int = 2048,
hop_length: int = 512,
n_mels: int = 128,
n_mfcc: int = 13
) -> dict:

```

```

"""
Calcule les principales representations spectrales d'un
signal audio.

Representations calculees :
- STFT          : transformee de Fourier court terme
  (matrice complexe)
- Spectrogramme : magnitude au carre du STFT (en dB)
- Mel           : spectrogramme sur l'echelle perceptuelle
  Mel
- MFCC          : coefficients cepstraux (13 par default,
  standard ASR)

Args:
y          : Signal audio (ndarray)
sr         : Frequence d'echantillonnage (Hz)
n_fft      : Taille de la fenetre FFT (puissance de 2)
hop_length : Decalage entre fenetres successives
n_mels     : Nombre de bandes Mel
n_mfcc     : Nombre de coefficients MFCC

Returns:
Dictionnaire des representations spectrales
"""
# STFT : transformee complexe (frequence x temps)
stft = librosa.stft(y, n_fft=n_fft, hop_length=hop_length)

# Spectrogramme magnitude en dB
spec_db = librosa.amplitude_to_db(np.abs(stft), ref=np.max)

# Mel-spectrogramme (echelle perceptuelle)
mel = librosa.feature.melspectrogram(
    y=y, sr=sr, n_fft=n_fft,
    hop_length=hop_length, n_mels=n_mels
)
mel_db = librosa.power_to_db(mel, ref=np.max)

# MFCC : coefficients cepstraux (vecteur de features par
  trame)
mfcc = librosa.feature.mfcc(
    y=y, sr=sr, n_mfcc=n_mfcc,
    n_fft=n_fft, hop_length=hop_length
)

print("=== Representations spectrales ===")
print(f"STFT          : {stft.shape}      (freq x temps,
  complexe)")
print(f"Spec (dB)     : {spec_db.shape}    (freq x temps, en
  dB)")
print(f"Mel-spec      : {mel_db.shape}      ({n_mels} bandes
  Mel)")

```

```

print(f"MFCC          : {mfcc.shape}          ({n_mfcc}
      coefficients)")
print(f"Resolution   : {sr/n_fft:.1f} Hz par bin
      fréquentiel")
print(f"Duree trame: {hop_length/sr*1000:.1f} ms par
      trame")

return {
    'stft'      : stft,
    'spec_db'   : spec_db,
    'mel_db'    : mel_db,
    'mfcc'      : mfcc
}

specs = calculer_spectrogrammes(y, sr, n_mfcc=13)

```

6.4 Tempo, Beats et Caractéristiques Musicales

Listing 6.4: Détection de tempo, segmentation et features musicales

```

import librosa
import numpy as np

def analyser_musique(y: np.ndarray, sr: int) -> dict:
    """
    Extrait les caracteristiques musicales d'un signal audio.

    Features extraites :
    - Tempo (BPM)          : vitesse globale
    - Beats                : positions des temps forts
      (samples + secondes)
    - Onsets               : debuts d'evenements sonores
      (notes, percussions)
    - Chromagram           : energie par classe de hauteur
      (12 notes)
    - Spectral centroid    : centre de gravite spectral
      (brillance)
    - Spectral rolloff     : frequence sous laquelle est 85%
      de l'energie
    - Zero-crossing rate   : indicateur de bruit / consonnes
    - Harmonique / Percussif : separation HPSS

    Args:
    y : Signal audio
    sr : Frequence d'echantillonnage

    Returns:
    Dictionnaire des features musicales
    """
    # Tempo et beats

```

```

tempo, beats = librosa.beat.beat_track(y=y, sr=sr)
beat_times = librosa.frames_to_time(beats, sr=sr)

# Onsets (debut de notes)
onset_frames = librosa.onset.onset_detect(y=y, sr=sr)
onset_times = librosa.frames_to_time(onset_frames, sr=sr)

# Chromagram (12 classes de hauteur)
chroma = librosa.feature.chroma_stft(y=y, sr=sr)

# Caracteristiques spectrales scalaires (par trame)
centroid = librosa.feature.spectral_centroid(y=y, sr=sr)[0]
rolloff = librosa.feature.spectral_rolloff(y=y, sr=sr)[0]
zcr = librosa.feature.zero_crossing_rate(y)[0]

# Separation harmonique / percussive (HPSS)
y_harm, y_perc = librosa.effects.hpss(y)

print("=== Analyse musicale ===")
print(f"Tempo estime : {float(tempo):.1f} BPM")
print(f"Nombre de beats : {len(beats)}")
print(f"Nombre d'onsets : {len(onset_frames)}")
print(f"Chroma : {chroma.shape} (12 notes)")
print(f"Centroide spectral : {np.mean(centroid):.1f} Hz (moyenne)")
print(f"Rolloff spectral : {np.mean(rolloff):.1f} Hz (moyenne)")
print(f"ZCR moyen : {np.mean(zcr):.4f}")
print(f"Energie harmonique : {np.sum(y_harm**2):.2f}")
print(f"Energie percussive : {np.sum(y_perc**2):.2f}")

return {
    'tempo' : float(tempo),
    'beats' : beats,
    'beat_times' : beat_times,
    'onset_frames' : onset_frames,
    'onset_times' : onset_times,
    'chroma' : chroma,
    'centroid' : centroid,
    'rolloff' : rolloff,
    'zcr' : zcr,
    'y_harmonic' : y_harm,
    'y_percussive' : y_perc
}

features = analyser_musique(y, sr)

```

6.5 Visualisation et Export

Listing 6.5: Visualisation des spectrogrammes et export des résultats


```

import librosa
import librosa.display
import matplotlib.pyplot as plt
import numpy as np
import soundfile as sf

def visualiser_audio(
    y: np.ndarray,
    sr: int,
    specs: dict,
    chemin_sortie: str = "spectrogrammes.png"
) -> str:
    """
    Genere une visualisation multi-panneaux d'un signal audio :
    forme d'onde, spectrogramme, mel-spectrogramme et MFCC.

    Args:
        y          : Signal audio
        sr         : Frequence d'echantillonnage
        specs      : Dictionnaire retourne par
                     calculer_spectrogrammes()
        chemin_sortie : Chemin du fichier image de sortie

    Returns:
        Chemin du fichier image cree
    """
    fig, axes = plt.subplots(4, 1, figsize=(12, 10))

    # Forme d'onde
    librosa.display.waveshow(y, sr=sr, ax=axes[0],
                             color='steelblue')
    axes[0].set(title="Forme d'onde", xlabel="",
                ylabel="Amplitude")

    # Spectrogramme lineaire
    img1 = librosa.display.specshow(
        specs['spec_db'], sr=sr, x_axis='time', y_axis='log',
        ax=axes[1]
    )
    axes[1].set(title="Spectrogramme (echelle log)")
    fig.colorbar(img1, ax=axes[1], format="+2.0f dB")

    # Mel-spectrogramme
    img2 = librosa.display.specshow(
        specs['mel_db'], sr=sr, x_axis='time', y_axis='mel',
        ax=axes[2]
    )
    axes[2].set(title="Mel-spectrogramme")
    fig.colorbar(img2, ax=axes[2], format="+2.0f dB")

    # MFCC

```

```

img3 = librosa.display.specshow(
    specs['mfcc'], sr=sr, x_axis='time', ax=axes[3]
)
axes[3].set(title="MFCC (13 coefficients)",
            ylabel="Coefficient")
fig.colorbar(img3, ax=axes[3])

plt.tight_layout()
plt.savefig(chemin_sortie, dpi=150, bbox_inches='tight')
plt.close()

print(f"Visualisation sauvegardee : {chemin_sortie}")
return chemin_sortie

def exporter_audio_librosa(
    y: np.ndarray,
    sr: int,
    chemin_sortie: str,
    sr_cible: int = None
) -> str:
    """
    Exporte un signal audio vers un fichier WAV/FLAC via
    soundfile.

    Note : Librosa lui-meme n'ecrit pas d'audio. Il delegue
    cette tache a
    soundfile (formats sans compression) ou Pydub (formats
    compresses).

    Args:
    y          : Signal audio (ndarray float32)
    sr         : Frequence d'echantillonnage source
    chemin_sortie : Chemin du fichier (extension determine le
                    format)
    sr_cible   : Frequence cible (None = conserver)

    Returns:
    Chemin du fichier exporte
    """
    # Re-echantillonnage si necessaire
    if sr_cible is not None and sr_cible != sr:
        y = librosa.resample(y, orig_sr=sr, target_sr=sr_cible)
        sr = sr_cible
    print(f"Re-echantillonnage : {sr} Hz")

    # Export via soundfile (WAV, FLAC, OGG)
    sf.write(chemin_sortie, y, sr)

    import os
    taille = os.path.getsize(chemin_sortie) / 1024
    print(f"Exporte : {chemin_sortie} ({taille:.1f} Ko)")

```

```

return chemin_sortie

# Exemple d'utilisation
visualiser_audio(y, sr, specs, "analyse_complexe.png")
exporter_audio_librosa(y, sr, "audio_16k.wav",
    sr_cible=16000)

```

6.6 Fonctions Librosa les plus utilisées

6.6.1 Installation et Configuration

Définition

Librosa est une bibliothèque Python open-source dédiée à l'**analyse audio et musicale**. Développée par le groupe LabROSA de l'Université Columbia, elle est devenue le standard de facto pour l'extraction de caractéristiques audio en recherche scientifique et en Machine Learning. Elle s'appuie sur **NumPy**, **SciPy** et **soundfile** (pour l'I/O), et fournit plus de 200 fonctions spécialisées en traitement du signal audio.

Utilité

La configuration initiale est simple : un `pip install librosa` installe automatiquement les dépendances NumPy, SciPy, soundfile et numba. Sur Linux, le paquet système `libsndfile1` est requis pour soundfile. Librosa est particulièrement adaptée aux pipelines de **deep learning audio** (classification de genres, reconnaissance d'émotions, détection de keywords) car elle retourne des tableaux NumPy directement utilisables comme tenseurs PyTorch ou TensorFlow.

Fonction	Fonctionnalité	Instruction
<code>pip install</code>	Installer Librosa	<code>pip install librosa</code>
<code>pip install soundfile</code>	Backend I/O audio (recommandé)	<code>pip install soundfile</code>
<code>apt install libsndfile1</code>	Dépendance système (Linux)	<code>sudo apt install libsndfile1</code>
<code>import librosa</code>	Importer la bibliothèque	<code>import librosa</code>
<code>import librosa.display</code>	Module de visualisation	<code>import librosa.display</code>
<code>librosa.__version__</code>	Vérifier la version installée	<code>print(librosa.__version__)</code>
<code>librosa.ex()</code>	Charger un exemple intégré	<code>path = librosa.ex('trumpet')</code>
<code>librosa.util.example_audio_file()</code>	Fichier d'exemple	<code>librosa.util.example_audio_file()</code>

6.6.2 Chargement et Sauvegarde

Définition

Le **chargement** avec Librosa retourne toujours un **couple (signal, sr)** où **signal** est un `ndarray float32` normalisé dans $[-1, 1]$ et **sr** est la fréquence d'échantillonnage. Par défaut, Librosa **re-échantillonne à 22050 Hz** et convertit en mono. Pour conserver les paramètres d'origine, il faut spécifier `sr=None` et `mono=False`.

Exemple : `y, sr = librosa.load("audio.mp3", sr=16000)` charge et ré-échantillonne à 16 kHz mono.

Utilité

La normalisation automatique en `float32 [-1, 1]` simplifie énormément l'intégration dans les pipelines de Machine Learning : pas besoin de gérer manuellement les conversions entre `int16`, `int32` ou `float`. Pour la **sauvegarde**, Librosa délègue à `soundfile` (formats sans perte) ou à `Pydub` (formats compressés MP3). La fonction `librosa.resample` permet de changer la fréquence d'échantillonnage avec un algorithme de haute qualité (kaiser_best par défaut).

Fonction	Fonctionnalité	Instruction
<code>librosa.load()</code>	Charger un fichier audio (default 22050 Hz)	<code>y, sr = librosa.load("a.wav")</code>
<code>load(sr=None)</code>	Conserver la fréquence d'origine	<code>librosa.load("a.wav", sr=None)</code>
<code>load(sr=16000)</code>	Ré-échantillonner à 16 kHz	<code>librosa.load("a.wav", sr=16000)</code>
<code>load(mono=False)</code>	Conserver la stéréo	<code>librosa.load("a.wav", mono=False)</code>
<code>load(offset=, duration=)</code>	Charger une portion (secondes)	<code>librosa.load(p, offset=10, duration=5)</code>
<code>librosa.stream()</code>	Lecture par morceaux (gros fichiers)	<code>librosa.stream(p, block_length=256, ...)</code>
<code>librosa.get_duration</code>	Obtenir la durée en secondes	<code>librosa.get_duration(y=y, sr=sr)</code>
<code>librosa.get_samplerate</code>	Lire le SR sans charger le signal	<code>librosa.get_samplerate("a.wav")</code>
<code>soundfile.write()</code>	Écrire un fichier WAV/FLAC	<code>import soundfile as sf ; sf.write(...)</code>

Fonction	Fonctionnalité	Instruction
<code>librosa.resample()</code>	Ré-échantillonner un signal	<code>librosa.resample(y, orig_sr=sr, target_sr=16000)</code>

6.6.3 Représentation du Signal et Conversions

Définition

Librosa propose plusieurs **conversions fondamentales** entre les unités utilisées en traitement du signal audio :

- **Samples** ↔ **secondes** (via le SR).
- **Frames** ↔ **secondes** (via le `hop_length`).
- **Amplitude** ↔ **décibels** (échelle log).
- **Power** ↔ **décibels** (puissance vs amplitude).
- **Hz** ↔ **Mel** ↔ **notes MIDI**.

Utilité

Les conversions sont indispensables pour :

- Convertir des **positions temporelles** entre représentations (samples pour traitement bas niveau, secondes pour affichage).
- Passer de l'**amplitude linéaire** à l'échelle **décibel** pour une perception plus fidèle.
- Convertir des **fréquences en notes musicales** ($A4 = 440 \text{ Hz} = \text{MIDI } 69$).
- Aligner les **sorties d'analyse** (frames STFT) avec les **annotations temporelles** (secondes).

Fonction	Fonctionnalité	Instruction
<code>librosa.samples_to_time</code>	Échantillons → secondes	<code>librosa.samples_to_time(samples, sr=sr)</code>
<code>librosa.time_to_samples</code>	Secondes → échantillons	<code>librosa.time_to_samples(times, sr=sr)</code>
<code>librosa.frames_to_time</code>	Frames STFT → secondes	<code>librosa.frames_to_time(frames, sr=sr)</code>
<code>librosa.time_to_frames</code>	Secondes → frames STFT	<code>librosa.time_to_frames(times, sr=sr)</code>
<code>librosa.amplitude_to_db</code>	Amplitude linéaire → dB	<code>librosa.amplitude_to_db(np.abs(stft))</code>
<code>librosa.db_to_amplitude</code>	dB → amplitude	<code>librosa.db_to_amplitude(spec_db)</code>
<code>librosa.power_to_db</code>	Puissance → dB	<code>librosa.power_to_db(mel)</code>

Fonction	Fonctionnalité	Instruction
<code>librosa.db_to_power()</code>	dB → puissance	<code>librosa.db_to_power(mel_db)</code>
<code>librosa.hz_to_mel()</code>	Hz → échelle Mel	<code>librosa.hz_to_mel(440)</code>
<code>librosa.mel_to_hz()</code>	Mel → Hz	<code>librosa.mel_to_hz(50)</code>
<code>librosa.hz_to_midi()</code>	Hz → note MIDI	<code>librosa.hz_to_midi(440) # = 69 (La)</code>
<code>librosa.midi_to_note()</code>	MIDI → nom de note	<code>librosa.midi_to_note(69) # = "A4"</code>

6.6.4 Transformée de Fourier Court Terme (STFT)

Définition

La **STFT** (*Short-Time Fourier Transform*) calcule la transformée de Fourier sur des fenêtres successives du signal, produisant une représentation **temps-fréquence** sous forme de matrice complexe de dimensions $(n_fft/2 + 1, n_frames)$.

Paramètres clés :

- **n_fft** : taille de la fenêtre (typiquement 2048).
- **hop_length** : décalage entre fenêtres (typiquement 512).
- **window** : type de fenêtre (*hann* par défaut).

Résolution : fréquence = sr/n_fft Hz/bin ; temps = hop_length/sr s/trame.

Utilité

La STFT est la **base de toutes les analyses spectrales** en audio :

- Calculer le **spectrogramme** (visualisation temps-fréquence).
- Extraire des **features** (Mel, MFCC, chroma).
- Appliquer un **filtrage spectral** (égaliseur, réduction de bruit).
- Effectuer du **pitch shifting** ou du **time stretching** (préservation de la hauteur lors d'un changement de vitesse).
- Reconstruire le signal via **ISTFT** après modifications.

Un compromis existe entre **résolution temporelle** (petite n_fft) et **résolution fréquentielle** (grande n_fft).

Fonction	Fonctionnalité	Instruction
<code>librosa.stft()</code>	STFT (matrice complexe)	<code>stft = librosa.stft(y, n_fft=2048)</code>

Fonction	Fonctionnalité	Instruction
<code>librosa.istft()</code>	STFT inverse (reconstruction)	<code>y_rec = librosa.istft(stft)</code>
<code>np.abs(stft)</code>	Magnitude (spectrogramme)	<code>magnitude = np.abs(stft)</code>
<code>np.angle(stft)</code>	Phase	<code>phase = np.angle(stft)</code>
<code>librosa.magphase()</code>	Séparer magnitude et phase	<code>mag, phase = librosa.magphase(stft)</code>
<code>n_fft=</code>	Taille de fenêtre FFT	<code>librosa.stft(y, n_fft=4096)</code>
<code>hop_length=</code>	Décalage entre fenêtres	<code>librosa.stft(y, hop_length=256)</code>
<code>win_length=</code>	Longueur effective de fenêtre	<code>librosa.stft(y, win_length=1024)</code>
<code>window='hann'</code>	Type de fenêtre	<code>librosa.stft(y, window='hamming')</code>
<code>librosa.cqt()</code>	Constant-Q Transform (musique)	<code>cqt = librosa.cqt(y, sr=sr)</code>
<code>librosa.iirt()</code>	IIR filterbank	<code>librosa.iirt(y, sr=sr)</code>

6.6.5 Mel-Spectrogramme et MFCC

Définition

Le **Mel-spectrogramme** projette le spectrogramme STFT sur l'échelle **Mel**, une échelle perceptuelle non linéaire qui correspond à la sensibilité fréquentielle de l'oreille humaine. Les **MFCC** (*Mel-Frequency Cepstral Coefficients*) sont obtenus en appliquant une **Transformée en Cosinus Discrète** (DCT) au log du Mel-spectrogramme.

Standard ASR : 13 MFCC + 13 deltas + 13 delta-deltas = 39 features par trame.

Utilité

Les MFCC sont les **features audio les plus utilisées** en Machine Learning :

- **Reconnaissance vocale** (ASR) : standard depuis 30 ans.
- **Identification du locuteur** (*speaker recognition*).
- **Classification de genres musicaux** (rock, jazz, classique...).
- **Détection d'émotions** dans la voix.
- Entrée des modèles **HMM-GMM** et **réseaux de neurones** audio.

Le Mel-spectrogramme (sans DCT) est privilégié pour les **CNN** car il préserve la

structure spatiale temps-fréquence.

Fonction	Fonctionnalité	Instruction
<code>librosa.feature.mels</code>	Mel-spectrogramme	<code>mel = librosa.feature.melspectrogram(y=y, sr=sr)</code>
<code>n_mels=</code>	Nombre de bandes Mel	<code>melspectrogram(y=y, sr=sr, n_mels=128)</code>
<code>fmin=, fmax=</code>	Plage fréquentielle	<code>melspectrogram(..., fmin=20, fmax=8000)</code>
<code>librosa.feature.mfcc</code>	Coefficients MFCC	<code>mfcc = librosa.feature.mfcc(y=y, sr=sr)</code>
<code>n_mfcc=</code>	Nombre de coefficients MFCC	<code>mfcc(y=y, sr=sr, n_mfcc=13)</code>
<code>librosa.feature.delta</code>	Dérivée première (deltas)	<code>delta = librosa.feature.delta(mfcc)</code>
<code>librosa.feature.delta</code>	Dérivée seconde	<code>delta2 = librosa.feature.delta(mfcc, order=2)</code>
<code>librosa.feature.inverse</code>	Reconstruction audio	<code>y_rec = librosa.feature.inverse.mel_to_audio(mel)</code>
<code>librosa.feature.inverse</code>	MFCC → audio	<code>y_rec = librosa.feature.inverse.mfcc_to_audio(mfcc)</code>
<code>librosa.filters.mel</code>	Banc de filtres Mel	<code>filters = librosa.filters.mel(sr=sr, n_fft=2048)</code>

6.6.6 Caractéristiques Spectrales

Définition

Les **caractéristiques spectrales** sont des descripteurs scalaires calculés trame par trame qui résument certaines propriétés du spectre :

- **Spectral centroid** : centre de gravité spectral (brillance perçue).
- **Spectral rolloff** : fréquence sous laquelle se trouve 85% de l'énergie.
- **Spectral bandwidth** : étalement du spectre autour du centroïde.
- **Spectral contrast** : différence pic/vallée par bande.
- **Spectral flatness** : mesure du caractère bruité (1 = bruit blanc, 0 = ton pur).
- **Zero-crossing rate** : taux de passages par zéro (consonnes, percussions).

Utilité

Ces features sont utilisées pour :

- **Classifier des sons** (parole vs musique vs bruit).
- Distinguer des **instruments** (violon brillant vs basse sourde).
- Détecter des **voyelles vs consonnes** (ZCR élevé = consonnes).
- Caractériser une **texture sonore** (rugueuse vs lisse).
- Alimenter des **classifieurs simples** (SVM, Random Forest) sur des caractéristiques moyennées.

Fonction	Fonctionnalité	Instruction
<code>librosa.feature.spectral_centroid</code>	Centre de gravité spectral	<code>centroid = librosa.feature.spectral_centroid(y=y, sr=sr)</code>
<code>librosa.feature.spectral_rolloff</code>	Rolloff (85% énergie)	<code>rolloff = librosa.feature.spectral_rolloff(y=y, sr=sr)</code>
<code>librosa.feature.spectral_bandwidth</code>	Étalement spectral	<code>bw = librosa.feature.spectral_bandwidth(y=y, sr=sr)</code>
<code>librosa.feature.spectral_contrast</code>	Contraste pic/vallée	<code>contrast = librosa.feature.spectral_contrast(y=y, sr=sr)</code>
<code>librosa.feature.spectral_flatness</code>	Mesure de bruit	<code>flat = librosa.feature.spectral_flatness(y=y)</code>
<code>librosa.feature.zero_crossing_rate</code>	Taux passages zéro	<code>zcr = librosa.feature.zero_crossing_rate(y)</code>
<code>librosa.feature.rms</code>	Énergie RMS par trame	<code>rms = librosa.feature.rms(y=y)</code>
<code>librosa.feature.poly_features</code>	Coefficients polynomiaux	<code>poly = librosa.feature.poly_features(y=y, sr=sr)</code>
<code>librosa.feature.tonnetz</code>	Réseau tonal (musique)	<code>tonnetz = librosa.feature.tonnetz(y=y, sr=sr)</code>

6.6.7 Tempo, Beats et Rythme

Définition

L'analyse rythmique extrait la structure temporelle d'un morceau musical :

- **Tempo** (BPM, *beats per minute*) : vitesse globale.
- **Beats** : positions des temps forts (souvent toutes les noires).
- **Onsets** : débuts d'événements sonores (notes, percussions).
- **Tempogram** : représentation locale du tempo dans le temps.

Algorithme : Librosa utilise une combinaison d'**onset strength envelope** et de **programmation dynamique** pour estimer le tempo et aligner les beats.

Utilité

L'analyse rythmique est centrale en **Music Information Retrieval** (MIR) :

- **Synchroniser des éclairages** ou des effets visuels sur la musique.
- **Mixer des morceaux** (DJ) en alignant les beats (beat-matching).
- **Segmenter** un morceau en mesures, refrains, couplets.
- **Classifier** par genre (techno = 120-140 BPM, ballade = 60-90 BPM).
- Préparer des **annotations rythmiques** pour entraînement de modèles.
- Construire un **métronome adaptatif** pour analyser un musicien.

Fonction	Fonctionnalité	Instruction
<code>librosa.beat.beat_track(y=y, sr=sr)</code>	Détection tempo + beats	<code>tempo, beats = librosa.beat.beat_track(y=y, sr=sr)</code>
<code>librosa.beat.tempo(y=y, sr=sr)</code>	Tempo seul (BPM estimé)	<code>tempo = librosa.beat.tempo(y=y, sr=sr)</code>
<code>librosa.onset.onset_detect(y=y, sr=sr)</code>	Détection d'onsets	<code>onsets = librosa.onset.onset_detect(y=y, sr=sr)</code>
<code>librosa.onset.onset_strength(y=y, sr=sr)</code>	Enveloppe d'attaque	<code>env = librosa.onset.onset_strength(y=y, sr=sr)</code>
<code>librosa.onset.onset_backtrack(onsets, env)</code>	Recaler sur le minimum	<code>librosa.onset.onset_backtrack(onsets, env)</code>
<code>librosa.feature.tempogram(y=y, sr=sr)</code>	Tempogram local	<code>tg = librosa.feature.tempogram(y=y, sr=sr)</code>
<code>librosa.beat.plp(y=y, sr=sr)</code>	Predominant Local Pulse	<code>pulse = librosa.beat.plp(y=y, sr=sr)</code>
<code>librosa.frames_to_time(beats, sr=sr)</code>	Convertir beats en secondes	<code>librosa.frames_to_time(beats, sr=sr)</code>
<code>librosa.util.sync(mfcc, beats, aggregate=np.mean)</code>	Synchroniser features sur les beats	<code>librosa.util.sync(mfcc, beats, aggregate=np.mean)</code>

6.6.8 Analyse Harmonique et Chroma

Définition

Le **chromagramme** projette le spectre sur les **12 classes de hauteur** de la gamme chromatique occidentale (C, C#, D, D#, E, F, F#, G, G#, A, A#, B). Il est invariant à l'octave et représente la **distribution harmonique** d'un signal musical.

La **séparation harmonique/percussive** (HPSS, *Harmonic-Percussive Source Separation*) décompose un signal en deux composantes : la partie **tonale** (notes tenues, accords) et la partie **transitoire** (percussions, attaques).

Utilité

Ces outils sont fondamentaux pour l'**analyse musicale** :

- **Reconnaissance d'accords** et de tonalité.
- **Cover song detection** : identifier des reprises malgré transposition ou changement d'instrumentation.
- **Extraction de mélodies** en isolant la composante harmonique.
- **Karaoké automatique** en supprimant la partie vocale.
- **Remix et mashups** en isolant les percussions.
- **Score-following** : alignement partition-audio.

Fonction	Fonctionnalité	Instruction
<code>librosa.feature.chroma_stft</code>	Chromagramme depuis STFT	<code>chroma = librosa.feature.chroma_stft(y=y, sr=sr)</code>
<code>librosa.feature.chroma_cqt</code>	Chromagramme depuis CQT (mieux)	<code>chroma = librosa.feature.chroma_cqt(y=y, sr=sr)</code>
<code>librosa.feature.chroma_cens</code>	CENS (chroma normalisé)	<code>cens = librosa.feature.chroma_cens(y=y, sr=sr)</code>
<code>librosa.effects.hpss</code>	Séparation harmonique/percussive	<code>y_harm, y_perc = librosa.effects.hpss(y)</code>
<code>librosa.effects.harmonic</code>	Composante harmonique seule	<code>y_harm = librosa.effects.harmonic(y)</code>
<code>librosa.effects.percussive</code>	Composante percussive seule	<code>y_perc = librosa.effects.percussive(y)</code>
<code>librosa.decompose.nmf</code>	Décomposition NMF	<code>W, H = librosa.decompose.decompose(S, n_components=8)</code>
<code>librosa.decompose.nn_filter</code>	Filtre par voisins proches	<code>librosa.decompose.nn_filter(S)</code>
<code>librosa.key_to_notes</code>	Notes d'une tonalité	<code>librosa.key_to_notes("C:maj")</code>

6.6.9 Effets Audio et Transformations

Définition

Le module `librosa.effects` regroupe des transformations **avancées** du signal audio préservant sa qualité perceptuelle :

- **Time stretching** : modifier la durée sans changer la hauteur.
- **Pitch shifting** : transposer en fréquence sans changer la durée.
- **Trim** : supprimer les silences en début/fin.
- **Split** : segmenter sur les zones non-silencieuses.
- **Preemphasis** : accentuer les hautes fréquences (préparation ASR).
- **Remix** : réordonner des segments selon des intervalles.

Utilité

Ces effets sont utilisés pour :

- **Data augmentation** : générer des variantes d'un même fichier (changement de tempo $\pm 10\%$, transposition ± 2 demi-tons).
- Adapter un **podcast** à une vitesse de lecture confortable.
- Effectuer un **karaoké** en transposant pour la tessiture du chanteur.
- **Préparer des features pour ASR** (preemphasis avant MFCC).
- Créer des **remixes** en réorganisant les segments musicaux.

La qualité du **phase vocoder** de Librosa permet des transformations sans artefacts audibles dans les plages raisonnables ($\pm 30\%$).

Fonction	Fonctionnalité	Instruction
<code>librosa.effects.time</code>	Modifier durée (sans pitch)	<code>y_slow = librosa.effects.time_stretch(y, rate=0.8)</code>
<code>librosa.effects.pit</code>	Transposer (sans durée)	<code>y_up = librosa.effects.pitch_shift(y, sr=sr, n_steps=4)</code>
<code>librosa.effects.trim</code>	Supprimer silences début/fin	<code>y_trim, idx = librosa.effects.trim(y, top_db=20)</code>
<code>librosa.effects.spl</code>	Segmenter sur non-silences	<code>intervals = librosa.effects.split(y, top_db=30)</code>
<code>librosa.effects.pre</code>	Filtre préaccentuation	<code>y_pre = librosa.effects.preemphasis(y)</code>
<code>librosa.effects.dee</code>	Filtre désaccentuation	<code>y_deem = librosa.effects.deemphasis(y_pre)</code>

Fonction	Fonctionnalité	Instruction
<code>librosa.effects.remix</code>	Réordonner par intervalles	<code>y_rmx = librosa.effects.remix(y, intervals)</code>
<code>librosa.effects.harmonic</code>	Composante harmonique	<code>y_harm = librosa.effects.harmonic(y, margin=8)</code>
<code>librosa.effects.percussive</code>	Composante percussive	<code>y_perc = librosa.effects.percussive(y, margin=8)</code>

6.6.10 Visualisation des Spectrogrammes

Définition

Le sous-module `librosa.display` fournit des fonctions de visualisation spécialisées pour l'audio, basées sur Matplotlib. Il gère automatiquement les axes temporels et fréquentiels (linéaire, log, Mel, CQT) et facilite la production de figures publication-ready pour la recherche audio.

Utilité

La visualisation est cruciale pour :

- **Inspecter visuellement** le contenu d'un signal audio.
- Identifier des **anomalies** (saturation, bruit, coupures).
- Présenter des **résultats d'analyse** dans des publications.
- **Déboguer des pipelines ML** en visualisant les features extraites.
- Créer des **supports pédagogiques** (visualisation des MFCC, des chromagrammes, des beats).
- Comparer visuellement deux signaux (avant/après filtrage).

Fonction	Fonctionnalité	Instruction
<code>librosa.display.waveplot</code>	Tracer la forme d'onde	<code>librosa.display.waveshow(y, sr=sr)</code>
<code>librosa.display.spectrogram</code>	Afficher un spectrogramme	<code>librosa.display.specshow(spec_db, sr=sr)</code>
<code>x_axis='time'</code>	Axe temporel en secondes	<code>specshow(spec, x_axis='time')</code>
<code>y_axis='log'</code>	Axe fréquentiel logarithmique	<code>specshow(spec, y_axis='log')</code>
<code>y_axis='mel'</code>	Axe Mel (perceptuel)	<code>specshow(mel_db, y_axis='mel')</code>
<code>y_axis='cqt_note'</code>	Axe en notes musicales	<code>specshow(cqt, y_axis='cqt_note')</code>

Fonction	Fonctionnalité	Instruction
<code>y_axis='chroma'</code>	Axe chromatique (12 notes)	<code>specshow(chroma, y_axis='chroma')</code>
<code>cmap='magma'</code>	Palette de couleurs	<code>specshow(spec_db, cmap='magma')</code>
<code>plt.colorbar()</code>	Ajouter une barre de couleur	<code>plt.colorbar(format="%+2.0f dB")</code>
<code>librosa.display.Time</code>	Formatage temporel personnalisé	<code>ax.xaxis.set_major_formatter(TimeFormatter())</code>

Principaux paramètres et conventions Librosa :

Paramètre / Convention	Catégorie	Description / Valeur typique
22050 Hz	Fréquence par défaut	SR par défaut de <code>librosa.load</code>
16000 Hz	Fréquence ASR	Standard pour Whisper, DeepSpeech
<code>n_fft = 2048</code>	Taille fenêtre FFT	Standard musical (93 ms à 22 kHz)
<code>n_fft = 512</code>	Taille fenêtre FFT	Standard parole (32 ms à 16 kHz)
<code>hop_length = 512</code>	Décalage trame	Standard musical (23 ms à 22 kHz)
<code>hop_length = 160</code>	Décalage trame	Standard parole (10 ms à 16 kHz)
<code>n_mels = 128</code>	Bandes Mel	Standard pour CNN spectrogrammes
<code>n_mels = 40</code>	Bandes Mel	Standard ASR (compromis taille/info)
<code>n_mfcc = 13</code>	Coefficients MFCC	Standard reconnaissance vocale
<code>n_mfcc = 20</code>	Coefficients MFCC	Reconnaissance musicale (plus de détails)
<code>window = 'hann'</code>	Fenêtre FFT	Par défaut (bon compromis)
<code>window = 'hamming'</code>	Fenêtre FFT	Alternative (lobes secondaires plus bas)
<code>float32 [-1, 1]</code>	Format signal	Sortie standard de <code>librosa.load</code>
<code>(freq, time)</code>	Convention matrices	Spectrogrammes : lignes=freq, colonnes=temps
440 Hz = A4 = MIDI 69	Référence musicale	La 440, standard d'accord
12 classes	Échelle chromatique	C, C#, D, ..., A, A#, B
<code>top_db = 60</code>	Seuil silence trim	60 dB sous le pic = silence

6.7 Comparaison Pydub vs Librosa

Critère	Pydub	Librosa
Niveau d'abstraction	Haut niveau (objets)	Bas niveau (ndarray)
Format de signal	<code>AudioSegment</code>	<code>ndarray float32</code>
Découpe / concaténation	Native (slicing ms)	Via NumPy (slicing samples)
Conversion de format	Native (export multi)	Via soundfile/Pydub
STFT / Spectrogramme	Non	Native
MFCC / Mel	Non	Native
Détection tempo / beats	Non	Native
Effets (fade, gain)	Native	Limités (trim, preemphasis)
Time stretching / pitch shift	Non	Native (qualité phase vocoder)
Visualisation	Non	Native (<code>display</code>)
Cas d'usage principal	Pipeline média / production	Analyse ML / MIR

6.8 Bonnes Pratiques

- **Combinaison Pydub + Librosa** : utilisez Pydub pour la **préparation** (chargement multi-format, découpe, normalisation) et Librosa pour l'**analyse** (STFT, MFCC, tempo). Les deux sont complémentaires.
- **Fréquence d'échantillonnage** : `librosa.load` ré-échantillonne par défaut à 22050 Hz. Utilisez `sr=None` pour conserver l'original ou `sr=16000` pour les pipelines ASR (Whisper, DeepSpeech).
- **`n_fft` et `hop_length`** : pour la **parole**, utilisez `n_fft=512`, `hop_length=160` (32 ms / 10 ms à 16 kHz). Pour la **musique**, préférez `n_fft=2048`, `hop_length=512`.
- **Mémoire** : Librosa charge tout le signal en RAM. Pour les fichiers très longs, utilisez `librosa.stream()` qui découpe en blocs.
- **Performance** : Librosa utilise `numba` pour accélérer les calculs critiques. Le premier appel à une fonction est lent (compilation JIT), les suivants sont rapides.
- **Format de spectrogramme** : convention Librosa = (freq, time). Si vous nourrissez un CNN PyTorch attendu en (batch, channels, height, width), pensez à transposer ou à ajouter une dimension : `spec[None, None, :, :]`.
- **Sauvegarde** : Librosa lui-même n'écrit pas. Utilisez `soundfile.write()` pour WAV/FLAC ou Pydub pour MP3.

Conseil

Pour un pipeline audio complet (**chargement multi-format** → **analyse** → **classification**), la combinaison gagnante est : **Pydub** pour I/O et prétraitement → **Librosa** pour extraction de features → **scikit-learn** ou **PyTorch** pour le modèle. Cette pile couvre 90 % des besoins en Machine Learning audio.

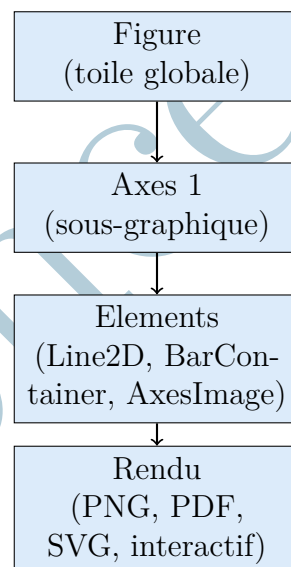
Chapitre 7 Matplotlib – Visualisation Multimodale

7.1 Principe Fondamental

Matplotlib est la bibliothèque de visualisation scientifique de référence en Python. Elle permet de créer des graphiques 2D de qualité publication : courbes, histogrammes, nuages de points, heatmaps, spectrogrammes et images. Sa fonction principale dans un pipeline multimodal est de rendre visible et interprétable ce qui est numérique.

Matplotlib suit un paradigme à deux niveaux : la **Figure** (la toile globale) contient un ou plusieurs **Axes** (sous-graphiques), chacun portant ses éléments visuels (lignes, barres, images, textes).

7.1.1 Architecture de Matplotlib



7.1.2 Installation

Listing 7.1: Installation de Matplotlib

```
pip install matplotlib seaborn
# seaborn étend matplotlib pour les graphiques statistiques
# avancées
```

7.2 Graphiques de Base

7.2.1 Configuration Globale

Listing 7.2: Configuration professionnelle de Matplotlib

```
import matplotlib.pyplot as plt
import matplotlib as mpl
import numpy as np

def configurer_style_rapport():
    """
    Configure Matplotlib pour produire des graphiques de
    qualite rapport.
    A appeler une seule fois en debut de notebook ou de script.
    """
    # Style de base
    plt.style.use('seaborn-v0_8-whitegrid')

    # Parametres globaux
    mpl.rcParams.update({
        # Police
        'font.size' : 11,
        'axes.titlesize' : 13,
        'axes.labelsize' : 11,
        'legend.fontsize' : 10,
        'xtick.labelsize' : 9,
        'ytick.labelsize' : 9,
        # Resolution
        'figure.dpi' : 100,
        'savefig.dpi' : 150,
        # Encodage
        'savefig.bbox' : 'tight',
        'figure.facecolor' : 'white',
        'axes.facecolor' : 'white',
        # Couleurs
        'axes.prop_cycle' : mpl.cycler(color=[
            '#2196F3', '#FF5722', '#4CAF50',
            '#9C27B0', '#FF9800', '#00BCD4'
        ])
    })
    print("Style Matplotlib configure.")

configurer_style_rapport()
```

7.2.2 Visualisation de Donnees Tabulaires

Listing 7.3: Graphiques pour donnees tabulaires

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

def visualiser_catalogue(df: pd.DataFrame, chemin_sortie: str
    = "catalogue.png"):
    """
```

Cree un dashboard de 4 graphiques pour analyser un catalogue media.

Graphiques produits :

1. Distribution des durees (histogramme)
2. Repartition par categorie (camembert)
3. Duree vs taille (nuage de points)
4. Statistiques par categorie (barres)

Args:

- df : DataFrame du catalogue media
- chemin_sortie: Chemin de sauvegarde de la figure

Returns:

Figure Matplotlib

```
"""
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
fig.suptitle("Tableau de bord du catalogue media",
             fontsize=15, fontweight='bold')

# --- 1. Histogramme des durees ---
ax = axes[0, 0]
ax.hist(df['duree_s'], bins=30, color='#2196F3',
        edgecolor='white', alpha=0.8)
ax.set_title("Distribution des durees")
ax.set_xlabel("Duree (secondes)")
ax.set_ylabel("Frequence")
ax.axvline(df['duree_s'].mean(), color='red',
           linestyle='--', label='Moyenne')
ax.legend()

# --- 2. Camembert par categorie ---
ax = axes[0, 1]
cat_counts = df['categorie'].value_counts()
ax.pie(cat_counts.values, labels=cat_counts.index,
        autopct='%1.1f%%', startangle=140,
        colors=plt.cm.Set3.colors[:len(cat_counts)])
ax.set_title("Repartition par categorie")

# --- 3. Nuage de points duree vs taille ---
ax = axes[1, 0]
scatter = ax.scatter(
    df['duree_s'], df['taille_ko'],
    c=df['categorie'].astype('category').cat.codes,
    cmap='tab10', alpha=0.7, s=50
)
ax.set_title("Duree vs Taille")
ax.set_xlabel("Duree (s)")
ax.set_ylabel("Taille (Ko)")
plt.colorbar(scatter, ax=ax, label='Categorie')
```

```

# --- 4. Barres des durees moyennes par categorie ---
ax = axes[1, 1]
stats =
    df.groupby('categorie')['duree_s'].mean().sort_values(ascending=True)
bars = ax.barh(stats.index, stats.values,
               color=['#2196F3', '#FF5722', '#4CAF50',
                     '#9C27B0'])
ax.set_title("Duree moyenne par categorie")
ax.set_xlabel("Duree moyenne (s)")
for bar, val in zip(bars, stats.values):
    ax.text(val + 0.5, bar.get_y() + bar.get_height()/2,
           f'{val:.0f}s', va='center', fontsize=9)

plt.tight_layout()
plt.savefig(chemin_sortie, dpi=150, bbox_inches='tight')
print(f"Dashboard sauvegarde : {chemin_sortie}")
plt.close()

return fig

# Exemple
fig = visualiser_catalogue(df_propre,
    "dashboard_catalogue.png")

```

7.3 Visualisation des Donnees Image

Listing 7.4: Visualisation d'images et de leurs caracteristiques

```

import matplotlib.pyplot as plt
import cv2
import numpy as np

def visualiser_traitement_image(
    img_bgr: np.ndarray,
    chemin_sortie: str = "analyse_image.png"
) -> None:
    """
    Cree une figure de 6 sous-graphiques pour analyser le
    traitement d'une image.

    Sous-graphiques :
    1. Image originale (RGB)
    2. Niveaux de gris
    3. Histogramme des niveaux de gris
    4. Image apres lissage Gaussien
    5. Contours Canny
    6. Histogramme par canal BGR

    Args:
        img_bgr      : Image source au format BGR (OpenCV)
    """

```

```

    chemin_sortie: Chemin de sauvegarde
"""
# Conversions
img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)
blurred = cv2.GaussianBlur(gray, (5, 5), 0)
canny = cv2.Canny(blurred, 50, 150)

fig, axes = plt.subplots(2, 3, figsize=(15, 9))
fig.suptitle("Pipeline de traitement d'image",
             fontsize=14, fontweight='bold')

# 1. Image originale
axes[0, 0].imshow(img_rgb)
axes[0, 0].set_title("Image originale")
axes[0, 0].axis('off')

# 2. Niveaux de gris
axes[0, 1].imshow(gray, cmap='gray')
axes[0, 1].set_title("Niveaux de gris")
axes[0, 1].axis('off')

# 3. Histogramme des niveaux de gris
axes[0, 2].hist(gray.ravel(), bins=64, color='#607D8B',
                edgecolor='white', alpha=0.85)
axes[0, 2].set_title("Histogramme (gris)")
axes[0, 2].set_xlabel("Intensite [0-255]")
axes[0, 2].set_ylabel("Frequence")

# 4. Image lisee
axes[1, 0].imshow(blurred, cmap='gray')
axes[1, 0].set_title("Apres lissage Gaussien")
axes[1, 0].axis('off')

# 5. Contours Canny
axes[1, 1].imshow(canny, cmap='hot')
axes[1, 1].set_title("Contours Canny")
axes[1, 1].axis('off')

# 6. Histogramme par canal BGR
couleurs = [('1565C0', 'B'), ('2E7D32', 'G'),
            ('C62828', 'R')]
for i, (couleur, nom) in enumerate(couleurs):
    canal = img_bgr[:, :, i]
    axes[1, 2].hist(canal.ravel(), bins=64, color=couleur,
                   alpha=0.5, label=nom)
axes[1, 2].set_title("Histogramme par canal BGR")
axes[1, 2].set_xlabel("Intensite [0-255]")
axes[1, 2].legend()

plt.tight_layout()

```

```
plt.savefig(chemin_sortie, dpi=150, bbox_inches='tight')
print(f"Analyse image sauvegardee : {chemin_sortie}")
plt.close()

# Exemple
img_bgr = cv2.imread("photo.jpg")
visualiser_traitement_image(img_bgr, "analyse_image.png")
```

7.4 Visualisation des Donnees Audio

Listing 7.5: Forme d'onde et spectrogramme avec Matplotlib

```
import matplotlib.pyplot as plt
import numpy as np
from pydub import AudioSegment

def visualiser_audio(
    chemin: str,
    chemin_sortie: str = "analyse_audio.png"
) -> None:
    """
    Cree une visualisation complete d'un fichier audio.

    Graphiques :
    1. Forme d'onde (amplitude vs temps)
    2. Spectrogramme (frequence vs temps, intensite en
       couleur)
    3. Histogramme des amplitudes
    4. Enveloppe RMS (energie locale)

    Args:
        chemin : Chemin vers le fichier audio
        chemin_sortie: Chemin de sauvegarde de la figure
    """
    # Chargement avec Pydub et conversion en mono 16 kHz
    seg = AudioSegment.from_file(chemin).set_channels(1).set_frame_rate(16000)
    fs = seg.frame_rate
    samples = np.array(seg.get_array_of_samples(), dtype=float)
    samples /= (2 ** (seg.sample_width * 8 - 1)) # Normalisation [-1, 1]
    t = np.linspace(0, len(samples) / fs, len(samples))

    # Enveloppe RMS (fenetre de 20ms)
    taille_fen = int(fs * 0.02)
    nb_fen = len(samples) // taille_fen
    rms_env = np.array([
        np.sqrt(np.mean(samples[i*taille_fen:(i+1)*taille_fen]**2))
        for i in range(nb_fen)
    ])
    ])
```

```

t_rms = np.linspace(0, t[-1], nb_fen)

fig, axes = plt.subplots(2, 2, figsize=(14, 9))
fig.suptitle(f"Analyse audio : {chemin}", fontsize=13,
             fontweight='bold')

# 1. Forme d'onde
axes[0, 0].plot(t, samples, color='#1565C0', lw=0.5,
               alpha=0.8)
axes[0, 0].set_title("Forme d'onde")
axes[0, 0].set_xlabel("Temps (s)")
axes[0, 0].set_ylabel("Amplitude")
axes[0, 0].set_xlim(0, t[-1])
axes[0, 0].axhline(0, color='gray', lw=0.5)

# 2. Spectrogramme
axes[0, 1].specgram(samples, Fs=fs, NFFT=512, noverlap=256,
                    cmap='inferno', scale='dB')
axes[0, 1].set_title("Spectrogramme")
axes[0, 1].set_xlabel("Temps (s)")
axes[0, 1].set_ylabel("Frequence (Hz)")
axes[0, 1].set_ylim(0, fs // 2)

# 3. Histogramme des amplitudes
axes[1, 0].hist(samples, bins=80, color='#FF5722',
                edgecolor='white', alpha=0.8)
axes[1, 0].set_title("Distribution des amplitudes")
axes[1, 0].set_xlabel("Amplitude")
axes[1, 0].set_ylabel("Frequence")
axes[1, 0].axvline(0, color='black', lw=1, linestyle='--')

# 4. Enveloppe RMS
axes[1, 1].fill_between(t_rms, rms_env, color='#4CAF50',
                       alpha=0.7)
axes[1, 1].set_title("Enveloppe RMS (energie locale)")
axes[1, 1].set_xlabel("Temps (s)")
axes[1, 1].set_ylabel("RMS")
axes[1, 1].set_xlim(0, t[-1])

plt.tight_layout()
plt.savefig(chemin_sortie, dpi=150, bbox_inches='tight')
print(f"Analyse audio sauvegardee : {chemin_sortie}")
plt.close()

# Exemple
visualiser_audio("cours_enregistre.mp3", "analyse_cours.png")

```

7.5 Dashboard Multimodal Integratif

Listing 7.6: Dashboard complet combinant toutes les modalités

```

import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import cv2, numpy as np
from pydub import AudioSegment

def dashboard_multimodal(
    chemin_image: str,
    chemin_audio: str,
    df_meta: 'pd.DataFrame',
    chemin_sortie: str = "dashboard_final.png"
) -> None:
    """
    Cree un dashboard de synthese combinant image, audio et
    donnees tabulaires.

    Mise en page :
        Ligne 1 : image originale | histogramme pixels |
                  barres categories
        Ligne 2 : forme d'onde audio | spectrogramme audio

    Args:
        chemin_image : Chemin vers l'image de reference
        chemin_audio : Chemin vers l'enregistrement audio
        df_meta       : DataFrame de metadonnees du projet
        chemin_sortie: Chemin de sauvegarde
    """
    fig = plt.figure(figsize=(16, 10))
    gs = gridspec.GridSpec(2, 3, figure=fig, hspace=0.4,
                           wspace=0.35)

    fig.suptitle("Dashboard Multimodal -- Synthese du Projet",
                 fontsize=15, fontweight='bold')

    # --- Chargements ---
    img_bgr = cv2.imread(chemin_image)
    img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
    gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)

    seg =
        AudioSegment.from_file(chemin_audio).set_channels(1)
    fs = seg.frame_rate
    samples = np.array(seg.get_array_of_samples(), dtype=float)
    samples /= max(np.max(np.abs(samples)), 1.0)
    t = np.linspace(0, len(samples)/fs, len(samples))

    # Axes
    ax_img = fig.add_subplot(gs[0, 0])
    ax_hist = fig.add_subplot(gs[0, 1])
    ax_cat = fig.add_subplot(gs[0, 2])
    ax_wave = fig.add_subplot(gs[1, 0:2])
    ax_spec = fig.add_subplot(gs[1, 2])

```

```

# 1. Image originale
ax_img.imshow(img_rgb)
ax_img.set_title("Image de reference")
ax_img.axis('off')

# 2. Histogramme pixels
ax_hist.hist(gray.ravel(), bins=64, color='#607D8B',
             alpha=0.85)
ax_hist.set_title("Histogramme pixels")
ax_hist.set_xlabel("Intensite [0-255]")
ax_hist.set_ylabel("Frequence")

# 3. Barres des categories
cats = df_meta['categorie'].value_counts()
ax_cat.bar(cats.index, cats.values,
           color=plt.cm.Set2.colors[:len(cats)],
           edgecolor='white')
ax_cat.set_title("Repartition des categories")
ax_cat.set_xlabel("Categorie")
ax_cat.set_ylabel("Nombre de fichiers")
ax_cat.tick_params(axis='x', rotation=30)

# 4. Forme d'onde
ax_wave.plot(t, samples, color='#1565C0', lw=0.4,
            alpha=0.8)
ax_wave.fill_between(t, samples, color='#1565C0',
                   alpha=0.15)
ax_wave.set_title("Forme d'onde audio")
ax_wave.set_xlabel("Temps (s)")
ax_wave.set_ylabel("Amplitude")
ax_wave.set_xlim(0, t[-1])

# 5. Spectrogramme
ax_spec.specgram(samples, Fs=fs, NFFT=256, noverlap=128,
                cmap='magma')
ax_spec.set_title("Spectrogramme")
ax_spec.set_xlabel("Temps (s)")
ax_spec.set_ylabel("Freq. (Hz)")

plt.savefig(chemin_sortie, dpi=150, bbox_inches='tight')
print(f"Dashboard sauvegarde : {chemin_sortie}")
plt.close()

# Exemple
dashboard_multimodal(
    chemin_image="scene.jpg",
    chemin_audio="narration.mp3",
    df_meta=df_propre,
    chemin_sortie="dashboard_final.png"
)

```


7.6 Bonnes Pratiques

- **Fermer les figures** : Appelez toujours `plt.close()` apres `savefig` pour liberer la memoire. En boucle sur 1000 images, l'oubli de `close` provoque un epuisement de la RAM.
- **DPI** : Utilisez `dpi=100` pour l'affichage interactif et `dpi=150` (ou plus) pour les exports de rapport.
- **bbox_inches='tight'** : Ajouter ce parametre a `savefig` evite que les etiquettes soient coupees.
- **Styles** : `plt.style.use('seaborn-v0_8-whitegrid')` produit des graphiques propres et lisibles pour les rapports academiques.
- **Couleurs accessibles** : Evitez rouge/vert sans motif supplementaire (daltonisme). Privilegiez la palette `Set2` ou `tab10`.

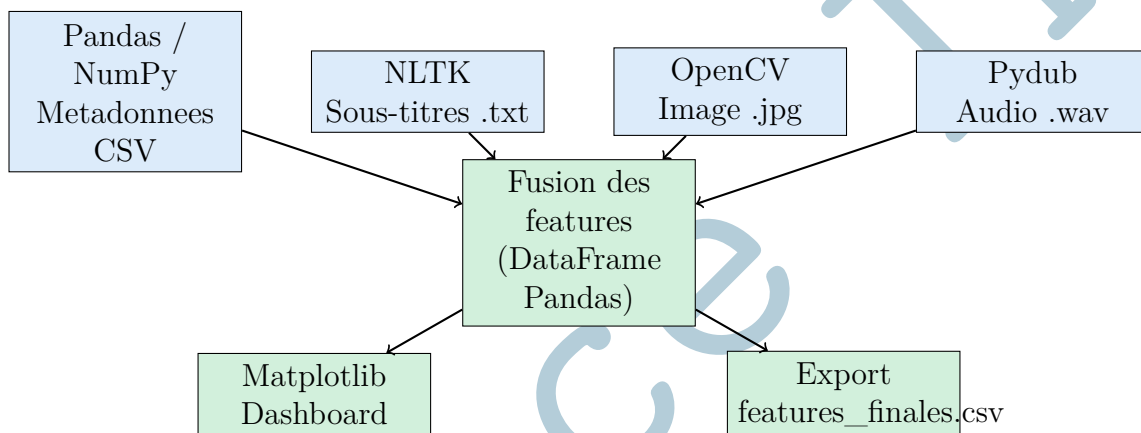
Remarque

La fonction `plt.specgram` de Matplotlib calcule et affiche un spectrogramme a court terme (STFT) directement depuis un tableau NumPy. Pour les analyses spectrales avancees (MFCC, Mel-spectrogramme, chroma), utilisez **librosa** dont les fonctions retournent des matrices NumPy que Matplotlib peut afficher avec `imshow`.

Chapitre 8 Pipeline Integratif Multimodal

8.1 Architecture Generale

Un pipeline de traitement multimodal complet assemble les cinq classes d'outils en une sequence coherente dont le but est de produire un **vecteur de features** unifie par entite multimodale, pret a etre fourni a un modele d'apprentissage.



8.2 Implementation Complete du Pipeline

Listing 8.1: Pipeline multimodal complet – extraction de features

```
import pandas as pd
import numpy as np
import cv2
from pydub import AudioSegment
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import SnowballStemmer
from collections import Counter
import os

def extraire_features_image(chemin_img: str) -> dict:
    """
    Extrait les features visuelles d'une image : couleur,
    texture, taille.
    Returns: dict de scalaires numeriques
    """
    img = cv2.imread(chemin_img)
    if img is None:
        return {'img_largeur': 0, 'img_hauteur': 0,
```

```

        'img_luminosite': 0.0, 'img_contraste': 0.0}

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

return {
    'img_largeur'      : img.shape[1],
    'img_hauteur'      : img.shape[0],
    'img_luminosite'   : float(gray.mean()),
    'img_contraste'    : float(gray.std()),
    'img_teinte_dom'   :
        float(np.argmax(cv2.calcHist([hsv],[0],None,[180],[0,180]))),
    'img_saturation'   : float(hsv[:, :, 1].mean())
}

def extraire_features_audio(chemin_audio: str) -> dict:
    """
    Extrait les features acoustiques : duree, RMS, niveau,
    taux.
    Returns: dict de scalaires numeriques
    """
    try:
        seg =
            AudioSegment.from_file(chemin_audio).set_channels(1)
        samples = np.array(seg.get_array_of_samples(),
            dtype=float)
        rms = np.sqrt(np.mean(samples**2))
        return {
            'aud_duree_s'      : len(seg) / 1000.0,
            'aud_rms'         : float(rms),
            'aud_dbfs'        : float(seg.dBFS),
            'aud_max_dbfs'    : float(seg.max_dBFS),
            'aud_frame_rate'  : seg.frame_rate
        }
    except Exception as e:
        print(f" Erreur audio : {e}")
        return {'aud_duree_s': 0.0, 'aud_rms': 0.0,
            'aud_dbfs': 0.0, 'aud_max_dbfs': 0.0,
            'aud_frame_rate': 0}

def extraire_features_texte(chemin_txt: str, langue: str =
    'french') -> dict:
    """
    Extrait les features linguistiques : richesse lexicale,
    longueur, top mots.
    Returns: dict de scalaires numeriques
    """
    try:
        texte = open(chemin_txt,
            encoding='utf-8').read().lower()
        tokens = word_tokenize(texte, language=langue)

```

```

sw = set(stopwords.words(langue))
stemmer = SnowballStemmer(langue)
tokens_f = [stemmer.stem(t) for t in tokens if
    t.isalpha() and t not in sw]
freq = Counter(tokens_f)
return {
    'txt_nb_tokens' : len(tokens),
    'txt_nb_mots' : len(tokens_f),
    'txt_nb_uniques' : len(set(tokens_f)),
    'txt_richesse' : len(set(tokens_f)) /
        max(len(tokens_f), 1),
    'txt_mot1' : freq.most_common(1)[0][0] if
        freq else ''
}
except Exception as e:
    print(f" Erreur texte : {e}")
    return {'txt_nb_tokens': 0, 'txt_nb_mots': 0,
        'txt_nb_uniques': 0, 'txt_richesse': 0.0,
        'txt_mot1': ''}

def executer_pipeline(
    dossier: str,
    fichier_csv: str = "metadata.csv",
    sortie: str = "features_finales.csv"
) -> pd.DataFrame:
    """
    Execute le pipeline complet sur un dossier de contenu
    multimodal.

    Structure attendue du dossier :
    dossier/
        metadata.csv      (titre, categorie)
        img/001.jpg, img/002.jpg, ...
        aud/001.wav, aud/002.wav, ...
        txt/001.txt, txt/002.txt, ...

    Returns:
        DataFrame des features fusionnees (une ligne par
        entite)
    """
    df_meta = pd.read_csv(os.path.join(dossier, fichier_csv))
    records = []

    for idx, row in df_meta.iterrows():
        iid = f"{idx:03d}"
        print(f"[{iid}] {row.get('titre', iid)}")

        record = {
            'id' : iid,
            'titre' : row.get('titre', ''),
            'categorie': row.get('categorie', '')

```

```

    }

    # Features image
    chemin_img = os.path.join(dossier, 'img', f"{iid}.jpg")
    if os.path.exists(chemin_img):
        record.update(extraire_features_image(chemin_img))
        print(f"    Image OK")

    # Features audio
    chemin_aud = os.path.join(dossier, 'aud', f"{iid}.wav")
    if os.path.exists(chemin_aud):
        record.update(extraire_features_audio(chemin_aud))
        print(f"    Audio OK")

    # Features texte
    chemin_txt = os.path.join(dossier, 'txt', f"{iid}.txt")
    if os.path.exists(chemin_txt):
        record.update(extraire_features_texte(chemin_txt))
        print(f"    Texte OK")

    records.append(record)

# Fusion en DataFrame
df_features = pd.DataFrame(records)

# Normalisation des colonnes numeriques
cols_num =
    df_features.select_dtypes(include=[np.number]).columns.tolist()
for col in cols_num:
    v = df_features[col].values.astype(float)
    rng = v.max() - v.min()
    if rng > 0:
        df_features[f"{col}_norm"] = (v - v.min()) / rng

# Export
df_features.to_csv(sortie, index=False)
print(f"\nPipeline termine : {len(df_features)} entites,
      {len(df_features.columns)} features")
print(f"Exporte : {sortie}")

return df_features

# Execution
df_final = executer_pipeline("./corpus_multimodal",
                             sortie="features_finales.csv")
print(df_final.describe())

```

Chapitre 9 Conclusion

9.1 Tableau Comparatif des 5 Classes d'Outils

Classe	Use Cases Principaux	Outils Python	Sortie Typique
Tabulaire	Metadonnees, stats, nettoyage	pandas, numpy	DataFrame, ndarray
Texte	Sous-titres, OCR, corpus	nltk, re	tokens, TF-IDF
Image	Vision, detection, annotation	opencv, Pillow	ndarray BGR/RGB
Audio	Parole, musique, pipeline ASR	pydub, librosa	AudioSegment, WAV
Visualis.	Rapport, dashboard, exploration	matplotlib, seaborn	PNG, PDF, SVG

9.2 Bonnes Pratiques Generales

1. Reproduire les resultats

- Fixer la graine aleatoire : `np.random.seed(42)`
- Versionner les dependances : `pip freeze > requirements.txt`
- Structurer le projet : `data/ | src/ | outputs/ | notebooks/`

2. Valider les données en entrée

- Toujours vérifier existence du fichier avant lecture (`os.path.exists`)
- Inspecter les types, dimensions et valeurs extrêmes
- Logger les erreurs sans interrompre le pipeline

3. Normaliser avant d'apprendre

- Images : pixels dans $[0.0, 1.0]$ ou selon les normes ImageNet
- Audio : amplitude dans $[-1.0, 1.0]$, taux a 16 kHz pour l'ASR
- Texte : casse basse, ponctuation supprimée, stopwords filters
- Tableau : min-max ou z-score selon le modèle cible

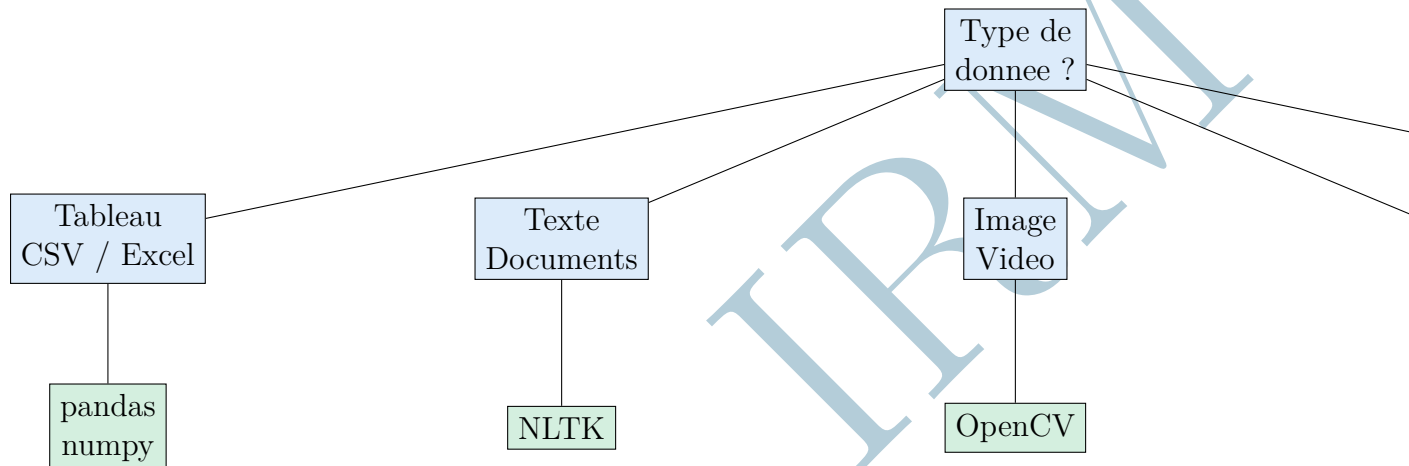
4. Gérer les erreurs gracieusement

- `try / except` pour chaque opération I/O
- Valeurs de retour par défaut (zéros, chaines vides) en cas d'échec
- Ne jamais planter un pipeline sur un fichier corrompu

5. Documenter et versionner

- Docstrings Python pour chaque fonction (Args, Returns, Note)
- Timestamps dans les noms de fichiers de sortie
- Journaux d'exécution (`logging` plutot que `print`)

9.3 Arbre de Décision



Conseil

Pour un projet IA multimodal complet, le flux recommande est :

- **Phase 1** – Acquisition : Rapports Réalité Audio / Réalité Image (voir guides dédiés)
- **Phase 2** – Traitement : Appliquer les 5 classes décrites dans ce rapport
- **Phase 3** – Fusion : Assembler les features dans un DataFrame Pandas unifié
- **Phase 4** – Modélisation : Alimenter scikit-learn, PyTorch ou TensorFlow
- **Phase 5** – Rapport : Visualiser et documenter avec Matplotlib

Chapitre 10 Bibliographie et Ressources

10.1 Ouvrages de Référence

- McKinney, W. (2022). *Python for Data Analysis*, 3rd ed. O'Reilly Media. — Référence complète pour NumPy et Pandas, par le créateur de Pandas.
- Bird, S., Klein, E. & Loper, E. (2009). *Natural Language Processing with Python*. O'Reilly Media. — Le livre fondateur de NLTK, disponible gratuitement en ligne.
- Bradski, G. & Kaehler, A. (2008). *Learning OpenCV: Computer Vision with the OpenCV Library*. O'Reilly Media. — Référence académique pour OpenCV.
- VanderPlas, J. (2016). *Python Data Science Handbook*. O'Reilly Media. — Couvre NumPy, Pandas et Matplotlib en profondeur. Disponible gratuitement sur GitHub.
- Geron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 3rd ed. O'Reilly Media. — Contexte des pipelines de traitement multimodal pour le ML.
- Muller, M. (2015). *Fundamentals of Music Processing*. Springer. — Référence pour le traitement du signal audio et les features spectrales.

10.2 Documentation Officielle

Outil	URL	Description
NumPy	numpy.org/doc	Reference API NumPy
Pandas	pandas.pydata.org/docs	Reference API Pandas
NLTK	nltk.org/book	Livre NLTK officiel
OpenCV Python	docs.opencv.org/4.x/d6/d00	Tutoriels Python
Pydub	github.com/jiaaro/pydub	README et exemples
Matplotlib	matplotlib.org/stable	Galerie et tutoriels
librosa	librosa.org	Analyse audio avancée
seaborn	seaborn.pydata.org	Stats visuelles

10.3 Tutoriels Recommandés

- **Real Python** (realpython.com) — Tutoriels de qualité professionnelle :
 - Pandas DataFrames 101
 - NLP with Python and NLTK
 - OpenCV in Python
 - Matplotlib Guide: Python Plotting

- **PyImageSearch** (pyimagesearch.com) — Tutoriels avancés en vision par ordinateur : détection d'objets, traitement en temps réel, deep learning avec OpenCV.
- **NLTK Book Online** (nltk.org/book) — Le livre NLTK original, libre d'accès, avec exemples interactifs.
- **Matplotlib Gallery** (matplotlib.org/stable/gallery) — Plus de 400 exemples de graphiques avec code source complet.

10.4 Jeux de Données pour Pratiquer

- **Kaggle** (kaggle.com/datasets) — Des milliers de datasets multimodaux, dont des compétitions intégrant images, textes et tableaux conjointement.
- **Hugging Face Datasets** (huggingface.co/datasets) — Accès unifié à des centaines de datasets audio, texte et vision, avec API Python simple.
- **Common Voice** (voice.mozilla.org) — Dataset audio multilingue de parole (100+ langues dont l'arabe et le français).
- **Open Images** (storage.googleapis.com/openimages/web) — 9 millions d'images annotées avec boîtes englobantes et segmentation.
- **Wikipedia Corpus** (dumps.wikimedia.org) — Corpus textuel massif multilingue, idéal pour l'entraînement de modèles NLP.
- **UCI Machine Learning Repository** (archive.ics.uci.edu) — Centaines de datasets tabulaires de référence pour le Machine Learning.

10.5 Aspects Légaux et Éthiques

- **Données personnelles** — Tout traitement de données identifiantes (visages, voix, textes personnels) est soumis au RGPD en Europe. L'anonymisation est obligatoire avant stockage ou partage.
- **Droits d'auteur** — Vérifiez les licences des datasets téléchargés. La licence Creative Commons CC0 ou le domaine public garantit l'usage libre.
- **Biais algorithmiques** — Les modèles entraînés sur des données déséquilibrées (par langue, genre, ethnicité) peuvent produire des résultats biaisés. Évaluez la diversité de vos datasets avant l'entraînement.
- **Traitement des mineurs** — Toute donnée impliquant des mineurs (voix, images) nécessite un consentement parental explicite et des protections supplémentaires.
- **Transparence** — Documentez la provenance de chaque dataset, les transformations appliquées et les limites connues du pipeline.

Conseil

Pour un démarrage rapide sans installation complexe :

- Données tabulaires : **pandas** + **numpy** (installation universelle)

- Texte (français) : NLTK avec `punkt_tab` et `stopwords`
- Images : `opencv-python-headless` (compatible serveurs et Colab)
- Audio : `pydub` + `ffmpeg` system (via `apt-get`)
- Visualisation : `matplotlib` avec `plt.style.use('seaborn-v0_8-whitegrid')`

Licence IRM